PROJECT HANDOFF DOCUMENT

Squareit CRM — Next.js rebuild

Everything needed for a person or another AI to understand the project and continue building it: background, decisions, how to run it, architecture, conventions, current status, the backlog, and the complete source code.

Prepared	22 October 2026
Status	Phase 1 complete, built and tested
Location (user's PC)	`C:\Users\ASHAR\Desktop\crmsquareit\crmsquareit-next`
Old system (unchanged)	`C:\Users\ASHAR\Desktop\crmsquareit` (Perfex CRM, PHP / CodeIgniter)
Source size	108 source files, about 6,365 lines (all included in Appendix A)

Contents

1. How to use this document (prompt to paste into another AI)
2. Background and decisions
3. How to run it
4. Tech stack and version gotchas
5. Architecture and coding conventions
6. Data model
7. Pages and routes
8. Features completed (Phase 1)
9. Testing done
10. Known limitations
11. Backlog: Phase 2 and beyond
12. Appendix A: complete source code

1. How to use this document

Give this PDF to another AI assistant (or developer) along with the prompt below, and fill in the last line with what you want next. The appendix holds every source file verbatim, so the assistant can see the real code instead of guessing.

```
I am continuing development of "Squareit CRM", a Next.js rebuild of an old Perfex CRM
(PHP/CodeIgniter) system.
The attached PDF is the full handoff: context, decisions, architecture, conventions,
status, backlog and the COMPLETE source code (appendix).

Rules:
1. Keep the existing stack and versions (Next.js 15.5 App Router, React 19.1, TypeScript
5.9, Tailwind CSS 4, Prisma 6 with SQLite, jose + bcryptjs). Only free / open-source tools.
2. Follow the conventions in section 5 exactly (server components + server actions,
requirePerm() in every page and action, ActionForm / ActionState pattern, logActivity,
shared UI kit in src/components/ui.tsx, Squareit brand colours).
3. Read the relevant files in the appendix before changing them; give me complete files
(not fragments) for anything you change, plus any new files, with their exact paths.
4. If you change prisma/schema.prisma, tell me to run: npm run db:push
5. The next task is: <WRITE WHAT YOU WANT HERE, e.g. "Phase 2 item 1: send
invoices/estimates by email with Nodemailer SMTP">
```

Tip: if the other AI has a small upload limit, sections 1–11 alone (about the first 10 pages) are enough for planning. Then share only the files it asks for from the appendix.

2. Background and decisions

The business (Squareit, a digital marketing agency in India, tagline "Say Digital, Way Digital") ran a stock **Perfex CRM** install: PHP/CodeIgniter with MySQL, about 45 admin modules plus add-on modules (surveys, goals, backup, exports, menu_setup, theme_style). The owner asked to *rebuild the whole thing in Next.js with all features and an admin panel, revamping the website*.

Decisions made with the owner:

- **Fresh start:** no migration of old data. The new app has its own clean schema.
- **Free tools only:** no paid SaaS, auth providers or licence keys.
- **Phased delivery:** Phase 1 = core CRM + admin panel (done). Remaining Perfex features follow in later phases (section 11).
- **Stack and design left to the builder:** Next.js + Prisma + Tailwind, with a modern SaaS look in Squareit brand colours.

Brand: logo at `public/logo.png` (white text, so it must sit on a dark background). The colours are defined as Tailwind theme tokens in `src/app/globals.css`:

Token	Hex	Used for
brand-green	#27ae57	success, paid, progress
brand-red	#e84a37	danger, overdue, delete confirm

brand-blue	#4f74c4	charts, info
brand-yellow	#fbbc15	active nav icon, highlights
primary	#3f63b5	buttons and links
sidebar	#0e1424	dark sidebar / logo backgrounds

Font: Inter (Google Fonts). The four-colour strip (green/red/blue/yellow) is a recurring brand motif: sidebar bottom, invoice header, empty states.

3. How to run it

Requires Node.js 20 or newer.

```
cd crmsquareit-next
copy .env.example .env        (Mac/Linux: cp .env.example .env)
npm install                   (postinstall runs prisma generate)
npm run setup                 (prisma db push + seed: lookups + first admin)
npm run dev                   (http://localhost:3000)
```

Script	What it does
npm run dev / build / start	Next.js dev server / production build (build also runs prisma generate) / start built app
npm run setup	Create tables and seed defaults + admin. Safe to re-run (seed only inserts into empty tables)
npm run db:push	Apply schema.prisma changes to the database
npm run db:seed	Re-run the seed
npm run db:demo	Seed + sample customers, leads, invoices, projects, tasks, ticket (only if there are no customers yet)

First login: admin@squareit.in / admin123 (override with ADMIN_EMAIL / ADMIN_PASSWORD env vars before setup). Demo staff from `db:demo`: priya@squareit.in / staff123 and rahul@squareit.in / staff123 (role "Employee"). **.env:** `DATABASE_URL="file:./dev.db"` and `AUTH_SECRET` (a long random string in production). Reset: delete `prisma/dev.db`, then `npm run setup`.

Going live (free): switch `provider` in schema.prisma to "mysql" or "postgresql", set DATABASE_URL, then setup / build / start. Options: Vercel + Neon Postgres, Vercel + TiDB Serverless (MySQL), or any Node-capable VPS/cPanel with the existing MySQL. Don't use SQLite on Vercel (non-persistent filesystem).

4. Tech stack and version gotchas

Package	Version	Notes
next	15.5.28	App Router, server actions, middleware
react / react-dom	19.1.1	useActionState, useOptimistic

typescript	5.9.x	**Must stay on 5.x.** TS 7 (native) broke Next's path-alias resolution at build time
prisma / @prisma/client	6.19.x	**Stay on 6.** Prisma 7+/8 require a new config file and driver adapters
tailwindcss + @tailwindcss/postcss	4.x	CSS-first config via @theme in globals.css (no tailwind.config.js)
jose	latest	Edge-compatible JWT for session cookie (used in middleware)
bcryptjs	latest	Password hashing
recharts	latest	Dashboard and report charts (client components, animations disabled)
lucide-react	latest	Icons
clsx, zod, server-only, tsx	latest	Class merging, (zod reserved for future validation), server-only guard, running the seed

- Next config is `next.config.mjs`, not .ts: the .ts loader failed with the newer TypeScript.
- SQLite has no enums, so all status/priority fields are **String** columns with values documented in schema comments and in `src/lib/constants.ts`. Money is **Float**, rounded with `round2()`.
- `src/global.d.ts` declares `*.css` modules for type-checking.

5. Architecture and coding conventions

Pattern: server components fetch data directly with Prisma. Mutations are **server actions** in each module's `actions.ts` ("use server"). There's no REST API and no client-side data fetching. Client components only exist where interactivity is needed.

Folder layout

```
prisma/schema.prisma        all models          prisma/seed.ts   lookups + admin (+ --demo data)
src/middleware.ts           redirects /admin/* to /login when the JWT cookie is missing or
invalid
src/lib/db.ts               Prisma singleton
src/lib/session.ts          signSession / verifySession (jose, HS256, cookie "sq_session", 7 or
30 days)
src/lib/auth.ts             getCurrentUser (React cache), requireUser, requirePerm(module,
action), requireAdmin
src/lib/permissions.ts      MODULES list, ACTIONS (view/create/edit/delete), can(user, module,
action)
src/lib/settings.ts         key/value Setting table + DEFAULT_SETTINGS, getSettings(),
saveSettings()
src/lib/sales.ts            calcTotals, parseItems, newHash, next numbers, docNumber,
refreshInvoiceStatus, isOverdue
src/lib/forms.ts            FormData helpers (str, reqStr, num, intOrNull, dateOrNull, bool,
ints) + ActionState type
src/lib/format.ts           money (en-IN Intl), dates, timeAgo, round2, formatDuration
src/lib/activity.ts         logActivity(relType, relId, description, staffId)
src/lib/constants.ts        status maps { label, tone } for invoice/estimate/project/task/ticket
+ PRIORITY, CURRENCIES, PAGE_SIZE=20
src/components/ui.tsx       Card, Badge, StatusBadge, PageHeader, Field, Table, Stat, Avatar,
Tabs, DetailRow, EmptyState, btn(), ButtonLink
src/components/client.tsx   ("use client") SubmitButton, ActionForm, ConfirmButton, FilterSelect,
```

```
SearchBox, PrintButton, CopyLink
src/components/kanban.tsx   generic HTML5 drag-and-drop board with optimistic moves (leads +
tasks)
src/components/line-items.tsx   invoice/estimate line editor (client), posts JSON in hidden input
"items"
src/components/sales-form.tsx   shared invoice/estimate form (server)    sales-document.tsx
printable doc
src/components/notes.tsx    polymorphic notes card (relType/relId)  charts.tsx  RevenueChart,
DonutChart
src/components/sidebar.tsx, topbar.tsx   admin shell                pagination.tsx  Pagination +
pageArgs()
src/app/admin/<module>/     page.tsx (list), new/page.tsx, [id]/page.tsx, [id]/edit/page.tsx,
actions.ts, form.tsx
```

Rules to follow when adding features

1. **Every page and every server action calls** `requirePerm("module", "view"|"create"|"edit"|"delete")` (or requireUser / requireAdmin). Middleware only checks that a cookie exists; real authorisation is per page and per action. A new module must be added to `MODULES` in permissions.ts and to the sidebar groups in sidebar.tsx with a `perm` key.
2. **Forms:** wrap in `<ActionForm action={fn}>` where `fn(prevState, formData): Promise<ActionState>`. Return `{ error }` to show a red message, `{ ok: true }` for success (props `savedText` and `resetOnSuccess`), or `redirect()`. For edits, bind the id: `action={updateX.bind(null, id)}`.
3. **Deletes** use `<ConfirmButton action={deleteX} hidden={{ id }} />` (click twice to confirm; no browser confirm() dialogs).
4. After a mutation that stays on the page, call `revalidatePath(path)`. Log meaningful actions with `logActivity()`; they appear on the dashboard feed and lead history.
5. **Lists** take `searchParams` (q, filters, page). Use `SearchBox`, `FilterSelect name=...`, `pageArgs()` + `<Pagination/>`. Search uses Prisma `contains`.
6. **Statuses:** add values to the maps in constants.ts and render with `<StatusBadge map={X_STATUS} value=... />`. "OVERDUE" and "EXPIRED" are *computed*, not stored.
7. **Sales maths** live only in `calcTotals()`: discount % applies before tax, tax is per line (taxRate), adjustment is added last. The client editor mirrors it for live totals. After any payment change call `refreshInvoiceStatus()` (DRAFT/CANCELLED are kept; otherwise UNPAID / PARTIAL / PAID from payments).
8. Numbers: invoices and estimates store integer `number` + `prefix`; display with `docNumber(prefix, number)` giving INV-000001. Public links use the random 32-hex `hash`.
9. Styling: Tailwind utilities plus the component classes `.input .label .card .th .td .link` from globals.css. Use the `no-print` class to hide UI when printing.
10. Keep deletes safe: a customer with invoices/estimates/projects is *deactivated*, not deleted. Staff "delete" = deactivate. You can't deactivate or demote yourself.

6. Data model (prisma/schema.prisma)

Models	Notes
Setting	key/value company + invoice settings (see DEFAULT_SETTINGS)

Role, Staff	Role.permissions = JSON {{module: [actions]}}; Staff.isAdmin bypasses all checks; Staff.active
Customer, Contact	Contact.isPrimary (one per customer)
LeadStatus, LeadSource, Lead	statuses = kanban columns (color, order, isDefault); Lead.lost / junk flags; Lead.customerId set on conversion
Note, Activity	polymorphic via relType ('lead','customer','project',...) + relId
Tax, Item, PaymentMode	lookups; Item = saved services for the line editor
Invoice, Estimate, LineItem	LineItem has invoiceId OR estimateId; totals stored; Estimate.invoiceId set when converted
Payment	belongs to Invoice; amount, mode (string), date, transactionId
ExpenseCategory, Expense	optional customer / project links, billable flag
Project	members (m2m Staff), status, FIXED/HOURLY billing + rate, progress auto-computed from tasks
Task, TaskComment, TimeEntry	assignees m2m; TimeEntry endTime null = running timer (one running per staff)
Department, Ticket, TicketReply	replies from staff (staffId) or customer (fromName)
Todo	personal to-do list per staff member

Field	Allowed values
Invoice.status	DRAFT, UNPAID, PARTIAL, PAID, CANCELLED (+ computed OVERDUE)
Estimate.status	DRAFT, SENT, ACCEPTED, DECLINED (+ computed EXPIRED)
Project.status	NOT_STARTED, IN_PROGRESS, ON_HOLD, CANCELLED, FINISHED
Task.status	NOT_STARTED, IN_PROGRESS, TESTING, AWAITING_FEEDBACK, COMPLETE
priority (Task/Ticket)	LOW, MEDIUM, HIGH, URGENT
Ticket.status	OPEN, IN_PROGRESS, ANSWERED, ON_HOLD, CLOSED

Seeded defaults: roles Manager / Sales / Employee; lead statuses New, Contacted, Qualified, Proposal sent, Customer; lead sources Website, Google Ads, Facebook, Instagram, Referral, Cold call; GST 18/12/5%; payment modes Bank transfer, UPI, Cash, Cheque, Card; expense categories; departments Support / Billing / Technical. Default currency INR (en-IN formatting).

7. Pages and routes

Route	Purpose
`/login`	Staff login
`/admin`	Dashboard
`/admin/leads`	Leads list (?view=kanban for board)
`/admin/leads/new, /[id], /[id]/edit`	Lead create / view / edit

`/admin/customers (+ new, [id]?tab=, [id]/edit)`	Customers; tabs: profile, contacts, invoices, estimates, projects, tickets, notes
`/admin/invoices (+ new, [id], [id]/edit)`	Invoices; new accepts ?customerId=&projectId=
`/admin/estimates (+ new, [id], [id]/edit)`	Estimates / quotes
`/admin/payments`	Payment register (payments are recorded from an invoice)
`/admin/expenses (+ new, [id])`	Expenses
`/admin/projects (+ new, [id]?tab=, [id]/edit)`	Projects; tabs: overview, tasks, finance, notes
`/admin/tasks (+ new, [id], [id]/edit)`	Tasks (?view=kanban, ?mine=1, ?project=)
`/admin/tickets (+ new, [id])`	Support tickets
`/admin/reports`	Reports (?year=)
`/admin/staff (+ new, [id])`	Staff
`/admin/staff/roles (+ new, [id])`	Roles & permission matrix (admins only)
`/admin/settings?tab=company|finance|items|leads|lists`	Settings
`/admin/profile`	Own profile & password
`/admin/search?q=`	Global search
`/admin/forbidden`	No-permission page
`/invoice/[hash]`	PUBLIC client invoice view (hidden while DRAFT)
`/estimate/[hash]`	PUBLIC client estimate view with Accept / Decline

8. Features completed (Phase 1)

- **Auth and roles:** login/logout, remember me, bcrypt, JWT cookie, middleware, role permission matrix (view/create/edit/delete for 12 modules), administrators, forbidden page, sidebar hides modules without view permission.
- **Dashboard:** outstanding/overdue, received this month, new leads, active projects, open tickets; 12-month payments vs expenses chart; leads pipeline donut; my open tasks; overdue invoices; personal to-do list; latest activity.
- **Leads:** list + drag-and-drop kanban; search; filters by status, source, assignee; lost/junk views; detail with notes, activity log, "log contact now", mark lost/junk; convert to customer (creates customer + primary contact, copies notes, moves to "Customer" status).
- **Customers:** CRUD, contacts manager (add/edit/delete, primary), tabs, billed/paid/outstanding stats, quick "New invoice".
- **Invoices:** line-item editor (saved items, qty/unit/rate, per-line tax, discount %, adjustment, live totals), auto-numbering, mark sent, record/delete payments with auto status, cancel/re-open/back to draft, duplicate, delete, print/PDF via browser, copy client link, public client page.
- **Estimates:** same editor, sent/accepted/declined, client accept/decline page, convert to invoice.
- **Payments** register with month/year totals; **Expenses** CRUD with categories, tax, customer/project link, billable.

- **Projects:** CRUD, members, billing type, deadline countdown, progress synced from tasks, time logged and hourly billable estimate, linked invoices/expenses, notes.
- **Tasks:** list + kanban by status, my tasks, filters, assignees, comments, start/stop timer, timesheet, quick status buttons.
- **Support tickets:** open ticket for a contact or free name/email, departments, priority, assignment, threaded replies that set status, status counters.
- **Reports:** per year: income, expenses, net profit, tax invoiced, monthly chart, top customers, expenses by category, lead conversion by source.
- **Staff:** CRUD (deactivate instead of delete), roles; **Profile:** own details and password change.
- **Settings:** company details, currency, invoice/estimate prefixes, due/expiry days, default notes/terms, bank details; editable lists for saved items, taxes, payment modes, lead statuses (colour/order), lead sources, expense categories, departments.
- **Global search** (customers, leads, invoice number, projects, tasks, tickets), "+ New" quick-create menu, responsive layout with mobile sidebar, Squareit favicon/logo.

9. Testing done

- `npm run build` passes (type-check + lint). A clean-folder install (`npm ci` then `setup` then `build`) was verified.
- Automated Playwright smoke test: 49 admin URLs all return 200 with no client-side errors.
- Automated flow test, all passing: bad-login rejection; create customer with contact; create invoice (₹13,000 + 18% GST = ₹15,340 verified); mark sent; partial payment leaves Partially paid; full payment gives PAID (verified in DB); public invoice page; estimate converted to invoice; lead converted to customer; kanban drag moved a lead to "Qualified"; task timer start/stop, comment, complete (project progress auto-updated); ticket reply; settings save; saved item added; to-do added; Employee role can't open invoices (redirected to /admin/forbidden) and sidebar hides sales modules.
- Visual checks at desktop (1440px) and phone (390px) widths.

10. Known limitations

- No email sending yet ("Mark as sent" only changes status; the client link must be copied manually).
- PDF = browser print-to-PDF (print CSS included). There's no server-generated PDF download.
- No file uploads/attachments; no customer login portal; no recurring invoices; no custom fields.
- Search uses `contains` : case-insensitive on SQLite/MySQL but case-sensitive on Postgres (add `mode: "insensitive"` if moving to Postgres).
- Detail pages (e.g. /admin/invoices/[id]) use the generic browser tab title; `generateMetadata` could be added.
- Permissions are module-wide (no "only own records" level yet). No audit of login attempts / rate limiting.
- Currency formatting uses en-IN; multi-currency is stored per document but reports sum amounts without conversion.
- Lookup deletes that are still referenced (e.g. a lead status in use) are silently ignored.

11. Backlog: Phase 2 and beyond

These are the remaining Perfex features, in a suggested order of value. All can be done with free libraries.

#	Feature	Implementation notes
1	Email: send invoice/estimate/ticket reply	Nodemailer + SMTP settings tab (host, port, user, pass, from); email templates in Setting; log sentAt; payment reminders for overdue invoices
2	Client portal	Contact login (add password + portal flag to Contact); /portal pages: invoices, estimates, projects (read-only progress), tickets (open + reply), profile
3	PDF download	@react-pdf/renderer route /api/invoice/[hash]/pdf, attach to emails
4	File attachments	Upload to /uploads (local) or free S3-compatible storage (Cloudflare R2 free tier); Attachment model polymorphic like Note
5	Proposals	Rich-text proposal with line items, client accept + sign, convert to estimate/invoice
6	Contracts	Contract types, value, start/end, client e-signature (canvas), renewal reminders
7	Credit notes + refunds	Apply credits to invoices; refund records
8	Recurring invoices / subscriptions	Recurring settings on Invoice + cron (Vercel Cron or node-cron) to generate and email
9	Knowledge base	Articles and groups, public /kb pages
10	Custom fields	CustomField definitions per module + values table; render in forms and detail pages
11	Goals, surveys, announcements, calendar	Perfex add-on equivalents; calendar of tasks, deadlines, invoices due
12	Import/export	CSV import for customers/leads; CSV export for lists; optional one-off importer from the old Perfex MySQL (tables tblclients, tblleads, tblinvoices, ...)
13	Hardening	Login rate limiting, 2FA (otplib), 'view own' permissions, audit log, backups
14	Public marketing website	If wanted: a public Squareit site (home, services, contact form that creates a Lead) in the same Next.js app under / with the CRM at /admin

Appendix A: complete source code

Every source file of the project, verbatim. Omitted: `package-lock.json` (regenerated by npm install) and the two images (`public/logo.png` , `src/app/icon.png`). `.env` is a copy of `.env.example` .

- .env.example
- .gitignore
- README.md
- next.config.mjs
- package.json
- postcss.config.mjs
- prisma/schema.prisma
- prisma/seed.ts
- src/app/admin/customers/[id]/contacts.tsx
- src/app/admin/customers/[id]/edit/page.tsx
- src/app/admin/customers/[id]/page.tsx
- src/app/admin/customers/actions.ts
- src/app/admin/customers/form.tsx
- src/app/admin/customers/new/page.tsx
- src/app/admin/customers/page.tsx
- src/app/admin/estimates/[id]/edit/page.tsx
- src/app/admin/estimates/[id]/page.tsx
- src/app/admin/estimates/actions.ts
- src/app/admin/estimates/new/page.tsx
- src/app/admin/estimates/page.tsx
- src/app/admin/expenses/[id]/page.tsx
- src/app/admin/expenses/actions.ts
- src/app/admin/expenses/form.tsx
- src/app/admin/expenses/new/page.tsx
- src/app/admin/expenses/page.tsx
- src/app/admin/forbidden/page.tsx
- src/app/admin/invoices/[id]/edit/page.tsx
- src/app/admin/invoices/[id]/page.tsx
- src/app/admin/invoices/actions.ts
- src/app/admin/invoices/new/page.tsx
- src/app/admin/invoices/page.tsx
- src/app/admin/layout.tsx
- src/app/admin/leads/[id]/edit/page.tsx
- src/app/admin/leads/[id]/page.tsx
- src/app/admin/leads/actions.ts
- src/app/admin/leads/form.tsx
- src/app/admin/leads/new/page.tsx
- src/app/admin/leads/page.tsx
- src/app/admin/page.tsx
- src/app/admin/payments/page.tsx
- src/app/admin/profile/actions.ts
- src/app/admin/profile/page.tsx
- src/app/admin/projects/[id]/edit/page.tsx
- src/app/admin/projects/[id]/page.tsx
- src/app/admin/projects/actions.ts
- src/app/admin/projects/form.tsx
- src/app/admin/projects/new/page.tsx
- src/app/admin/projects/page.tsx
- src/app/admin/reports/page.tsx
- src/app/admin/search/page.tsx
- src/app/admin/settings/actions.ts
- src/app/admin/settings/lookup.tsx
- src/app/admin/settings/page.tsx
- src/app/admin/shared-actions.ts
- src/app/admin/staff/[id]/page.tsx
- src/app/admin/staff/actions.ts
- src/app/admin/staff/form.tsx
- src/app/admin/staff/new/page.tsx
- src/app/admin/staff/page.tsx
- src/app/admin/staff/roles/[id]/page.tsx
- src/app/admin/staff/roles/new/page.tsx
- src/app/admin/staff/roles/page.tsx
- src/app/admin/staff/roles/role-form.tsx
- src/app/admin/tasks/[id]/edit/page.tsx
- src/app/admin/tasks/[id]/page.tsx
- src/app/admin/tasks/actions.ts
- src/app/admin/tasks/form.tsx
- src/app/admin/tasks/new/page.tsx
- src/app/admin/tasks/page.tsx
- src/app/admin/tickets/[id]/page.tsx
- src/app/admin/tickets/actions.ts
- src/app/admin/tickets/new/page.tsx
- src/app/admin/tickets/page.tsx
- src/app/admin/todo-actions.ts
- src/app/admin/todo-widget.tsx
- src/app/estimate/[hash]/actions.ts
- src/app/estimate/[hash]/page.tsx
- src/app/globals.css
- src/app/invoice/[hash]/page.tsx
- src/app/layout.tsx
- src/app/login/actions.ts
- src/app/login/page.tsx
- src/app/not-found.tsx
- src/app/page.tsx
- src/components/charts.tsx
- src/components/client.tsx
- src/components/kanban.tsx
- src/components/line-items.tsx
- src/components/notes.tsx
- src/components/pagination.tsx
- src/components/sales-document.tsx

- src/components/sales-form.tsx
- src/components/sidebar.tsx
- src/components/topbar.tsx
- src/components/ui.tsx
- src/global.d.ts
- src/lib/activity.ts
- src/lib/auth.ts
- src/lib/constants.ts
- src/lib/db.ts
- src/lib/format.ts
- src/lib/forms.ts
- src/lib/permissions.ts
- src/lib/sales.ts
- src/lib/session.ts
- src/lib/settings.ts
- src/middleware.ts
- tsconfig.json

.env.example

```
# SQLite file (zero setup). For MySQL: mysql://user:pass@localhost:3306/squareit_crm
DATABASE_URL="file:./dev.db"
# Long random string used to sign login sessions. Change this in production!
AUTH_SECRET="change-me-to-a-long-random-string"
```

.gitignore

```
node_modules
.next
.env
prisma/*.db
prisma/*.db-journal
next-env.d.ts
*.tsbuildinfo
```

README.md

````
# Squareit CRM (Next.js rebuild)

A modern rebuild of the old Perfex-based Squareit CRM. It runs on **Next.js 15 (App Router) + TypeScript +
Tailwind CSS + Prisma**.
Every dependency is free and open source, with no paid services or licence keys.

## Quick start (Windows / Mac / Linux)

Requires **Node.js 20 or newer** (https://nodejs.org).

```bash
cd crmsquareit-next
copy .env.example .env        # Mac/Linux: cp .env.example .env
npm install
npm run setup                 # creates the database + admin account
npm run dev                   # http://localhost:3000
```

Log in with **admin@squareit.in / admin123**, then change the password from *My profile*.
You can set your own first login before running setup with the `ADMIN_EMAIL` / `ADMIN_PASSWORD` env vars.

Want sample data to click around? Run `npm run db:demo` on an empty database.
To start over completely, delete `prisma/dev.db` and run `npm run setup` again.

## What's included (Phase 1)

| Area | Features |
|---|---|
| **Auth & security** | Email/password login (bcrypt + signed HTTP-only JWT cookie), protected `/admin`,
roles with per-module view/create/edit/delete permissions, administrators |
| **Dashboard** | Outstanding/overdue invoices, payments this month, new leads, active projects, 12-month
payments vs expenses chart, lead pipeline donut, my tasks, overdue invoices, personal to-do list, activity
feed |
| **Leads** | List + drag-and-drop **Kanban**, filters (status/source/assignee), lost/junk, notes, activity
log, one-click **convert to customer** (copies notes) |
| **Customers** | Company profile, multiple contacts (primary contact), tabs for
invoices/estimates/projects/tickets/notes, billed/paid/outstanding totals |
| **Invoices** | Line-item editor (saved items, per-line GST/tax, discount %, adjustment), auto numbering &
````

````
prefix, draft → sent → partially paid → paid, overdue detection, payments, duplicate, cancel, print/PDF,
**shareable client link** `/invoice/<hash>` |
| **Estimates** | Same editor, sent/accepted/declined/expired, **client accept/decline page**, convert to
invoice |
| **Payments / Expenses** | Payment register by mode; expenses with categories, tax, billable flag, linked
customer/project |
| **Projects** | Members, fixed/hourly billing, deadline, auto progress from tasks, time logged, linked
invoices/expenses, notes |
| **Tasks** | List + Kanban, priorities, multiple assignees, comments, **start/stop timer & timesheet** |
| **Support** | Tickets with departments, priority, assignment, threaded staff replies, status workflow |
| **Reports** | Income vs expenses by month/year, net profit, tax invoiced, top customers, expenses by
category, lead conversion by source |
| **Admin** | Staff management, roles & permission matrix, company settings, invoice/estimate defaults, bank
details, tax rates, saved items, payment modes, lead statuses (kanban columns)/sources, expense categories,
departments |
| **Extras** | Global search, "+ New" quick menu, mobile-friendly sidebar, Squareit branding |

## Going live (free options)

SQLite is fine for local use. For a hosted site, switch to MySQL or Postgres:

1. In `prisma/schema.prisma` change `provider = "sqlite"` to `"mysql"` (or `"postgresql"`).
2. Set `DATABASE_URL` in `.env`, for example `mysql://user:pass@host:3306/squareit_crm`.
3. Set a long random `AUTH_SECRET`.
4. Run `npm run setup`, then `npm run build` and `npm start`.

Free hosting combos: **Vercel + Neon (Postgres)**, **Vercel + TiDB Serverless (MySQL)**, or any VPS/cPanel
host with Node.js support plus the MySQL database you already have.
Don't use SQLite on Vercel, because its filesystem is not persistent.

## Project structure

```
prisma/schema.prisma     database models
prisma/seed.ts           default lookups + admin (+ demo data)
src/lib/                 db, auth/session, permissions, settings, sales maths, formatting
src/components/          UI kit, kanban, line-item editor, invoice document, charts
src/app/login            login page
src/app/admin/*          one folder per module (page.tsx, actions.ts, form.tsx, [id]/…)
src/app/invoice|estimate public client pages
```

## Suggested Phase 2

Client portal (customer login), proposals, contracts with e-signature, credit notes, recurring invoices and
subscriptions,
email sending (Nodemailer via SMTP, free), PDF download, file attachments, knowledge base, surveys, goals,
custom fields, CSV import/export, and a data import from the old Perfex database if you ever need it.
````

next.config.mjs

```
/** @type {import("next").NextConfig} */
const nextConfig = {};
export default nextConfig;
```

package.json

```
{
  "name": "squareit-crm",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "prisma generate && next build",
    "start": "next start",
    "db:push": "prisma db push",
    "db:seed": "tsx prisma/seed.ts",
    "db:demo": "tsx prisma/seed.ts --demo",
    "setup": "prisma db push && tsx prisma/seed.ts",
    "postinstall": "prisma generate"
  },
  "dependencies": {
```

```
    "@prisma/client": "^6.19.3",
    "bcryptjs": "^3.0.3",
    "clsx": "^2.1.1",
    "jose": "^6.2.13",
    "lucide-react": "^1.61.0",
    "next": "^15.5.28",
    "react": "^19.1.1",
    "react-dom": "^19.1.1",
    "recharts": "^3.10.1",
    "server-only": "^0.0.1",
    "zod": "^4.6.7"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4.3.3",
    "@types/bcryptjs": "^2.4.6",
    "@types/node": "^26.6.5",
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "playwright": "^1.64.1",
    "postcss": "^8.5.31",
    "prisma": "^6.19.3",
    "tailwindcss": "^4.3.3",
    "tsx": "^4.23.15",
    "typescript": "5.9"
  }
}
```

postcss.config.mjs

```
export default { plugins: { "@tailwindcss/postcss": {} } };
```

prisma/schema.prisma

```
// Squareit CRM — database schema
// Default: SQLite (zero setup). To use MySQL/Postgres change `provider`
// and DATABASE_URL in .env, then run `npm run db:push`.

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Setting {
  key   String @id
  value String
}

model Role {
  id          Int     @id @default(autoincrement())
  name        String  @unique
  permissions String  @default("{}") // JSON: { module: ["view","create","edit","delete"] }
  staff       Staff[]
}

model Staff {
  id         Int       @id @default(autoincrement())
  firstName  String
  lastName   String
  email      String    @unique
  password   String
  phone      String?
  isAdmin    Boolean   @default(false)
  active     Boolean   @default(true)
  roleId     Int?
  role       Role?     @relation(fields: [roleId], references: [id], onDelete: SetNull)
  lastLogin  DateTime?
  createdAt  DateTime  @default(now())
```

```
  leads          Lead[]          @relation("LeadAssignee")
  projects       Project[]       @relation("ProjectMembers")
  tasks          Task[]          @relation("TaskAssignees")
  tasksCreated   Task[]          @relation("TaskCreator")
  taskComments   TaskComment[]
  timeEntries    TimeEntry[]
  tickets        Ticket[]        @relation("TicketAssignee")
  ticketReplies  TicketReply[]
  notes          Note[]
  activities     Activity[]
  todos          Todo[]
}

model Customer {
  id        Int       @id @default(autoincrement())
  company   String
  vat       String?
  phone     String?
  website   String?
  address   String?
  city      String?
  state     String?
  zip       String?
  country   String?
  currency  String?
  notes     String?
  active    Boolean   @default(true)
  createdAt DateTime  @default(now())

  contacts  Contact[]
  invoices  Invoice[]
  estimates Estimate[]
  projects  Project[]
  tasks     Task[]
  tickets   Ticket[]
  expenses  Expense[]
  leads     Lead[]
}

model Contact {
  id         Int      @id @default(autoincrement())
  customerId Int
  customer   Customer @relation(fields: [customerId], references: [id], onDelete: Cascade)
  firstName  String
  lastName   String
  email      String
  phone      String?
  title      String?
  isPrimary  Boolean  @default(false)
  active     Boolean  @default(true)
  createdAt  DateTime @default(now())
  tickets    Ticket[]
}

model LeadStatus {
  id        Int     @id @default(autoincrement())
  name      String
  color     String  @default("#567bbf")
  order     Int     @default(0)
  isDefault Boolean @default(false)
  leads     Lead[]
}

model LeadSource {
  id    Int    @id @default(autoincrement())
  name  String
  leads Lead[]
}

model Lead {
  id          Int         @id @default(autoincrement())
  name        String
  company     String?
  title       String?
  email       String?
```

```
  phone       String?
  website     String?
  address     String?
  city        String?
  country     String?
  value       Float       @default(0)
  description String?
  statusId    Int
  status      LeadStatus  @relation(fields: [statusId], references: [id])
  sourceId    Int?
  source      LeadSource? @relation(fields: [sourceId], references: [id], onDelete: SetNull)
  assignedId  Int?
  assigned    Staff?      @relation("LeadAssignee", fields: [assignedId], references: [id], onDelete:
SetNull)
  customerId  Int?        // set once converted
  customer    Customer?   @relation(fields: [customerId], references: [id], onDelete: SetNull)
  lost        Boolean     @default(false)
  junk        Boolean     @default(false)
  lastContact DateTime?
  sortOrder   Int         @default(0)
  createdAt   DateTime    @default(now())
}

model Note {
  id        Int      @id @default(autoincrement())
  relType   String   // lead | customer | project ...
  relId     Int
  content   String
  staffId   Int?
  staff     Staff?   @relation(fields: [staffId], references: [id], onDelete: SetNull)
  createdAt DateTime @default(now())

  @@index([relType, relId])
}

model Activity {
  id          Int      @id @default(autoincrement())
  relType     String
  relId       Int
  description String
  staffId     Int?
  staff       Staff?   @relation(fields: [staffId], references: [id], onDelete: SetNull)
  createdAt   DateTime @default(now())

  @@index([relType, relId])
}

model Tax {
  id   Int    @id @default(autoincrement())
  name String
  rate Float
}

model Item {
  id              Int     @id @default(autoincrement())
  description     String
  longDescription String?
  rate            Float
  unit            String?
  taxRate         Float   @default(0)
}

model PaymentMode {
  id     Int     @id @default(autoincrement())
  name   String
  active Boolean @default(true)
}

model Invoice {
  id              Int        @id @default(autoincrement())
  number          Int
  prefix          String     @default("INV-")
  customerId      Int
  customer        Customer   @relation(fields: [customerId], references: [id])
  projectId       Int?
```

```
  project         Project?   @relation(fields: [projectId], references: [id], onDelete: SetNull)
  date            DateTime
  dueDate         DateTime?
  status          String     @default("DRAFT") // DRAFT UNPAID PARTIAL PAID CANCELLED
  currency        String     @default("INR")
  subtotal        Float      @default(0)
  discountPercent Float      @default(0)
  discountTotal   Float      @default(0)
  taxTotal        Float      @default(0)
  adjustment      Float      @default(0)
  total           Float      @default(0)
  clientNote      String?
  terms           String?
  hash            String     @unique
  sentAt          DateTime?
  createdAt       DateTime   @default(now())
  items           LineItem[]
  payments        Payment[]
  estimate        Estimate?
}

model Estimate {
  id              Int        @id @default(autoincrement())
  number          Int
  prefix          String     @default("EST-")
  customerId      Int
  customer        Customer   @relation(fields: [customerId], references: [id])
  projectId       Int?
  project         Project?   @relation(fields: [projectId], references: [id], onDelete: SetNull)
  date            DateTime
  expiryDate      DateTime?
  status          String     @default("DRAFT") // DRAFT SENT ACCEPTED DECLINED
  currency        String     @default("INR")
  subtotal        Float      @default(0)
  discountPercent Float      @default(0)
  discountTotal   Float      @default(0)
  taxTotal        Float      @default(0)
  adjustment      Float      @default(0)
  total           Float      @default(0)
  clientNote      String?
  terms           String?
  hash            String     @unique
  invoiceId       Int?       @unique
  invoice         Invoice?   @relation(fields: [invoiceId], references: [id], onDelete: SetNull)
  createdAt       DateTime   @default(now())
  items           LineItem[]
}

model LineItem {
  id              Int       @id @default(autoincrement())
  invoiceId       Int?
  invoice         Invoice?  @relation(fields: [invoiceId], references: [id], onDelete: Cascade)
  estimateId      Int?
  estimate        Estimate? @relation(fields: [estimateId], references: [id], onDelete: Cascade)
  description     String
  longDescription String?
  qty             Float     @default(1)
  rate            Float     @default(0)
  unit            String?
  taxRate         Float     @default(0)
  sortOrder       Int       @default(0)
}

model Payment {
  id            Int      @id @default(autoincrement())
  invoiceId     Int
  invoice       Invoice  @relation(fields: [invoiceId], references: [id], onDelete: Cascade)
  amount        Float
  mode          String
  date          DateTime
  transactionId String?
  note          String?
  createdAt     DateTime @default(now())
}
```

```
model ExpenseCategory {
  id       Int       @id @default(autoincrement())
  name     String
  expenses Expense[]
}

model Expense {
  id          Int             @id @default(autoincrement())
  name        String
  categoryId  Int
  category    ExpenseCategory @relation(fields: [categoryId], references: [id])
  amount      Float
  taxRate     Float           @default(0)
  date        DateTime
  customerId  Int?
  customer    Customer?       @relation(fields: [customerId], references: [id], onDelete: SetNull)
  projectId   Int?
  project     Project?        @relation(fields: [projectId], references: [id], onDelete: SetNull)
  paymentMode String?
  reference   String?
  note        String?
  billable    Boolean         @default(false)
  createdAt   DateTime        @default(now())
}

model Project {
  id          Int        @id @default(autoincrement())
  name        String
  customerId  Int
  customer    Customer   @relation(fields: [customerId], references: [id])
  status      String     @default("NOT_STARTED") // NOT_STARTED IN_PROGRESS ON_HOLD CANCELLED FINISHED
  billingType String     @default("FIXED")       // FIXED HOURLY
  rate        Float      @default(0)
  startDate   DateTime
  deadline    DateTime?
  description String?
  progress    Int        @default(0)
  createdAt   DateTime   @default(now())
  members     Staff[]    @relation("ProjectMembers")
  tasks       Task[]
  invoices    Invoice[]
  estimates   Estimate[]
  expenses    Expense[]
  tickets     Ticket[]
}

model Task {
  id          Int           @id @default(autoincrement())
  name        String
  description String?
  priority    String        @default("MEDIUM")      // LOW MEDIUM HIGH URGENT
  status      String        @default("NOT_STARTED") // NOT_STARTED IN_PROGRESS TESTING AWAITING_FEEDBACK
COMPLETE
  startDate   DateTime      @default(now())
  dueDate     DateTime?
  projectId   Int?
  project     Project?      @relation(fields: [projectId], references: [id], onDelete: Cascade)
  customerId  Int?
  customer    Customer?     @relation(fields: [customerId], references: [id], onDelete: SetNull)
  createdById Int?
  createdBy   Staff?        @relation("TaskCreator", fields: [createdById], references: [id], onDelete:
SetNull)
  billable    Boolean       @default(false)
  completedAt DateTime?
  sortOrder   Int           @default(0)
  createdAt   DateTime      @default(now())
  assignees   Staff[]       @relation("TaskAssignees")
  comments    TaskComment[]
  timeEntries TimeEntry[]
}

model TaskComment {
  id        Int      @id @default(autoincrement())
  taskId    Int
  task      Task     @relation(fields: [taskId], references: [id], onDelete: Cascade)
```

```
  staffId   Int?
  staff     Staff?   @relation(fields: [staffId], references: [id], onDelete: SetNull)
  content   String
  createdAt DateTime @default(now())
}

model TimeEntry {
  id        Int       @id @default(autoincrement())
  taskId    Int
  task      Task      @relation(fields: [taskId], references: [id], onDelete: Cascade)
  staffId   Int
  staff     Staff     @relation(fields: [staffId], references: [id], onDelete: Cascade)
  startTime DateTime
  endTime   DateTime?
  note      String?
}

model Department {
  id      Int      @id @default(autoincrement())
  name    String
  email   String?
  tickets Ticket[]
}

model Ticket {
  id           Int           @id @default(autoincrement())
  subject      String
  message      String
  customerId   Int?
  customer     Customer?     @relation(fields: [customerId], references: [id], onDelete: SetNull)
  contactId    Int?
  contact      Contact?      @relation(fields: [contactId], references: [id], onDelete: SetNull)
  name         String?       // for non-contact submitters
  email        String?
  departmentId Int?
  department   Department?   @relation(fields: [departmentId], references: [id], onDelete: SetNull)
  projectId    Int?
  project      Project?      @relation(fields: [projectId], references: [id], onDelete: SetNull)
  priority     String        @default("MEDIUM") // LOW MEDIUM HIGH URGENT
  status       String        @default("OPEN")   // OPEN IN_PROGRESS ANSWERED ON_HOLD CLOSED
  assignedId   Int?
  assigned     Staff?        @relation("TicketAssignee", fields: [assignedId], references: [id], onDelete:
SetNull)
  lastReply    DateTime?
  createdAt    DateTime      @default(now())
  replies      TicketReply[]
}

model TicketReply {
  id        Int      @id @default(autoincrement())
  ticketId  Int
  ticket    Ticket   @relation(fields: [ticketId], references: [id], onDelete: Cascade)
  staffId   Int?
  staff     Staff?   @relation(fields: [staffId], references: [id], onDelete: SetNull)
  fromName  String?
  message   String
  createdAt DateTime @default(now())
}

model Todo {
  id        Int      @id @default(autoincrement())
  staffId   Int
  staff     Staff    @relation(fields: [staffId], references: [id], onDelete: Cascade)
  text      String
  done      Boolean  @default(false)
  createdAt DateTime @default(now())
}
```

prisma/seed.ts

```
/* Seeds required lookup data + the first admin account.
   Run `npm run db:demo` to also add sample customers, leads, invoices etc. */
import { PrismaClient } from "@prisma/client";
```

```
import bcrypt from "bcryptjs";
import { randomBytes } from "crypto";

const db = new PrismaClient();
const hash = () => randomBytes(16).toString("hex");
const days = (n: number) => new Date(Date.now() + n * 86400000);

async function main() {
  const email = process.env.ADMIN_EMAIL || "admin@squareit.in";
  const password = process.env.ADMIN_PASSWORD || "admin123";

  if ((await db.staff.count()) === 0) {
    await db.staff.create({
      data: { firstName: "Super", lastName: "Admin", email, password: await bcrypt.hash(password, 10),
isAdmin: true },
    });
    console.log(`Admin created → ${email} / ${password}  (change this after first login!)`);
  }

  if ((await db.role.count()) === 0) {
    const all = ["view", "create", "edit", "delete"];
    await db.role.createMany({
      data: [
        { name: "Manager", permissions: JSON.stringify({ customers: all, leads: all, invoices: all,
estimates: all, payments: all, expenses: all, projects: all, tasks: all, tickets: all, reports: ["view"] })
},
        { name: "Sales", permissions: JSON.stringify({ customers: ["view", "create", "edit"], leads: all,
estimates: ["view", "create", "edit"], invoices: ["view"] }) },
        { name: "Employee", permissions: JSON.stringify({ customers: ["view"], projects: ["view"], tasks:
["view", "create", "edit"], tickets: ["view", "create", "edit"] }) },
      ],
    });
  }

  if ((await db.leadStatus.count()) === 0) {
    await db.leadStatus.createMany({
      data: [
        { name: "New", color: "#4f74c4", order: 1, isDefault: true },
        { name: "Contacted", color: "#0f9fb0", order: 2 },
        { name: "Qualified", color: "#e0a400", order: 3 },
        { name: "Proposal sent", color: "#7c5cc4", order: 4 },
        { name: "Customer", color: "#27ae57", order: 5 },
      ],
    });
  }
  if ((await db.leadSource.count()) === 0)
    await db.leadSource.createMany({ data: ["Website", "Google Ads", "Facebook", "Instagram", "Referral",
"Cold call"].map((name) => ({ name })) });
  if ((await db.tax.count()) === 0)
    await db.tax.createMany({ data: [{ name: "GST", rate: 18 }, { name: "GST", rate: 12 }, { name: "GST",
rate: 5 }] });
  if ((await db.paymentMode.count()) === 0)
    await db.paymentMode.createMany({ data: ["Bank transfer", "UPI", "Cash", "Cheque", "Card"].map((name) =>
({ name })) });
  if ((await db.expenseCategory.count()) === 0)
    await db.expenseCategory.createMany({ data: ["Software", "Advertising", "Office", "Travel", "Salaries",
"Utilities"].map((name) => ({ name })) });
  if ((await db.department.count()) === 0)
    await db.department.createMany({ data: [{ name: "Support" }, { name: "Billing" }, { name: "Technical" }]
});

  if ((process.env.SEED_DEMO === "1" || process.argv.includes("--demo")) && (await db.customer.count()) ===
0) await demo();
}

async function demo() {
  const admin = (await db.staff.findFirst({ where: { isAdmin: true } }))!;
  const role = await db.role.findFirst({ where: { name: "Employee" } });
  const pw = await bcrypt.hash("staff123", 10);
  const priya = await db.staff.create({ data: { firstName: "Priya", lastName: "Sharma", email:
"priya@squareit.in", password: pw, roleId: role?.id } });
  const rahul = await db.staff.create({ data: { firstName: "Rahul", lastName: "Verma", email:
"rahul@squareit.in", password: pw, roleId: role?.id } });

  const companies = [
```

```
    ["Sunrise Dental Clinic", "Mumbai"], ["GreenLeaf Organics", "Pune"], ["Urban Fitness Studio",
"Bengaluru"],
    ["Kaveri Textiles", "Chennai"], ["BlueWave Travels", "Goa"], ["Metro Realty", "Delhi"],
  ];
  const customers = [];
  for (const [company, city] of companies) {
    const c = await db.customer.create({
      data: {
        company, city, country: "India", phone: "+91 98" + Math.floor(10000000 + Math.random() * 89999999),
website: `https://${company.toLowerCase().replace(/[^a-z]/g, "")}.in`,
        contacts: { create: { firstName: "Contact", lastName: company.split(" ")[0], email:
`hello@${company.toLowerCase().replace(/[^a-z]/g, "")}.in`, isPrimary: true } },
      },
    });
    customers.push(c);
  }

  const statuses = await db.leadStatus.findMany();
  const sources = await db.leadSource.findMany();
  const leadNames = ["Amit Patel", "Neha Gupta", "Vikram Singh", "Sneha Iyer", "Arjun Mehta", "Kavya Nair",
"Rohan Das", "Pooja Reddy"];
  for (let i = 0; i < leadNames.length; i++)
    await db.lead.create({
      data: {
        name: leadNames[i], company: `${leadNames[i].split(" ")[1]} Enterprises`, email:
`${leadNames[i].split(" ")[0].toLowerCase()}@example.com`,
        value: (i + 1) * 15000, statusId: statuses[i % 4].id, sourceId: sources[i % sources.length].id,
assignedId: i % 2 ? priya.id : admin.id,
      },
    });

  const services = [
    ["SEO — monthly retainer", 25000], ["Google Ads management", 18000], ["Website redesign", 85000],
    ["Social media management", 20000], ["Google Business Profile optimisation", 8000],
  ] as const;
  for (const [d, r] of services) await db.item.create({ data: { description: d, rate: r, taxRate: 18 } });

  for (let i = 0; i < 10; i++) {
    const c = customers[i % customers.length];
    const lines = [services[i % 5], services[(i + 2) % 5]].map(([d, r], k) => ({ description: d, qty: 1,
rate: r, taxRate: 18, sortOrder: k }));
    const subtotal = lines.reduce((s, l) => s + l.rate, 0);
    const tax = subtotal * 0.18;
    const status = ["PAID", "PAID", "UNPAID", "PARTIAL", "DRAFT"][i % 5];
    const date = days(-i * 18);
    const inv = await db.invoice.create({
      data: {
        number: i + 1, customerId: c.id, date, dueDate: new Date(date.getTime() + 15 * 86400000), status,
        subtotal, taxTotal: tax, total: subtotal + tax, hash: hash(), clientNote: "Thank you for your
business.",
        items: { create: lines },
      },
    });
    if (status === "PAID") await db.payment.create({ data: { invoiceId: inv.id, amount: inv.total, mode:
"Bank transfer", date } });
    if (status === "PARTIAL") await db.payment.create({ data: { invoiceId: inv.id, amount:
Math.round(inv.total / 2), mode: "UPI", date } });
  }
  await db.estimate.create({
    data: {
      number: 1, customerId: customers[2].id, date: new Date(), expiryDate: days(15), status: "SENT",
subtotal: 85000, taxTotal: 15300, total: 100300, hash: hash(),
      items: { create: [{ description: "Website redesign", qty: 1, rate: 85000, taxRate: 18 }] },
    },
  });

  const proj = await db.project.create({
    data: {
      name: "Website revamp", customerId: customers[0].id, status: "IN_PROGRESS", startDate: days(-10),
deadline: days(30), progress: 40,
      description: "Full redesign of the clinic website with booking integration.", members: { connect: [{
id: admin.id }, { id: priya.id }] },
    },
  });
  await db.project.create({ data: { name: "SEO — Q4 campaign", customerId: customers[1].id, status:
```

```
"NOT_STARTED", startDate: days(2), members: { connect: [{ id: rahul.id }] } } });
  const tasks = [
    ["Collect brand assets", "COMPLETE", "LOW"], ["Wireframes for home page", "IN_PROGRESS", "HIGH"],
["Booking widget integration", "NOT_STARTED", "URGENT"],
    ["Content migration", "NOT_STARTED", "MEDIUM"], ["Client review round 1", "AWAITING_FEEDBACK", "MEDIUM"],
  ];
  for (const [name, status, priority] of tasks)
    await db.task.create({ data: { name, status, priority, projectId: proj.id, dueDate: days(7), createdById:
admin.id, assignees: { connect: [{ id: priya.id }] } } });

  const dept = await db.department.findFirst();
  await db.ticket.create({
    data: {
      subject: "Contact form not sending emails", message: "Hi, our website contact form stopped sending
emails since yesterday.", customerId: customers[0].id,
      departmentId: dept?.id, priority: "HIGH", status: "OPEN", replies: { create: [] },
    },
  });
  const cat = await db.expenseCategory.findFirst();
  await db.expense.create({ data: { name: "SEMrush subscription", categoryId: cat!.id, amount: 9999, date:
days(-3), paymentMode: "Card" } });
  console.log("Demo data added (staff logins: priya@squareit.in / staff123)");
}

main().finally(() => db.$disconnect());
```

src/app/admin/customers/[id]/contacts.tsx

```
"use client";
import { useState } from "react";
import type { Contact } from "@prisma/client";
import { ActionForm, ConfirmButton, SubmitButton } from "@/components/client";
import { Badge, Card, Field, btn } from "@/components/ui";
import type { ActionState } from "@/lib/forms";

export function Contacts({
  contacts,
  save,
  remove,
  canEdit,
}: {
  contacts: Contact[];
  save: (s: ActionState, f: FormData) => Promise<ActionState>;
  remove: (f: FormData) => Promise<void>;
  canEdit: boolean;
}) {
  const [editing, setEditing] = useState<Partial<Contact> | null>(null);
  return (
    <Card title={`Contacts (${contacts.length})`} actions={canEdit && <button className={btn("secondary",
"py-1.5 text-xs")} onClick={() => setEditing({})}>+ Add contact</button>}>
      {editing && (
        <ActionForm
          key={editing.id ?? "new"}
          action={async (s, f) => {
            const r = await save(s, f);
            if (r?.ok) setEditing(null);
            return r;
          }}
          className="grid gap-3 border-b border-slate-100 bg-slate-50/60 p-5 sm:grid-cols-3"
        >
          <input type="hidden" name="id" value={editing.id ?? ""} />
          <Field label="First name *"><input name="firstName" required defaultValue={editing.firstName}
className="input" /></Field>
          <Field label="Last name"><input name="lastName" defaultValue={editing.lastName} className="input"
/></Field>
          <Field label="Email *"><input name="email" type="email" required defaultValue={editing.email}
className="input" /></Field>
          <Field label="Phone"><input name="phone" defaultValue={editing.phone ?? ""} className="input" />
</Field>
          <Field label="Position"><input name="title" defaultValue={editing.title ?? ""} className="input" />
</Field>
          <label className="flex items-end gap-2 pb-2 text-sm text-slate-700">
            <input type="checkbox" name="isPrimary" defaultChecked={editing.isPrimary} /> Primary contact
```

```
          </label>
          <div className="flex gap-2 sm:col-span-3">
            <SubmitButton>Save contact</SubmitButton>
            <button type="button" className={btn("ghost")} onClick={() => setEditing(null)}>Cancel</button>
          </div>
        </ActionForm>
      )}
      {contacts.length === 0 && !editing ? (
        <p className="px-5 py-8 text-center text-sm text-slate-400">No contacts yet.</p>
      ) : (
        <ul className="divide-y divide-slate-100">
          {contacts.map((c) => (
            <li key={c.id} className="flex flex-wrap items-center gap-3 px-5 py-3">
              <div className="min-w-0 flex-1">
                <p className="text-sm font-medium text-slate-800">
                  {c.firstName} {c.lastName} {c.isPrimary && <Badge tone="blue">Primary</Badge>}
                </p>
                <p className="text-xs text-slate-500">
                  {[c.title, c.email, c.phone].filter(Boolean).join(" · ")}
                </p>
              </div>
              {canEdit && (
                <div className="flex gap-1">
                  <button className={btn("ghost", "px-2 py-1 text-xs")} onClick={() =>
setEditing(c)}>Edit</button>
                  <ConfirmButton action={remove} hidden={{ id: c.id }} label="Delete" variant="ghost"
className="px-2 py-1 text-xs" confirmLabel="Confirm" />
                </div>
              )}
            </li>
          ))}
        </ul>
      )}
    </Card>
  );
}
```

src/app/admin/customers/[id]/edit/page.tsx

```
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { CustomerForm } from "../../form";
import { updateCustomer } from "../../actions";

export default async function EditCustomer({ params }: { params: Promise<{ id: string }> }) {
  await requirePerm("customers", "edit");
  const { id } = await params;
  const customer = await db.customer.findUnique({ where: { id: Number(id) } });
  if (!customer) notFound();
  return (
    <div className="max-w-4xl">
      <PageHeader title={`Edit ${customer.company}`} back={{ href: `/admin/customers/${customer.id}`, label:
customer.company }} />
      <CustomerForm action={updateCustomer.bind(null, customer.id)} customer={customer} />
    </div>
  );
}
```

src/app/admin/customers/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { fmtDate, money } from "@/lib/format";
import { docNumber, isOverdue } from "@/lib/sales";
import { INVOICE_STATUS, ESTIMATE_STATUS, PROJECT_STATUS, TICKET_STATUS } from "@/lib/constants";
import { Badge, ButtonLink, Card, DetailRow, EmptyState, PageHeader, Stat, StatusBadge, Table, Tabs } from
"@/components/ui";
```

```
import { ConfirmButton } from "@/components/client";
import { Notes } from "@/components/notes";
import { Contacts } from "./contacts";
import { deleteCustomer, saveContact, deleteContact } from "../actions";

export default async function CustomerPage({ params, searchParams }: { params: Promise<{ id: string }>;
searchParams: Promise<{ tab?: string }> }) {
  const user = await requirePerm("customers");
  const { id } = await params;
  const { tab = "profile" } = await searchParams;
  const c = await db.customer.findUnique({
    where: { id: Number(id) },
    include: {
      contacts: { orderBy: [{ isPrimary: "desc" }, { firstName: "asc" }] },
      invoices: { orderBy: { date: "desc" }, include: { payments: true } },
      estimates: { orderBy: { date: "desc" } },
      projects: { orderBy: { createdAt: "desc" } },
      tickets: { orderBy: { createdAt: "desc" } },
    },
  });
  if (!c) notFound();
  const cur = c.currency || "INR";
  const billed = c.invoices.filter((i) => i.status !== "DRAFT" && i.status !== "CANCELLED").reduce((s, i) =>
s + i.total, 0);
  const paid = c.invoices.reduce((s, i) => s + i.payments.reduce((a, p) => a + p.amount, 0), 0);
  const base = `/admin/customers/${c.id}`;

  return (
    <>
      <PageHeader
        title={c.company}
        subtitle={[c.city, c.country].filter(Boolean).join(", ")}
        back={{ href: "/admin/customers", label: "Customers" }}
        actions={
          <>
            {!c.active && <Badge>Inactive</Badge>}
            {can(user, "invoices", "create") && <ButtonLink variant="secondary" href={`/admin/invoices/new?
customerId=${c.id}`}>New invoice</ButtonLink>}
            {can(user, "customers", "edit") && <ButtonLink variant="secondary" href=
{`${base}/edit`}>Edit</ButtonLink>}
            {can(user, "customers", "delete") && <ConfirmButton action={deleteCustomer} hidden={{ id: c.id }}
/>}
          </>
        }
      />
      <div className="mb-6 grid gap-4 sm:grid-cols-3">
        <Stat label="Total billed" value={money(billed, cur)} accent="blue" />
        <Stat label="Paid" value={money(paid, cur)} accent="green" />
        <Stat label="Outstanding" value={money(Math.max(0, billed - paid), cur)} accent="red" />
      </div>
      <Tabs
        base={base}
        active={tab}
        tabs={[
          { key: "profile", label: "Profile" },
          { key: "contacts", label: "Contacts", count: c.contacts.length },
          { key: "invoices", label: "Invoices", count: c.invoices.length },
          { key: "estimates", label: "Estimates", count: c.estimates.length },
          { key: "projects", label: "Projects", count: c.projects.length },
          { key: "tickets", label: "Tickets", count: c.tickets.length },
          { key: "notes", label: "Notes" },
        ]}
      />

      {tab === "profile" && (
        <div className="grid gap-6 lg:grid-cols-2">
          <Card title="Company">
            <dl className="divide-y divide-slate-100 px-5">
              <DetailRow label="Company">{c.company}</DetailRow>
              <DetailRow label="GST / VAT">{c.vat}</DetailRow>
              <DetailRow label="Phone">{c.phone}</DetailRow>
              <DetailRow label="Website">{c.website && <a href={c.website} target="_blank" className="link">
{c.website}</a>}</DetailRow>
              <DetailRow label="Currency">{cur}</DetailRow>
              <DetailRow label="Customer since">{fmtDate(c.createdAt)}</DetailRow>
```

```
            </dl>
          </Card>
          <Card title="Address">
            <dl className="divide-y divide-slate-100 px-5">
              <DetailRow label="Address">{c.address}</DetailRow>
              <DetailRow label="City">{c.city}</DetailRow>
              <DetailRow label="State">{c.state}</DetailRow>
              <DetailRow label="ZIP / PIN">{c.zip}</DetailRow>
              <DetailRow label="Country">{c.country}</DetailRow>
            </dl>
            {c.notes && <p className="whitespace-pre-wrap border-t border-slate-100 px-5 py-4 text-sm text-
slate-600">{c.notes}</p>}
          </Card>
        </div>
      )}

      {tab === "contacts" && (
        <Contacts contacts={c.contacts} save={saveContact.bind(null, c.id)} remove={deleteContact} canEdit=
{can(user, "customers", "edit")} />
      )}

      {tab === "invoices" && (
        <Card>
          {c.invoices.length === 0 ? (
            <EmptyState title="No invoices" action={<ButtonLink href={`/admin/invoices/new?
customerId=${c.id}`}>Create invoice</ButtonLink>} />
          ) : (
            <Table head={["Invoice", "Date", "Due", "Total", "Status"]}>
              {c.invoices.map((i) => (
                <tr key={i.id}>
                  <td className="td"><Link className="link" href={`/admin/invoices/${i.id}`}>
{docNumber(i.prefix, i.number)}</Link></td>
                  <td className="td">{fmtDate(i.date)}</td>
                  <td className="td">{fmtDate(i.dueDate)}</td>
                  <td className="td font-medium">{money(i.total, i.currency)}</td>
                  <td className="td"><StatusBadge map={INVOICE_STATUS} value={isOverdue(i) ? "OVERDUE" :
i.status} /></td>
                </tr>
              ))}
            </Table>
          )}
        </Card>
      )}

      {tab === "estimates" && (
        <Card>
          {c.estimates.length === 0 ? (
            <EmptyState title="No estimates" action={<ButtonLink href={`/admin/estimates/new?
customerId=${c.id}`}>Create estimate</ButtonLink>} />
          ) : (
            <Table head={["Estimate", "Date", "Expiry", "Total", "Status"]}>
              {c.estimates.map((e) => (
                <tr key={e.id}>
                  <td className="td"><Link className="link" href={`/admin/estimates/${e.id}`}>
{docNumber(e.prefix, e.number)}</Link></td>
                  <td className="td">{fmtDate(e.date)}</td>
                  <td className="td">{fmtDate(e.expiryDate)}</td>
                  <td className="td font-medium">{money(e.total, e.currency)}</td>
                  <td className="td"><StatusBadge map={ESTIMATE_STATUS} value={e.status} /></td>
                </tr>
              ))}
            </Table>
          )}
        </Card>
      )}

      {tab === "projects" && (
        <Card>
          {c.projects.length === 0 ? (
            <EmptyState title="No projects" action={<ButtonLink href={`/admin/projects/new?
customerId=${c.id}`}>Create project</ButtonLink>} />
          ) : (
            <Table head={["Project", "Start", "Deadline", "Progress", "Status"]}>
              {c.projects.map((p) => (
                <tr key={p.id}>
```

```
                  <td className="td"><Link className="link" href={`/admin/projects/${p.id}`}>{p.name}</Link>
</td>
                  <td className="td">{fmtDate(p.startDate)}</td>
                  <td className="td">{fmtDate(p.deadline)}</td>
                  <td className="td">{p.progress}%</td>
                  <td className="td"><StatusBadge map={PROJECT_STATUS} value={p.status} /></td>
                </tr>
              ))}
            </Table>
          )}
        </Card>
      )}

      {tab === "tickets" && (
        <Card>
          {c.tickets.length === 0 ? (
            <EmptyState title="No tickets" />
          ) : (
            <Table head={["#", "Subject", "Created", "Status"]}>
              {c.tickets.map((t) => (
                <tr key={t.id}>
                  <td className="td">#{t.id}</td>
                  <td className="td"><Link className="link" href={`/admin/tickets/${t.id}`}>{t.subject}
</Link></td>
                  <td className="td">{fmtDate(t.createdAt)}</td>
                  <td className="td"><StatusBadge map={TICKET_STATUS} value={t.status} /></td>
                </tr>
              ))}
            </Table>
          )}
        </Card>
      )}

      {tab === "notes" && <Notes relType="customer" relId={c.id} path={`${base}?tab=notes`} />}
    </>
  );
}
```

src/app/admin/customers/actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { requirePerm } from "@/lib/auth";
import { logActivity } from "@/lib/activity";
import { str, reqStr, bool, type ActionState } from "@/lib/forms";

function customerData(f: FormData) {
  return {
    company: reqStr(f, "company"),
    vat: str(f, "vat"),
    phone: str(f, "phone"),
    website: str(f, "website"),
    address: str(f, "address"),
    city: str(f, "city"),
    state: str(f, "state"),
    zip: str(f, "zip"),
    country: str(f, "country"),
    currency: str(f, "currency"),
    notes: str(f, "notes"),
  };
}

export async function createCustomer(_: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("customers", "create");
  const data = customerData(f);
  if (!data.company) return { error: "Company name is required." };
  const c = await db.customer.create({ data });
  const first = str(f, "contact_first");
  const email = str(f, "contact_email");
  if (first && email)
    await db.contact.create({ data: { customerId: c.id, firstName: first, lastName: reqStr(f,
```

```
"contact_last"), email, phone: str(f, "contact_phone"), isPrimary: true } });
  await logActivity("customer", c.id, `created customer ${c.company}`, user.id);
  redirect(`/admin/customers/${c.id}`);
}

export async function updateCustomer(id: number, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("customers", "edit");
  const data = { ...customerData(f), active: bool(f, "active") };
  if (!data.company) return { error: "Company name is required." };
  await db.customer.update({ where: { id }, data });
  await logActivity("customer", id, `updated customer ${data.company}`, user.id);
  redirect(`/admin/customers/${id}`);
}

export async function deleteCustomer(f: FormData) {
  const user = await requirePerm("customers", "delete");
  const id = Number(f.get("id"));
  const c = await db.customer.findUnique({ where: { id }, include: { _count: { select: { invoices: true,
estimates: true, projects: true } } } });
  if (!c) redirect("/admin/customers");
  if (c._count.invoices + c._count.estimates + c._count.projects > 0) {
    // Keep financial history: deactivate instead of deleting
    await db.customer.update({ where: { id }, data: { active: false } });
  } else {
    await db.customer.delete({ where: { id } });
  }
  await logActivity("customer", id, `deleted customer ${c.company}`, user.id);
  redirect("/admin/customers");
}

export async function saveContact(customerId: number, _: ActionState, f: FormData): Promise<ActionState> {
  await requirePerm("customers", "edit");
  const id = Number(f.get("id") || 0);
  const data = {
    firstName: reqStr(f, "firstName"),
    lastName: reqStr(f, "lastName"),
    email: reqStr(f, "email"),
    phone: str(f, "phone"),
    title: str(f, "title"),
    isPrimary: bool(f, "isPrimary"),
  };
  if (!data.firstName || !data.email) return { error: "First name and email are required." };
  if (data.isPrimary) await db.contact.updateMany({ where: { customerId }, data: { isPrimary: false } });
  if (id) await db.contact.update({ where: { id }, data });
  else await db.contact.create({ data: { ...data, customerId } });
  revalidatePath(`/admin/customers/${customerId}`);
  return { ok: true };
}

export async function deleteContact(f: FormData) {
  await requirePerm("customers", "edit");
  const id = Number(f.get("id"));
  const c = await db.contact.delete({ where: { id } });
  revalidatePath(`/admin/customers/${c.customerId}`);
}
```

src/app/admin/customers/form.tsx

```
import type { Customer } from "@prisma/client";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field } from "@/components/ui";
import { CURRENCIES } from "@/lib/constants";
import type { ActionState } from "@/lib/forms";

export function CustomerForm({ action, customer }: { action: (s: ActionState, f: FormData) =>
Promise<ActionState>; customer?: Customer }) {
  const c = customer;
  return (
    <ActionForm action={action} className="space-y-6">
      <Card title="Company details">
        <div className="grid gap-4 p-5 sm:grid-cols-2">
          <Field label="Company name *" className="sm:col-span-2">
            <input name="company" required defaultValue={c?.company} className="input" />
```

```
          </Field>
          <Field label="GST / VAT number">
            <input name="vat" defaultValue={c?.vat ?? ""} className="input" />
          </Field>
          <Field label="Phone">
            <input name="phone" defaultValue={c?.phone ?? ""} className="input" />
          </Field>
          <Field label="Website">
            <input name="website" defaultValue={c?.website ?? ""} className="input" placeholder="https://" />
          </Field>
          <Field label="Currency">
            <select name="currency" defaultValue={c?.currency ?? ""} className="input">
              <option value="">System default</option>
              {CURRENCIES.map((x) => (
                <option key={x}>{x}</option>
              ))}
            </select>
          </Field>
          <Field label="Address" className="sm:col-span-2">
            <textarea name="address" rows={2} defaultValue={c?.address ?? ""} className="input" />
          </Field>
          <Field label="City">
            <input name="city" defaultValue={c?.city ?? ""} className="input" />
          </Field>
          <Field label="State">
            <input name="state" defaultValue={c?.state ?? ""} className="input" />
          </Field>
          <Field label="ZIP / PIN">
            <input name="zip" defaultValue={c?.zip ?? ""} className="input" />
          </Field>
          <Field label="Country">
            <input name="country" defaultValue={c?.country ?? "India"} className="input" />
          </Field>
          <Field label="Internal notes" className="sm:col-span-2">
            <textarea name="notes" rows={3} defaultValue={c?.notes ?? ""} className="input" />
          </Field>
          {c && (
            <label className="flex items-center gap-2 text-sm text-slate-700">
              <input type="checkbox" name="active" defaultChecked={c.active} /> Active customer
            </label>
          )}
        </div>
      </Card>
      {!c && (
        <Card title="Primary contact (optional)">
          <div className="grid gap-4 p-5 sm:grid-cols-2">
            <Field label="First name">
              <input name="contact_first" className="input" />
            </Field>
            <Field label="Last name">
              <input name="contact_last" className="input" />
            </Field>
            <Field label="Email">
              <input name="contact_email" type="email" className="input" />
            </Field>
            <Field label="Phone">
              <input name="contact_phone" className="input" />
            </Field>
          </div>
        </Card>
      )}
      <div className="flex justify-end">
        <SubmitButton>{c ? "Save changes" : "Create customer"}</SubmitButton>
      </div>
    </ActionForm>
  );
}
```

src/app/admin/customers/new/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { PageHeader } from "@/components/ui";
import { CustomerForm } from "../form";
```

```
import { createCustomer } from "../actions";

export const metadata = { title: "New customer" };

export default async function NewCustomer() {
  await requirePerm("customers", "create");
  return (
    <div className="max-w-4xl">
      <PageHeader title="New customer" back={{ href: "/admin/customers", label: "Customers" }} />
      <CustomerForm action={createCustomer} />
    </div>
  );
}
```

src/app/admin/customers/page.tsx

```
import Link from "next/link";
import type { Prisma } from "@prisma/client";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { fmtDate } from "@/lib/format";
import { Badge, ButtonLink, Card, EmptyState, PageHeader, Table } from "@/components/ui";
import { FilterSelect, SearchBox } from "@/components/client";
import { Pagination, pageArgs } from "@/components/pagination";

export const metadata = { title: "Customers" };

export default async function CustomersPage({ searchParams }: { searchParams: Promise<Record<string, string |
undefined>> }) {
  await requirePerm("customers");
  const sp = await searchParams;
  const { page, skip, take } = pageArgs(sp.page);
  const where: Prisma.CustomerWhereInput = {};
  if (sp.q) where.OR = [{ company: { contains: sp.q } }, { phone: { contains: sp.q } }, { city: { contains:
sp.q } }, { contacts: { some: { email: { contains: sp.q } } } }];
  if (sp.active) where.active = sp.active === "1";
  const [rows, total, counts] = await Promise.all([
    db.customer.findMany({
      where,
      skip,
      take,
      orderBy: { createdAt: "desc" },
      include: { contacts: { where: { isPrimary: true }, take: 1 }, _count: { select: { projects: true,
invoices: true } } },
    }),
    db.customer.count({ where }),
    db.customer.groupBy({ by: ["active"], _count: true }),
  ]);
  const active = counts.find((c) => c.active)?._count ?? 0;
  const inactive = counts.find((c) => !c.active)?._count ?? 0;

  return (
    <>
      <PageHeader title="Customers" subtitle={`${active} active · ${inactive} inactive`} actions={<ButtonLink
href="/admin/customers/new">+ New customer</ButtonLink>} />
      <Card>
        <div className="flex flex-wrap gap-2 border-b border-slate-100 p-4">
          <SearchBox placeholder="Search company, phone, email…" />
          <FilterSelect name="active" placeholder="All customers" options={[{ value: "1", label: "Active" },
{ value: "0", label: "Inactive" }]} />
        </div>
        {rows.length === 0 ? (
          <EmptyState title="No customers found" action={<ButtonLink href="/admin/customers/new">Add
customer</ButtonLink>} />
        ) : (
          <Table head={["Company", "Primary contact", "Phone", "City", "Projects", "Invoices", "Status",
"Since"]}>
            {rows.map((c) => (
              <tr key={c.id} className="hover:bg-slate-50/60">
                <td className="td">
                  <Link href={`/admin/customers/${c.id}`} className="link">
                    {c.company}
                  </Link>
```

```
                </td>
                <td className="td">
                  {c.contacts[0] ? (
                    <>
                      <div>{c.contacts[0].firstName} {c.contacts[0].lastName}</div>
                      <div className="text-xs text-slate-500">{c.contacts[0].email}</div>
                    </>
                  ) : (
                    "—"
                  )}
                </td>
                <td className="td">{c.phone ?? "—"}</td>
                <td className="td">{c.city ?? "—"}</td>
                <td className="td">{c._count.projects}</td>
                <td className="td">{c._count.invoices}</td>
                <td className="td">{c.active ? <Badge tone="green">Active</Badge> : <Badge>Inactive</Badge>}
</td>
                <td className="td text-slate-500">{fmtDate(c.createdAt)}</td>
              </tr>
            ))}
          </Table>
        )}
        <Pagination total={total} page={page} params={sp} />
      </Card>
    </>
  );
}
```

src/app/admin/estimates/[id]/edit/page.tsx

```
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { docNumber } from "@/lib/sales";
import { PageHeader } from "@/components/ui";
import { SalesForm } from "@/components/sales-form";
import { updateEstimate } from "../../actions";

export default async function EditEstimate({ params }: { params: Promise<{ id: string }> }) {
  await requirePerm("estimates", "edit");
  const { id } = await params;
  const est = await db.estimate.findUnique({ where: { id: Number(id) }, include: { items: { orderBy: {
sortOrder: "asc" } } } });
  if (!est) notFound();
  const num = docNumber(est.prefix, est.number);
  return (
    <div className="max-w-6xl">
      <PageHeader title={`Edit ${num}`} back={{ href: `/admin/estimates/${est.id}`, label: num }} />
      <SalesForm kind="estimate" action={updateEstimate.bind(null, est.id)} submitLabel="Save estimate"
values={{ ...est, number: num, secondDate: est.expiryDate }} />
    </div>
  );
}
```

src/app/admin/estimates/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { getSettings } from "@/lib/settings";
import { docNumber } from "@/lib/sales";
import { ESTIMATE_STATUS } from "@/lib/constants";
import { ButtonLink, PageHeader, StatusBadge, btn } from "@/components/ui";
import { ConfirmButton, CopyLink, PrintButton } from "@/components/client";
import { SalesDocument } from "@/components/sales-document";
import { convertToInvoice, deleteEstimate, setEstimateStatus } from "../actions";

export default async function EstimatePage({ params }: { params: Promise<{ id: string }> }) {
  const user = await requirePerm("estimates");
  const { id } = await params;
```

```
  const est = await db.estimate.findUnique({
    where: { id: Number(id) },
    include: { items: { orderBy: { sortOrder: "asc" } }, customer: { include: { contacts: true } }, invoice:
true },
  });
  if (!est) notFound();
  const settings = await getSettings();
  const number = docNumber(est.prefix, est.number);
  const canEdit = can(user, "estimates", "edit");
  const statusBtn = (s: string, label: string, v: "primary" | "secondary" | "success" = "secondary") => (
    <form action={setEstimateStatus}>
      <input type="hidden" name="id" value={est.id} />
      <input type="hidden" name="status" value={s} />
      <button className={btn(v)}>{label}</button>
    </form>
  );
  return (
    <>
      <div className="no-print">
        <PageHeader
          title={number}
          subtitle={<Link className="link" href={`/admin/customers/${est.customerId}`}>{est.customer.company}
</Link>}
          back={{ href: "/admin/estimates", label: "Estimates" }}
          actions={
            <>
              <StatusBadge map={ESTIMATE_STATUS} value={est.status} />
              {canEdit && est.status === "DRAFT" && statusBtn("SENT", "Mark as sent", "primary")}
              {canEdit && est.status === "SENT" && statusBtn("ACCEPTED", "Mark accepted", "success")}
              {canEdit && est.status === "SENT" && statusBtn("DECLINED", "Mark declined")}
              {est.invoice ? (
                <ButtonLink variant="success" href={`/admin/invoices/${est.invoice.id}`}>View invoice
{docNumber(est.invoice.prefix, est.invoice.number)}</ButtonLink>
              ) : (
                can(user, "invoices", "create") && est.status !== "DECLINED" && (
                  <form action={convertToInvoice}>
                    <input type="hidden" name="id" value={est.id} />
                    <button className={btn("success")}>Convert to invoice</button>
                  </form>
                )
              )}
              {canEdit && !est.invoice && <ButtonLink variant="secondary" href=
{`/admin/estimates/${est.id}/edit`}>Edit</ButtonLink>}
              <CopyLink path={`/estimate/${est.hash}`} />
              <PrintButton />
              {can(user, "estimates", "delete") && <ConfirmButton action={deleteEstimate} hidden={{ id:
est.id }} />}
            </>
          }
        />
      </div>
      <div className="max-w-4xl">
        <SalesDocument settings={settings} doc={{ ...est, title: "Estimate", number, secondLabel: "Valid
until", secondDate: est.expiryDate, statusBadge: <StatusBadge map={ESTIMATE_STATUS} value={est.status} /> }}
/>
      </div>
    </>
  );
}
```

src/app/admin/estimates/actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { requirePerm } from "@/lib/auth";
import { logActivity } from "@/lib/activity";
import { getSettings } from "@/lib/settings";
import { calcTotals, docNumber, newHash, nextEstimateNumber, nextInvoiceNumber, parseItems } from
"@/lib/sales";
import { str, num, intOrNull, dateOrNull, reqStr, type ActionState } from "@/lib/forms";
```

```
function docData(f: FormData) {
  const items = parseItems(f);
  const discountPercent = num(f, "discountPercent");
  const adjustment = num(f, "adjustment");
  return {
    items,
    data: {
      customerId: intOrNull(f, "customerId") ?? 0,
      projectId: intOrNull(f, "projectId"),
      date: dateOrNull(f, "date") ?? new Date(),
      expiryDate: dateOrNull(f, "secondDate"),
      currency: reqStr(f, "currency") || "INR",
      discountPercent,
      adjustment,
      clientNote: str(f, "clientNote"),
      terms: str(f, "terms"),
      ...calcTotals(items, discountPercent, adjustment),
    },
  };
}
const itemRows = (items: ReturnType<typeof parseItems>) => items.map((i, k) => ({ ...i, sortOrder: k }));

export async function createEstimate(_: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("estimates", "create");
  const { items, data } = docData(f);
  if (!data.customerId) return { error: "Please select a customer." };
  if (!items.length) return { error: "Add at least one line item." };
  const s = await getSettings();
  const est = await db.estimate.create({
    data: { ...data, number: await nextEstimateNumber(), prefix: s.estimate_prefix, hash: newHash(), items: {
create: itemRows(items) } },
  });
  await logActivity("estimate", est.id, `created estimate ${docNumber(est.prefix, est.number)}`, user.id);
  redirect(`/admin/estimates/${est.id}`);
}

export async function updateEstimate(id: number, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("estimates", "edit");
  const { items, data } = docData(f);
  if (!data.customerId) return { error: "Please select a customer." };
  if (!items.length) return { error: "Add at least one line item." };
  await db.$transaction([db.lineItem.deleteMany({ where: { estimateId: id } }), db.estimate.update({ where: {
id }, data: { ...data, items: { create: itemRows(items) } } })]);
  await logActivity("estimate", id, `updated estimate`, user.id);
  redirect(`/admin/estimates/${id}`);
}

export async function setEstimateStatus(f: FormData) {
  const user = await requirePerm("estimates", "edit");
  const id = Number(f.get("id"));
  const status = String(f.get("status"));
  const est = await db.estimate.update({ where: { id }, data: { status } });
  await logActivity("estimate", id, `marked estimate ${docNumber(est.prefix, est.number)} as
${status.toLowerCase()}`, user.id);
  revalidatePath(`/admin/estimates/${id}`);
}

export async function convertToInvoice(f: FormData) {
  const user = await requirePerm("invoices", "create");
  const id = Number(f.get("id"));
  const est = await db.estimate.findUnique({ where: { id }, include: { items: true } });
  if (!est) redirect("/admin/estimates");
  if (est.invoiceId) redirect(`/admin/invoices/${est.invoiceId}`);
  const s = await getSettings();
  const inv = await db.invoice.create({
    data: {
      number: await nextInvoiceNumber(),
      prefix: s.invoice_prefix,
      hash: newHash(),
      customerId: est.customerId,
      projectId: est.projectId,
      date: new Date(),
      dueDate: new Date(Date.now() + Number(s.invoice_due_days || 0) * 86400000),
      currency: est.currency,
      subtotal: est.subtotal,
```

```
      discountPercent: est.discountPercent,
      discountTotal: est.discountTotal,
      taxTotal: est.taxTotal,
      adjustment: est.adjustment,
      total: est.total,
      clientNote: s.invoice_client_note,
      terms: est.terms,
      items: { create: est.items.map(({ id: _a, invoiceId: _b, estimateId: _c, ...it }) => it) },
    },
  });
  await db.estimate.update({ where: { id }, data: { invoiceId: inv.id, status: "ACCEPTED" } });
  await logActivity("estimate", id, `converted estimate ${docNumber(est.prefix, est.number)} to invoice`,
user.id);
  redirect(`/admin/invoices/${inv.id}`);
}

export async function deleteEstimate(f: FormData) {
  const user = await requirePerm("estimates", "delete");
  const id = Number(f.get("id"));
  const est = await db.estimate.delete({ where: { id } });
  await logActivity("estimate", id, `deleted estimate ${docNumber(est.prefix, est.number)}`, user.id);
  redirect("/admin/estimates");
}
```

src/app/admin/estimates/new/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { getSettings } from "@/lib/settings";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { SalesForm } from "@/components/sales-form";
import { createEstimate } from "../actions";

export const metadata = { title: "New estimate" };

export default async function NewEstimate({ searchParams }: { searchParams: Promise<{ customerId?: string }>
}) {
  await requirePerm("estimates", "create");
  const sp = await searchParams;
  const s = await getSettings();
  const customer = sp.customerId ? await db.customer.findUnique({ where: { id: Number(sp.customerId) } }) :
null;
  return (
    <div className="max-w-6xl">
      <PageHeader title="New estimate" back={{ href: "/admin/estimates", label: "Estimates" }} />
      <SalesForm
        kind="estimate"
        action={createEstimate}
        submitLabel="Create estimate"
        values={{ customerId: customer?.id, secondDate: new Date(Date.now() + Number(s.estimate_expiry_days
|| 0) * 86400000), currency: customer?.currency || s.currency }}
      />
    </div>
  );
}
```

src/app/admin/estimates/page.tsx

```
import Link from "next/link";
import type { Prisma } from "@prisma/client";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { fmtDate, money } from "@/lib/format";
import { docNumber } from "@/lib/sales";
import { ESTIMATE_STATUS } from "@/lib/constants";
import { ButtonLink, Card, EmptyState, PageHeader, StatusBadge, Table } from "@/components/ui";
import { FilterSelect, SearchBox } from "@/components/client";
import { Pagination, pageArgs } from "@/components/pagination";

export const metadata = { title: "Estimates" };

export default async function EstimatesPage({ searchParams }: { searchParams: Promise<Record<string, string |
```

```
undefined>> }) {
  await requirePerm("estimates");
  const sp = await searchParams;
  const { page, skip, take } = pageArgs(sp.page);
  const where: Prisma.EstimateWhereInput = {};
  if (sp.status === "EXPIRED") Object.assign(where, { status: { in: ["DRAFT", "SENT"] }, expiryDate: { lt:
new Date() } });
  else if (sp.status) where.status = sp.status;
  if (sp.q) {
    const n = parseInt(sp.q.replace(/\D/g, ""), 10);
    where.OR = [{ customer: { company: { contains: sp.q } } }, ...(Number.isFinite(n) ? [{ number: n }] :
[])];
  }
  const [rows, total] = await Promise.all([
    db.estimate.findMany({ where, skip, take, orderBy: [{ date: "desc" }, { number: "desc" }], include: {
customer: { select: { company: true } } } }),
    db.estimate.count({ where }),
  ]);
  const now = new Date();
  return (
    <>
      <PageHeader title="Estimates" subtitle="Quotes you send before work starts" actions={<ButtonLink
href="/admin/estimates/new">+ New estimate</ButtonLink>} />
      <Card>
        <div className="flex flex-wrap gap-2 border-b border-slate-100 p-4">
          <SearchBox placeholder="Estimate # or customer…" />
          <FilterSelect name="status" placeholder="All statuses" options=
{Object.entries(ESTIMATE_STATUS).map(([value, s]) => ({ value, label: s.label }))} />
        </div>
        {rows.length === 0 ? (
          <EmptyState title="No estimates found" action={<ButtonLink href="/admin/estimates/new">Create
estimate</ButtonLink>} />
        ) : (
          <Table head={["Estimate", "Customer", "Date", "Expiry", "Amount", "Status", "Invoice"]}>
            {rows.map((e) => (
              <tr key={e.id} className="hover:bg-slate-50/60">
                <td className="td"><Link className="link" href={`/admin/estimates/${e.id}`}>
{docNumber(e.prefix, e.number)}</Link></td>
                <td className="td">{e.customer.company}</td>
                <td className="td">{fmtDate(e.date)}</td>
                <td className="td">{fmtDate(e.expiryDate)}</td>
                <td className="td font-medium">{money(e.total, e.currency)}</td>
                <td className="td"><StatusBadge map={ESTIMATE_STATUS} value={["DRAFT",
"SENT"].includes(e.status) && e.expiryDate && e.expiryDate < now ? "EXPIRED" : e.status} /></td>
                <td className="td">{e.invoiceId ? <Link className="link" href=
{`/admin/invoices/${e.invoiceId}`}>View</Link> : "—"}</td>
              </tr>
            ))}
          </Table>
        )}
        <Pagination total={total} page={page} params={sp} />
      </Card>
    </>
  );
}
```

src/app/admin/expenses/[id]/page.tsx

```
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { ConfirmButton } from "@/components/client";
import { ExpenseForm } from "../form";
import { deleteExpense, saveExpense } from "../actions";

export default async function ExpensePage({ params }: { params: Promise<{ id: string }> }) {
  const user = await requirePerm("expenses", "edit");
  const { id } = await params;
  const e = await db.expense.findUnique({ where: { id: Number(id) } });
  if (!e) notFound();
  return (
```

```
    <div className="max-w-3xl">
      <PageHeader title={e.name} back={{ href: "/admin/expenses", label: "Expenses" }} actions={can(user,
"expenses", "delete") && <ConfirmButton action={deleteExpense} hidden={{ id: e.id }} />} />
      <ExpenseForm action={saveExpense.bind(null, e.id)} expense={e} />
    </div>
  );
}
```

src/app/admin/expenses/actions.ts

```
"use server";
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { requirePerm } from "@/lib/auth";
import { logActivity } from "@/lib/activity";
import { str, reqStr, num, intOrNull, dateOrNull, bool, type ActionState } from "@/lib/forms";

function data(f: FormData) {
  return {
    name: reqStr(f, "name"),
    categoryId: intOrNull(f, "categoryId") ?? 0,
    amount: num(f, "amount"),
    taxRate: num(f, "taxRate"),
    date: dateOrNull(f, "date") ?? new Date(),
    customerId: intOrNull(f, "customerId"),
    projectId: intOrNull(f, "projectId"),
    paymentMode: str(f, "paymentMode"),
    reference: str(f, "reference"),
    note: str(f, "note"),
    billable: bool(f, "billable"),
  };
}

export async function saveExpense(id: number | null, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("expenses", id ? "edit" : "create");
  const d = data(f);
  if (!d.name || !d.categoryId || d.amount <= 0) return { error: "Name, category and a positive amount are
required." };
  const e = id ? await db.expense.update({ where: { id }, data: d }) : await db.expense.create({ data: d });
  await logActivity("expense", e.id, `${id ? "updated" : "recorded"} expense ${e.name}`, user.id);
  redirect("/admin/expenses");
}

export async function deleteExpense(f: FormData) {
  await requirePerm("expenses", "delete");
  await db.expense.delete({ where: { id: Number(f.get("id")) } });
  redirect("/admin/expenses");
}
```

src/app/admin/expenses/form.tsx

```
import type { Expense } from "@prisma/client";
import { db } from "@/lib/db";
import { inputDate } from "@/lib/format";
import type { ActionState } from "@/lib/forms";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field } from "@/components/ui";

export async function ExpenseForm({ action, expense }: { action: (s: ActionState, f: FormData) =>
Promise<ActionState>; expense?: Expense }) {
  const [cats, customers, projects, modes, taxes] = await Promise.all([
    db.expenseCategory.findMany({ orderBy: { name: "asc" } }),
    db.customer.findMany({ where: { active: true }, orderBy: { company: "asc" }, select: { id: true, company:
true } }),
    db.project.findMany({ orderBy: { name: "asc" }, select: { id: true, name: true } }),
    db.paymentMode.findMany({ where: { active: true } }),
    db.tax.findMany(),
  ]);
  const e = expense;
  return (
    <ActionForm action={action}>
      <Card>
```

```
        <div className="grid gap-4 p-5 sm:grid-cols-2">
          <Field label="Expense name *" className="sm:col-span-2"><input name="name" required defaultValue=
{e?.name} className="input" /></Field>
          <Field label="Category *">
            <select name="categoryId" required defaultValue={e?.categoryId} className="input">
              {cats.map((c) => <option key={c.id} value={c.id}>{c.name}</option>)}
            </select>
          </Field>
          <Field label="Date"><input name="date" type="date" defaultValue={inputDate(e?.date ?? new Date())}
className="input" /></Field>
          <Field label="Amount *"><input name="amount" type="number" step="0.01" min="0" required
defaultValue={e?.amount} className="input" /></Field>
          <Field label="Tax">
            <select name="taxRate" defaultValue={e?.taxRate ?? 0} className="input">
              <option value={0}>No tax</option>
              {taxes.map((t) => <option key={t.id} value={t.rate}>{t.name} {t.rate}%</option>)}
            </select>
          </Field>
          <Field label="Customer">
            <select name="customerId" defaultValue={e?.customerId ?? ""} className="input">
              <option value="">—</option>
              {customers.map((c) => <option key={c.id} value={c.id}>{c.company}</option>)}
            </select>
          </Field>
          <Field label="Project">
            <select name="projectId" defaultValue={e?.projectId ?? ""} className="input">
              <option value="">—</option>
              {projects.map((p) => <option key={p.id} value={p.id}>{p.name}</option>)}
            </select>
          </Field>
          <Field label="Payment mode">
            <select name="paymentMode" defaultValue={e?.paymentMode ?? ""} className="input">
              <option value="">—</option>
              {modes.map((m) => <option key={m.id}>{m.name}</option>)}
            </select>
          </Field>
          <Field label="Reference #"><input name="reference" defaultValue={e?.reference ?? ""}
className="input" /></Field>
          <Field label="Note" className="sm:col-span-2"><textarea name="note" rows={3} defaultValue={e?.note
?? ""} className="input" /></Field>
          <label className="flex items-center gap-2 text-sm"><input type="checkbox" name="billable"
defaultChecked={e?.billable} /> Billable to customer</label>
        </div>
        <div className="flex justify-end border-t border-slate-100 p-4"><SubmitButton>{e ? "Save expense" :
"Record expense"}</SubmitButton></div>
      </Card>
    </ActionForm>
  );
}
```

src/app/admin/expenses/new/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { PageHeader } from "@/components/ui";
import { ExpenseForm } from "../form";
import { saveExpense } from "../actions";

export const metadata = { title: "New expense" };
export default async function NewExpense() {
  await requirePerm("expenses", "create");
  return (
    <div className="max-w-3xl">
      <PageHeader title="Record expense" back={{ href: "/admin/expenses", label: "Expenses" }} />
      <ExpenseForm action={saveExpense.bind(null, null)} />
    </div>
  );
}
```

src/app/admin/expenses/page.tsx

```
import Link from "next/link";
import type { Prisma } from "@prisma/client";
```

```
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { fmtDate, money } from "@/lib/format";
import { getSettings } from "@/lib/settings";
import { Badge, ButtonLink, Card, EmptyState, PageHeader, Stat, Table } from "@/components/ui";
import { FilterSelect, SearchBox } from "@/components/client";
import { Pagination, pageArgs } from "@/components/pagination";

export const metadata = { title: "Expenses" };

export default async function ExpensesPage({ searchParams }: { searchParams: Promise<Record<string, string |
undefined>> }) {
  await requirePerm("expenses");
  const sp = await searchParams;
  const { page, skip, take } = pageArgs(sp.page);
  const { currency } = await getSettings();
  const where: Prisma.ExpenseWhereInput = {};
  if (sp.category) where.categoryId = Number(sp.category);
  if (sp.q) where.OR = [{ name: { contains: sp.q } }, { reference: { contains: sp.q } }];
  const monthStart = new Date(new Date().getFullYear(), new Date().getMonth(), 1);
  const [rows, total, cats, month, sum] = await Promise.all([
    db.expense.findMany({ where, skip, take, orderBy: { date: "desc" }, include: { category: true, customer:
true, project: true } }),
    db.expense.count({ where }),
    db.expenseCategory.findMany(),
    db.expense.aggregate({ _sum: { amount: true }, where: { date: { gte: monthStart } } }),
    db.expense.aggregate({ _sum: { amount: true }, where }),
  ]);
  return (
    <>
      <PageHeader title="Expenses" actions={<ButtonLink href="/admin/expenses/new">+ Record
expense</ButtonLink>} />
      <div className="mb-6 grid gap-4 sm:grid-cols-2">
        <Stat label="This month" value={money(month._sum.amount ?? 0, currency)} accent="red" />
        <Stat label="Total (filtered)" value={money(sum._sum.amount ?? 0, currency)} accent="yellow" />
      </div>
      <Card>
        <div className="flex flex-wrap gap-2 border-b border-slate-100 p-4">
          <SearchBox placeholder="Search expenses…" />
          <FilterSelect name="category" placeholder="All categories" options={cats.map((c) => ({ value:
String(c.id), label: c.name }))} />
        </div>
        {rows.length === 0 ? (
          <EmptyState title="No expenses" action={<ButtonLink href="/admin/expenses/new">Record
expense</ButtonLink>} />
        ) : (
          <Table head={["Name", "Category", "Date", "Customer / project", "Mode", "Amount"]}>
            {rows.map((e) => (
              <tr key={e.id} className="hover:bg-slate-50/60">
                <td className="td"><Link className="link" href={`/admin/expenses/${e.id}`}>{e.name}</Link>
{e.billable && <Badge tone="blue">Billable</Badge>}</td>
                <td className="td">{e.category.name}</td>
                <td className="td">{fmtDate(e.date)}</td>
                <td className="td text-slate-500">{[e.customer?.company,
e.project?.name].filter(Boolean).join(" / ") || "—"}</td>
                <td className="td">{e.paymentMode ?? "—"}</td>
                <td className="td font-semibold">{money(e.amount, currency)}</td>
              </tr>
            ))}
          </Table>
        )}
        <Pagination total={total} page={page} params={sp} />
      </Card>
    </>
  );
}
```

src/app/admin/forbidden/page.tsx

```
import { ButtonLink, EmptyState, Card } from "@/components/ui";
export default function Forbidden() {
  return (
    <Card>
```

```
      <EmptyState title="You don't have access to this page" text="Ask an administrator to update your role
permissions." action={<ButtonLink href="/admin">Back to dashboard</ButtonLink>} />
    </Card>
  );
}
```

src/app/admin/invoices/[id]/edit/page.tsx

```
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { docNumber } from "@/lib/sales";
import { PageHeader } from "@/components/ui";
import { SalesForm } from "@/components/sales-form";
import { updateInvoice } from "../../actions";

export default async function EditInvoice({ params }: { params: Promise<{ id: string }> }) {
  await requirePerm("invoices", "edit");
  const { id } = await params;
  const inv = await db.invoice.findUnique({ where: { id: Number(id) }, include: { items: { orderBy: {
sortOrder: "asc" } } } });
  if (!inv) notFound();
  const num = docNumber(inv.prefix, inv.number);
  return (
    <div className="max-w-6xl">
      <PageHeader title={`Edit ${num}`} back={{ href: `/admin/invoices/${inv.id}`, label: num }} />
      <SalesForm
        kind="invoice"
        action={updateInvoice.bind(null, inv.id)}
        submitLabel="Save invoice"
        values={{ ...inv, number: num, secondDate: inv.dueDate }}
      />
    </div>
  );
}
```

src/app/admin/invoices/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { getSettings } from "@/lib/settings";
import { fmtDate, inputDate, money, round2 } from "@/lib/format";
import { docNumber, isOverdue } from "@/lib/sales";
import { INVOICE_STATUS } from "@/lib/constants";
import { ButtonLink, Card, Field, PageHeader, StatusBadge, btn } from "@/components/ui";
import { ActionForm, ConfirmButton, CopyLink, PrintButton, SubmitButton } from "@/components/client";
import { SalesDocument } from "@/components/sales-document";
import { deleteInvoice, deletePayment, duplicateInvoice, recordPayment, setInvoiceStatus } from "../actions";

export default async function InvoicePage({ params }: { params: Promise<{ id: string }> }) {
  const user = await requirePerm("invoices");
  const { id } = await params;
  const inv = await db.invoice.findUnique({
    where: { id: Number(id) },
    include: { items: { orderBy: { sortOrder: "asc" } }, customer: { include: { contacts: true } }, payments:
{ orderBy: { date: "desc" } }, project: true, estimate: true },
  });
  if (!inv) notFound();
  const [settings, modes] = await Promise.all([getSettings(), db.paymentMode.findMany({ where: { active: true
} })]);
  const number = docNumber(inv.prefix, inv.number);
  const paid = round2(inv.payments.reduce((s, p) => s + p.amount, 0));
  const due = Math.max(0, round2(inv.total - paid));
  const canEdit = can(user, "invoices", "edit");
  const status = isOverdue(inv) ? "OVERDUE" : inv.status;
  const statusForm = (s: string, label: string, variant: "primary" | "secondary" = "secondary") => (
    <form action={setInvoiceStatus}>
      <input type="hidden" name="id" value={inv.id} />
      <input type="hidden" name="status" value={s} />
```

```
      <button className={btn(variant)}>{label}</button>
    </form>
  );

  return (
    <>
      <div className="no-print">
        <PageHeader
          title={number}
          subtitle={
            <>
              <Link className="link" href={`/admin/customers/${inv.customerId}`}>{inv.customer.company}
</Link>
              {inv.project && <> · project <Link className="link" href={`/admin/projects/${inv.project.id}`}>
{inv.project.name}</Link></>}
              {inv.estimate && <> · from <Link className="link" href={`/admin/estimates/${inv.estimate.id}`}>
{docNumber(inv.estimate.prefix, inv.estimate.number)}</Link></>}
            </>
          }
          back={{ href: "/admin/invoices", label: "Invoices" }}
          actions={
            <>
              <StatusBadge map={INVOICE_STATUS} value={status} />
              {canEdit && inv.status === "DRAFT" && statusForm("UNPAID", "Mark as sent", "primary")}
              {canEdit && inv.status !== "CANCELLED" && inv.status !== "PAID" && <ButtonLink
variant="secondary" href={`/admin/invoices/${inv.id}/edit`}>Edit</ButtonLink>}
              <CopyLink path={`/invoice/${inv.hash}`} />
              <ButtonLink variant="secondary" href={`/invoice/${inv.hash}`}>Client view</ButtonLink>
              <PrintButton />
            </>
          }
        />
      </div>
      <div className="grid gap-6 xl:grid-cols-3">
        <div className="xl:col-span-2">
          <SalesDocument settings={settings} doc={{ ...inv, title: "Invoice", number, secondLabel: "Due
date", secondDate: inv.dueDate, statusBadge: <StatusBadge map={INVOICE_STATUS} value={status} /> }} />
        </div>
        <div className="no-print space-y-6">
          <Card title="Summary">
            <dl className="space-y-2 p-5 text-sm">
              <div className="flex justify-between"><dt className="text-slate-500">Total</dt><dd
className="font-semibold">{money(inv.total, inv.currency)}</dd></div>
              <div className="flex justify-between"><dt className="text-slate-500">Paid</dt><dd
className="font-semibold text-brand-green">{money(paid, inv.currency)}</dd></div>
              <div className="flex justify-between border-t border-slate-100 pt-2"><dt className="text-slate-
500">Amount due</dt><dd className="text-lg font-bold text-brand-red">{money(due, inv.currency)}</dd></div>
            </dl>
          </Card>

          {can(user, "payments", "create") && inv.status !== "CANCELLED" && due > 0 && (
            <Card title="Record payment">
              <ActionForm action={recordPayment.bind(null, inv.id)} resetOnSuccess className="grid gap-3 p-5
sm:grid-cols-2">
                <Field label="Amount"><input name="amount" type="number" step="0.01" min="0.01" max={due}
defaultValue={due} required className="input" /></Field>
                <Field label="Date"><input name="date" type="date" defaultValue={inputDate(new Date())}
className="input" /></Field>
                <Field label="Payment mode">
                  <select name="mode" className="input">{modes.map((m) => <option key={m.id}>{m.name}
</option>)}</select>
                </Field>
                <Field label="Transaction ID"><input name="transactionId" className="input" /></Field>
                <Field label="Note" className="sm:col-span-2"><input name="note" className="input" /></Field>
                <div className="sm:col-span-2"><SubmitButton variant="success" className="w-full">Record
payment</SubmitButton></div>
              </ActionForm>
            </Card>
          )}

          <Card title={`Payments (${inv.payments.length})`}>
            {inv.payments.length === 0 ? (
              <p className="px-5 py-6 text-center text-sm text-slate-400">No payments recorded.</p>
            ) : (
              <ul className="divide-y divide-slate-100">
```

```
                {inv.payments.map((p) => (
                  <li key={p.id} className="flex items-center justify-between gap-3 px-5 py-3 text-sm">
                    <div>
                      <p className="font-semibold">{money(p.amount, inv.currency)}</p>
                      <p className="text-xs text-slate-500">{p.mode} · {fmtDate(p.date)}{p.transactionId && `
· ${p.transactionId}`}</p>
                    </div>
                    {can(user, "payments", "delete") && <ConfirmButton action={deletePayment} hidden={{ id:
p.id }} label="x" confirmLabel="Delete?" variant="ghost" className="px-2 py-1 text-xs" />}
                  </li>
                ))}
              </ul>
            )}
          </Card>

          <Card title="More actions">
            <div className="flex flex-wrap gap-2 p-4">
              {can(user, "invoices", "create") && (
                <form action={duplicateInvoice}><input type="hidden" name="id" value={inv.id} /><button
className={btn("secondary")}>Duplicate</button></form>
              )}
              {canEdit && inv.status !== "CANCELLED" && statusForm("CANCELLED", "Cancel invoice")}
              {canEdit && inv.status === "CANCELLED" && statusForm("UNPAID", "Re-open invoice")}
              {canEdit && inv.status !== "DRAFT" && paid === 0 && inv.status !== "CANCELLED" &&
statusForm("DRAFT", "Back to draft")}
              {can(user, "invoices", "delete") && <ConfirmButton action={deleteInvoice} hidden={{ id: inv.id
}} label="Delete invoice" />}
            </div>
            {inv.sentAt && <p className="px-5 pb-4 text-xs text-slate-500">Sent on {fmtDate(inv.sentAt)}</p>}
          </Card>
        </div>
      </div>
    </>
  );
}
```

src/app/admin/invoices/actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { requirePerm } from "@/lib/auth";
import { logActivity } from "@/lib/activity";
import { getSettings } from "@/lib/settings";
import { calcTotals, docNumber, newHash, nextInvoiceNumber, parseItems, refreshInvoiceStatus } from
"@/lib/sales";
import { str, num, intOrNull, dateOrNull, reqStr, type ActionState } from "@/lib/forms";

function docData(f: FormData) {
  const items = parseItems(f);
  const discountPercent = num(f, "discountPercent");
  const adjustment = num(f, "adjustment");
  return {
    items,
    data: {
      customerId: intOrNull(f, "customerId") ?? 0,
      projectId: intOrNull(f, "projectId"),
      date: dateOrNull(f, "date") ?? new Date(),
      dueDate: dateOrNull(f, "secondDate"),
      currency: reqStr(f, "currency") || "INR",
      discountPercent,
      adjustment,
      clientNote: str(f, "clientNote"),
      terms: str(f, "terms"),
      ...calcTotals(items, discountPercent, adjustment),
    },
  };
}
const itemRows = (items: ReturnType<typeof parseItems>) => items.map((i, k) => ({ ...i, sortOrder: k }));

export async function createInvoice(_: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("invoices", "create");
```

```
  const { items, data } = docData(f);
  if (!data.customerId) return { error: "Please select a customer." };
  if (!items.length) return { error: "Add at least one line item." };
  const s = await getSettings();
  const inv = await db.invoice.create({
    data: { ...data, number: await nextInvoiceNumber(), prefix: s.invoice_prefix, hash: newHash(), status:
"DRAFT", items: { create: itemRows(items) } },
  });
  await logActivity("invoice", inv.id, `created invoice ${docNumber(inv.prefix, inv.number)}`, user.id);
  redirect(`/admin/invoices/${inv.id}`);
}

export async function updateInvoice(id: number, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("invoices", "edit");
  const { items, data } = docData(f);
  if (!data.customerId) return { error: "Please select a customer." };
  if (!items.length) return { error: "Add at least one line item." };
  await db.$transaction([db.lineItem.deleteMany({ where: { invoiceId: id } }), db.invoice.update({ where: {
id }, data: { ...data, items: { create: itemRows(items) } } })]);
  await refreshInvoiceStatus(id);
  await logActivity("invoice", id, `updated invoice`, user.id);
  redirect(`/admin/invoices/${id}`);
}

export async function setInvoiceStatus(f: FormData) {
  const user = await requirePerm("invoices", "edit");
  const id = Number(f.get("id"));
  const status = String(f.get("status"));
  const data: { status: string; sentAt?: Date } = { status };
  if (status === "UNPAID") data.sentAt = new Date();
  const inv = await db.invoice.update({ where: { id }, data });
  if (status === "UNPAID") await refreshInvoiceStatus(id);
  await logActivity("invoice", id, `marked invoice ${docNumber(inv.prefix, inv.number)} as
${status.toLowerCase()}`, user.id);
  revalidatePath(`/admin/invoices/${id}`);
}

export async function recordPayment(invoiceId: number, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("payments", "create");
  const amount = num(f, "amount");
  if (amount <= 0) return { error: "Amount must be greater than zero." };
  const inv = await db.invoice.findUnique({ where: { id: invoiceId } });
  if (!inv) return { error: "Invoice not found." };
  if (inv.status === "DRAFT") await db.invoice.update({ where: { id: invoiceId }, data: { status: "UNPAID" }
});
  await db.payment.create({
    data: { invoiceId, amount, mode: reqStr(f, "mode") || "Bank transfer", date: dateOrNull(f, "date") ?? new
Date(), transactionId: str(f, "transactionId"), note: str(f, "note") },
  });
  await refreshInvoiceStatus(invoiceId);
  await logActivity("invoice", invoiceId, `recorded payment of ${amount} on ${docNumber(inv.prefix,
inv.number)}`, user.id);
  revalidatePath(`/admin/invoices/${invoiceId}`);
  return { ok: true };
}

export async function deletePayment(f: FormData) {
  await requirePerm("payments", "delete");
  const id = Number(f.get("id"));
  const p = await db.payment.delete({ where: { id } });
  await refreshInvoiceStatus(p.invoiceId);
  revalidatePath(`/admin/invoices/${p.invoiceId}`);
  revalidatePath(`/admin/payments`);
}

export async function deleteInvoice(f: FormData) {
  const user = await requirePerm("invoices", "delete");
  const id = Number(f.get("id"));
  const inv = await db.invoice.delete({ where: { id } });
  await logActivity("invoice", id, `deleted invoice ${docNumber(inv.prefix, inv.number)}`, user.id);
  redirect("/admin/invoices");
}

export async function duplicateInvoice(f: FormData) {
  const user = await requirePerm("invoices", "create");
```

```
  const id = Number(f.get("id"));
  const src = await db.invoice.findUnique({ where: { id }, include: { items: true } });
  if (!src) redirect("/admin/invoices");
  const { id: _i, hash: _h, number: _n, createdAt: _c, sentAt: _s, status: _st, ...rest } = src;
  const inv = await db.invoice.create({
    data: {
      ...rest,
      items: { create: src.items.map(({ id: _x, invoiceId: _y, estimateId: _z, ...it }) => it) },
      number: await nextInvoiceNumber(),
      hash: newHash(),
      status: "DRAFT",
      date: new Date(),
    },
  });
  await logActivity("invoice", inv.id, `duplicated invoice`, user.id);
  redirect(`/admin/invoices/${inv.id}`);
}
```

src/app/admin/invoices/new/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { getSettings } from "@/lib/settings";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { SalesForm } from "@/components/sales-form";
import { createInvoice } from "../actions";

export const metadata = { title: "New invoice" };

export default async function NewInvoice({ searchParams }: { searchParams: Promise<{ customerId?: string;
projectId?: string }> }) {
  await requirePerm("invoices", "create");
  const sp = await searchParams;
  const s = await getSettings();
  const customer = sp.customerId ? await db.customer.findUnique({ where: { id: Number(sp.customerId) } }) :
null;
  const due = new Date(Date.now() + Number(s.invoice_due_days || 0) * 86400000);
  return (
    <div className="max-w-6xl">
      <PageHeader title="New invoice" back={{ href: "/admin/invoices", label: "Invoices" }} />
      <SalesForm
        kind="invoice"
        action={createInvoice}
        submitLabel="Create invoice"
        values={{ customerId: customer?.id, projectId: sp.projectId ? Number(sp.projectId) : null,
secondDate: due, currency: customer?.currency || s.currency }}
      />
    </div>
  );
}
```

src/app/admin/invoices/page.tsx

```
import Link from "next/link";
import type { Prisma } from "@prisma/client";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { fmtDate, money } from "@/lib/format";
import { docNumber, isOverdue } from "@/lib/sales";
import { getSettings } from "@/lib/settings";
import { INVOICE_STATUS } from "@/lib/constants";
import { ButtonLink, Card, EmptyState, PageHeader, Stat, StatusBadge, Table } from "@/components/ui";
import { FilterSelect, SearchBox } from "@/components/client";
import { Pagination, pageArgs } from "@/components/pagination";

export const metadata = { title: "Invoices" };

export default async function InvoicesPage({ searchParams }: { searchParams: Promise<Record<string, string |
undefined>> }) {
  await requirePerm("invoices");
  const sp = await searchParams;
  const { page, skip, take } = pageArgs(sp.page);
```

```
  const { currency } = await getSettings();
  const today = new Date(new Date().toDateString());
  const where: Prisma.InvoiceWhereInput = {};
  if (sp.status === "OVERDUE") Object.assign(where, { status: { in: ["UNPAID", "PARTIAL"] }, dueDate: { lt:
today } });
  else if (sp.status) where.status = sp.status;
  if (sp.customer) where.customerId = Number(sp.customer);
  if (sp.q) {
    const n = parseInt(sp.q.replace(/\D/g, ""), 10);
    where.OR = [{ customer: { company: { contains: sp.q } } }, ...(Number.isFinite(n) ? [{ number: n }] :
[])];
  }
  const [rows, total, customers, all] = await Promise.all([
    db.invoice.findMany({ where, skip, take, orderBy: [{ date: "desc" }, { number: "desc" }], include: {
customer: { select: { company: true } }, payments: { select: { amount: true } } } }),
    db.invoice.count({ where }),
    db.customer.findMany({ orderBy: { company: "asc" }, select: { id: true, company: true } }),
    db.invoice.findMany({ where: { status: { notIn: ["DRAFT", "CANCELLED"] } }, select: { total: true,
status: true, dueDate: true, payments: { select: { amount: true } } } }),
  ]);
  const paidSum = all.reduce((s, i) => s + i.payments.reduce((a, p) => a + p.amount, 0), 0);
  const billed = all.reduce((s, i) => s + i.total, 0);
  const overdueSum = all.filter(isOverdue).reduce((s, i) => s + i.total - i.payments.reduce((a, p) => a +
p.amount, 0), 0);

  return (
    <>
      <PageHeader title="Invoices" actions={<ButtonLink href="/admin/invoices/new">+ New
invoice</ButtonLink>} />
      <div className="mb-6 grid gap-4 sm:grid-cols-3">
        <Stat label="Outstanding" value={money(billed - paidSum, currency)} accent="yellow"
href="/admin/invoices?status=UNPAID" />
        <Stat label="Overdue" value={money(overdueSum, currency)} accent="red" href="/admin/invoices?
status=OVERDUE" />
        <Stat label="Paid (all time)" value={money(paidSum, currency)} accent="green" href="/admin/invoices?
status=PAID" />
      </div>
      <Card>
        <div className="flex flex-wrap gap-2 border-b border-slate-100 p-4">
          <SearchBox placeholder="Invoice # or customer…" />
          <FilterSelect name="status" placeholder="All statuses" options=
{Object.entries(INVOICE_STATUS).map(([value, s]) => ({ value, label: s.label }))} />
          <FilterSelect name="customer" placeholder="All customers" options={customers.map((c) => ({ value:
String(c.id), label: c.company }))} />
        </div>
        {rows.length === 0 ? (
          <EmptyState title="No invoices found" action={<ButtonLink href="/admin/invoices/new">Create
invoice</ButtonLink>} />
        ) : (
          <Table head={["Invoice", "Customer", "Date", "Due date", "Amount", "Due", "Status"]}>
            {rows.map((i) => {
              const due = i.total - i.payments.reduce((a, p) => a + p.amount, 0);
              return (
                <tr key={i.id} className="hover:bg-slate-50/60">
                  <td className="td"><Link className="link" href={`/admin/invoices/${i.id}`}>
{docNumber(i.prefix, i.number)}</Link></td>
                  <td className="td">{i.customer.company}</td>
                  <td className="td">{fmtDate(i.date)}</td>
                  <td className="td">{fmtDate(i.dueDate)}</td>
                  <td className="td font-medium">{money(i.total, i.currency)}</td>
                  <td className="td">{i.status === "CANCELLED" ? "—" : money(Math.max(0, due), i.currency)}
</td>
                  <td className="td"><StatusBadge map={INVOICE_STATUS} value={isOverdue(i) ? "OVERDUE" :
i.status} /></td>
                </tr>
              );
            })}
          </Table>
        )}
        <Pagination total={total} page={page} params={sp} />
      </Card>
    </>
  );
}
```

src/app/admin/layout.tsx

```
import { requireUser } from "@/lib/auth";
import { can, MODULES } from "@/lib/permissions";
import { Sidebar } from "@/components/sidebar";
import { Topbar } from "@/components/topbar";
import { db } from "@/lib/db";

export default async function AdminLayout({ children }: { children: React.ReactNode }) {
  const user = await requireUser();
  const allowed = Object.fromEntries(MODULES.map((m) => [m.key, can(user, m.key)]));
  const openTickets = allowed.tickets ? await db.ticket.count({ where: { status: "OPEN" } }) : 0;
  return (
    <div className="min-h-screen">
      <Sidebar allowed={allowed} isAdmin={user.isAdmin} badges={{ tickets: openTickets }} />
      <div className="lg:pl-64">
        <Topbar name={`${user.firstName} ${user.lastName}`} role={user.roleName ?? "Staff"} />
        <main className="mx-auto max-w-[1400px] px-4 py-6 sm:px-6 lg:px-8">{children}</main>
      </div>
    </div>
  );
}
```

src/app/admin/leads/[id]/edit/page.tsx

```
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { LeadForm } from "../../form";
import { updateLead } from "../../actions";

export default async function EditLead({ params }: { params: Promise<{ id: string }> }) {
  const user = await requirePerm("leads", "edit");
  const { id } = await params;
  const [lead, statuses, sources, staff] = await Promise.all([
    db.lead.findUnique({ where: { id: Number(id) } }),
    db.leadStatus.findMany({ orderBy: { order: "asc" } }),
    db.leadSource.findMany({ orderBy: { name: "asc" } }),
    db.staff.findMany({ where: { active: true }, select: { id: true, firstName: true, lastName: true } }),
  ]);
  if (!lead) notFound();
  return (
    <div className="max-w-5xl">
      <PageHeader title={`Edit ${lead.name}`} back={{ href: `/admin/leads/${lead.id}`, label: lead.name }} />
      <LeadForm action={updateLead.bind(null, lead.id)} lead={lead} statuses={statuses} sources={sources}
staff={staff} currentUserId={user.id} />
    </div>
  );
}
```

src/app/admin/leads/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { fmtDate, fmtDateTime, money, timeAgo } from "@/lib/format";
import { getSettings } from "@/lib/settings";
import { Badge, ButtonLink, Card, DetailRow, PageHeader, btn } from "@/components/ui";
import { ConfirmButton } from "@/components/client";
import { Notes } from "@/components/notes";
import { convertLead, deleteLead, setLeadFlag, touchLead } from "../actions";

export default async function LeadPage({ params }: { params: Promise<{ id: string }> }) {
  const user = await requirePerm("leads");
  const { id } = await params;
  const lead = await db.lead.findUnique({ where: { id: Number(id) }, include: { status: true, source: true,
assigned: true, customer: true } });
  if (!lead) notFound();
```

```
  const { currency } = await getSettings();
  const activity = await db.activity.findMany({ where: { relType: "lead", relId: lead.id }, orderBy: {
createdAt: "desc" }, take: 20, include: { staff: true } });
  const canEdit = can(user, "leads", "edit");

  return (
    <>
      <PageHeader
        title={lead.name}
        subtitle={[lead.title, lead.company].filter(Boolean).join(" at ")}
        back={{ href: "/admin/leads", label: "Leads" }}
        actions={
          <>
            <Badge color={lead.status.color}>{lead.status.name}</Badge>
            {lead.lost && <Badge tone="red">Lost</Badge>}
            {lead.junk && <Badge tone="gray">Junk</Badge>}
            {lead.customer ? (
              <ButtonLink variant="success" href={`/admin/customers/${lead.customer.id}`}>View
customer</ButtonLink>
            ) : (
              can(user, "customers", "create") && (
                <form action={convertLead}>
                  <input type="hidden" name="id" value={lead.id} />
                  <button className={btn("success")}>Convert to customer</button>
                </form>
              )
            )}
            {canEdit && <ButtonLink variant="secondary" href=
{`/admin/leads/${lead.id}/edit`}>Edit</ButtonLink>}
            {can(user, "leads", "delete") && <ConfirmButton action={deleteLead} hidden={{ id: lead.id }} />}
          </>
        }
      />
      <div className="grid gap-6 lg:grid-cols-3">
        <div className="space-y-6">
          <Card title="Lead information">
            <dl className="divide-y divide-slate-100 px-5">
              <DetailRow label="Email">{lead.email && <a className="link" href={`mailto:${lead.email}`}>
{lead.email}</a>}</DetailRow>
              <DetailRow label="Phone">{lead.phone && <a className="link" href={`tel:${lead.phone}`}>
{lead.phone}</a>}</DetailRow>
              <DetailRow label="Website">{lead.website}</DetailRow>
              <DetailRow label="Value">{lead.value ? money(lead.value, currency) : null}</DetailRow>
              <DetailRow label="Source">{lead.source?.name}</DetailRow>
              <DetailRow label="Assigned">{lead.assigned ? `${lead.assigned.firstName}
${lead.assigned.lastName}` : null}</DetailRow>
              <DetailRow label="City">{[lead.city, lead.country].filter(Boolean).join(", ")}</DetailRow>
              <DetailRow label="Address">{lead.address}</DetailRow>
              <DetailRow label="Created">{fmtDate(lead.createdAt)}</DetailRow>
              <DetailRow label="Last contact">{lead.lastContact ? fmtDateTime(lead.lastContact) : null}
</DetailRow>
            </dl>
            {lead.description && <p className="whitespace-pre-wrap border-t border-slate-100 px-5 py-4 text-
sm text-slate-600">{lead.description}</p>}
          </Card>
          {canEdit && (
            <Card title="Quick actions">
              <div className="flex flex-wrap gap-2 p-4">
                <form action={touchLead}>
                  <input type="hidden" name="id" value={lead.id} />
                  <button className={btn("secondary")}>Log contact now</button>
                </form>
                {(["lost", "junk"] as const).map((flag) => (
                  <form key={flag} action={setLeadFlag}>
                    <input type="hidden" name="id" value={lead.id} />
                    <input type="hidden" name="flag" value={flag} />
                    <input type="hidden" name="value" value={lead[flag] ? "0" : "1"} />
                    <button className={btn("secondary")}>{lead[flag] ? `Unmark ${flag}` : `Mark as ${flag}`}
</button>
                  </form>
                ))}
              </div>
            </Card>
          )}
        </div>
```

```
        <div className="space-y-6 lg:col-span-2">
          <Notes relType="lead" relId={lead.id} path={`/admin/leads/${lead.id}`} />
          <Card title="Activity log">
            <ul className="divide-y divide-slate-100">
              {activity.length === 0 && <li className="px-5 py-6 text-center text-sm text-slate-400">No
activity yet.</li>}
              {activity.map((a) => (
                <li key={a.id} className="flex justify-between gap-4 px-5 py-3 text-sm">
                  <span><span className="font-medium">{a.staff ? `${a.staff.firstName} ${a.staff.lastName}` :
"System"}</span> {a.description}</span>
                  <span className="shrink-0 text-xs text-slate-400">{timeAgo(a.createdAt)}</span>
                </li>
              ))}
            </ul>
          </Card>
          {lead.customer && (
            <p className="text-sm text-slate-500">
              Converted to <Link className="link" href={`/admin/customers/${lead.customer.id}`}>
{lead.customer.company}</Link>.
            </p>
          )}
        </div>
      </div>
    </>
  );
}
```

src/app/admin/leads/actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { requirePerm } from "@/lib/auth";
import { logActivity } from "@/lib/activity";
import { str, reqStr, num, intOrNull, type ActionState } from "@/lib/forms";

function leadData(f: FormData) {
  return {
    name: reqStr(f, "name"),
    company: str(f, "company"),
    title: str(f, "title"),
    email: str(f, "email"),
    phone: str(f, "phone"),
    website: str(f, "website"),
    address: str(f, "address"),
    city: str(f, "city"),
    country: str(f, "country"),
    value: num(f, "value"),
    description: str(f, "description"),
    statusId: intOrNull(f, "statusId") ?? 0,
    sourceId: intOrNull(f, "sourceId"),
    assignedId: intOrNull(f, "assignedId"),
  };
}

export async function createLead(_: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("leads", "create");
  const data = leadData(f);
  if (!data.name) return { error: "Lead name is required." };
  if (!data.statusId) data.statusId = (await db.leadStatus.findFirst({ orderBy: [{ isDefault: "desc" }, {
order: "asc" }] }))!.id;
  const lead = await db.lead.create({ data });
  await logActivity("lead", lead.id, `created lead ${lead.name}`, user.id);
  redirect(`/admin/leads/${lead.id}`);
}

export async function updateLead(id: number, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("leads", "edit");
  const data = leadData(f);
  if (!data.name) return { error: "Lead name is required." };
  await db.lead.update({ where: { id }, data });
  await logActivity("lead", id, `updated lead ${data.name}`, user.id);
```

```
  redirect(`/admin/leads/${id}`);
}

export async function moveLead(id: number, statusId: number) {
  const user = await requirePerm("leads", "edit");
  const lead = await db.lead.update({ where: { id }, data: { statusId }, include: { status: true } });
  await logActivity("lead", id, `moved lead ${lead.name} to ${lead.status.name}`, user.id);
  revalidatePath("/admin/leads");
}

export async function setLeadFlag(f: FormData) {
  const user = await requirePerm("leads", "edit");
  const id = Number(f.get("id"));
  const flag = String(f.get("flag")) as "lost" | "junk";
  const value = f.get("value") === "1";
  const lead = await db.lead.update({ where: { id }, data: { [flag]: value } });
  await logActivity("lead", id, `${value ? "marked" : "unmarked"} lead ${lead.name} as ${flag}`, user.id);
  revalidatePath(`/admin/leads/${id}`);
}

export async function touchLead(f: FormData) {
  await requirePerm("leads", "edit");
  const id = Number(f.get("id"));
  await db.lead.update({ where: { id }, data: { lastContact: new Date() } });
  revalidatePath(`/admin/leads/${id}`);
}

export async function convertLead(f: FormData) {
  const user = await requirePerm("customers", "create");
  const id = Number(f.get("id"));
  const lead = await db.lead.findUnique({ where: { id } });
  if (!lead) redirect("/admin/leads");
  if (lead.customerId) redirect(`/admin/customers/${lead.customerId}`);
  const [first, ...rest] = lead.name.split(" ");
  const customer = await db.customer.create({
    data: {
      company: lead.company || lead.name,
      phone: lead.phone,
      website: lead.website,
      address: lead.address,
      city: lead.city,
      country: lead.country,
      contacts: lead.email ? { create: { firstName: first, lastName: rest.join(" "), email: lead.email,
phone: lead.phone, title: lead.title, isPrimary: true } } : undefined,
    },
  });
  const customerStatus = await db.leadStatus.findFirst({ where: { name: "Customer" } });
  await db.lead.update({ where: { id }, data: { customerId: customer.id, ...(customerStatus ? { statusId:
customerStatus.id } : {}) } });
  const notes = await db.note.findMany({ where: { relType: "lead", relId: id } });
  if (notes.length)
    await db.note.createMany({ data: notes.map((n) => ({ relType: "customer", relId: customer.id, content:
n.content, staffId: n.staffId, createdAt: n.createdAt })) });
  await logActivity("lead", id, `converted lead ${lead.name} to customer`, user.id);
  redirect(`/admin/customers/${customer.id}`);
}

export async function deleteLead(f: FormData) {
  const user = await requirePerm("leads", "delete");
  const id = Number(f.get("id"));
  const lead = await db.lead.delete({ where: { id } });
  await db.note.deleteMany({ where: { relType: "lead", relId: id } });
  await logActivity("lead", id, `deleted lead ${lead.name}`, user.id);
  redirect("/admin/leads");
}
```

src/app/admin/leads/form.tsx

```
import type { Lead, LeadSource, LeadStatus, Staff } from "@prisma/client";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field } from "@/components/ui";
import type { ActionState } from "@/lib/forms";
```

```
export function LeadForm({
  action,
  lead,
  statuses,
  sources,
  staff,
  currentUserId,
}: {
  action: (s: ActionState, f: FormData) => Promise<ActionState>;
  lead?: Lead;
  statuses: LeadStatus[];
  sources: LeadSource[];
  staff: Pick<Staff, "id" | "firstName" | "lastName">[];
  currentUserId: number;
}) {
  const l = lead;
  return (
    <ActionForm action={action} className="space-y-6">
      <Card title="Lead">
        <div className="grid gap-4 p-5 sm:grid-cols-3">
          <Field label="Status">
            <select name="statusId" defaultValue={l?.statusId ?? statuses.find((s) => s.isDefault)?.id}
className="input">
              {statuses.map((s) => <option key={s.id} value={s.id}>{s.name}</option>)}
            </select>
          </Field>
          <Field label="Source">
            <select name="sourceId" defaultValue={l?.sourceId ?? ""} className="input">
              <option value="">—</option>
              {sources.map((s) => <option key={s.id} value={s.id}>{s.name}</option>)}
            </select>
          </Field>
          <Field label="Assigned to">
            <select name="assignedId" defaultValue={l?.assignedId ?? currentUserId} className="input">
              <option value="">Unassigned</option>
              {staff.map((s) => <option key={s.id} value={s.id}>{s.firstName} {s.lastName}</option>)}
            </select>
          </Field>
          <Field label="Name *"><input name="name" required defaultValue={l?.name} className="input" />
</Field>
          <Field label="Position"><input name="title" defaultValue={l?.title ?? ""} className="input" />
</Field>
          <Field label="Company"><input name="company" defaultValue={l?.company ?? ""} className="input" />
</Field>
          <Field label="Email"><input name="email" type="email" defaultValue={l?.email ?? ""}
className="input" /></Field>
          <Field label="Phone"><input name="phone" defaultValue={l?.phone ?? ""} className="input" /></Field>
          <Field label="Website"><input name="website" defaultValue={l?.website ?? ""} className="input" />
</Field>
          <Field label="Lead value"><input name="value" type="number" step="0.01" min="0" defaultValue=
{l?.value ?? ""} className="input" /></Field>
          <Field label="City"><input name="city" defaultValue={l?.city ?? ""} className="input" /></Field>
          <Field label="Country"><input name="country" defaultValue={l?.country ?? "India"} className="input"
/></Field>
          <Field label="Address" className="sm:col-span-3"><input name="address" defaultValue={l?.address ??
""} className="input" /></Field>
          <Field label="Description" className="sm:col-span-3"><textarea name="description" rows={4}
defaultValue={l?.description ?? ""} className="input" /></Field>
        </div>
      </Card>
      <div className="flex justify-end"><SubmitButton>{l ? "Save changes" : "Create lead"}</SubmitButton>
</div>
    </ActionForm>
  );
}
```

src/app/admin/leads/new/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { LeadForm } from "../form";
import { createLead } from "../actions";
```

```
export const metadata = { title: "New lead" };

export default async function NewLead() {
  const user = await requirePerm("leads", "create");
  const [statuses, sources, staff] = await Promise.all([
    db.leadStatus.findMany({ orderBy: { order: "asc" } }),
    db.leadSource.findMany({ orderBy: { name: "asc" } }),
    db.staff.findMany({ where: { active: true }, select: { id: true, firstName: true, lastName: true } }),
  ]);
  return (
    <div className="max-w-5xl">
      <PageHeader title="New lead" back={{ href: "/admin/leads", label: "Leads" }} />
      <LeadForm action={createLead} statuses={statuses} sources={sources} staff={staff} currentUserId=
{user.id} />
    </div>
  );
}
```

src/app/admin/leads/page.tsx

```
import Link from "next/link";
import type { Prisma } from "@prisma/client";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { fmtDate, money } from "@/lib/format";
import { getSettings } from "@/lib/settings";
import { Avatar, Badge, ButtonLink, Card, EmptyState, PageHeader, Table, btn, cx } from "@/components/ui";
import { FilterSelect, SearchBox } from "@/components/client";
import { Pagination, pageArgs } from "@/components/pagination";
import { Kanban } from "@/components/kanban";
import { moveLead } from "./actions";

export const metadata = { title: "Leads" };

export default async function LeadsPage({ searchParams }: { searchParams: Promise<Record<string, string |
undefined>> }) {
  await requirePerm("leads");
  const sp = await searchParams;
  const view = sp.view === "kanban" ? "kanban" : "list";
  const { currency } = await getSettings();
  const where: Prisma.LeadWhereInput = { junk: sp.junk === "1" };
  if (sp.lost !== "1" && sp.junk !== "1") where.lost = false;
  if (sp.lost === "1") where.lost = true;
  if (sp.q) where.OR = [{ name: { contains: sp.q } }, { company: { contains: sp.q } }, { email: { contains:
sp.q } }, { phone: { contains: sp.q } }];
  if (sp.status) where.statusId = Number(sp.status);
  if (sp.source) where.sourceId = Number(sp.source);
  if (sp.assigned) where.assignedId = Number(sp.assigned);

  const [statuses, sources, staff] = await Promise.all([
    db.leadStatus.findMany({ orderBy: { order: "asc" } }),
    db.leadSource.findMany({ orderBy: { name: "asc" } }),
    db.staff.findMany({ where: { active: true }, select: { id: true, firstName: true, lastName: true } }),
  ]);
  const qs = (v: string) => {
    const p = new URLSearchParams(Object.entries(sp).filter(([k, x]) => x && k !== "view" && k !== "page") as
[string, string][]);
    p.set("view", v);
    return `?${p}`;
  };

  const header = (
    <PageHeader
      title="Leads"
      subtitle="Track and convert your prospects"
      actions={
        <>
          <div className="flex rounded-lg border border-slate-300 bg-white p-0.5">
            <Link href={qs("list")} className={cx("rounded-md px-3 py-1.5 text-sm", view === "list" ? "bg-
slate-100 font-medium" : "text-slate-500")}>List</Link>
            <Link href={qs("kanban")} className={cx("rounded-md px-3 py-1.5 text-sm", view === "kanban" ?
"bg-slate-100 font-medium" : "text-slate-500")}>Kanban</Link>
```

```
          </div>
          <ButtonLink href="/admin/leads/new">+ New lead</ButtonLink>
        </>
      }
    />
  );
  const filters = (
    <div className="flex flex-wrap gap-2">
      <SearchBox placeholder="Search leads…" />
      {view === "list" && <FilterSelect name="status" placeholder="All statuses" options={statuses.map((s) =>
({ value: String(s.id), label: s.name }))} />}
      <FilterSelect name="source" placeholder="All sources" options={sources.map((s) => ({ value:
String(s.id), label: s.name }))} />
      <FilterSelect name="assigned" placeholder="Anyone" options={staff.map((s) => ({ value: String(s.id),
label: `${s.firstName} ${s.lastName}` }))} />
      <Link href={sp.lost === "1" ? "?" : "?lost=1"} className={btn(sp.lost === "1" ? "primary" :
"secondary")}>Lost</Link>
      <Link href={sp.junk === "1" ? "?" : "?junk=1"} className={btn(sp.junk === "1" ? "primary" :
"secondary")}>Junk</Link>
    </div>
  );

  if (view === "kanban") {
    const leads = await db.lead.findMany({ where, orderBy: { createdAt: "desc" }, include: { assigned: true,
source: true } });
    return (
      <>
        {header}
        <div className="mb-4">{filters}</div>
        <Kanban
          columns={statuses.map((s) => ({ id: s.id, name: s.name, color: s.color }))}
          cards={leads.map((l) => ({
            id: l.id,
            columnId: l.statusId,
            title: l.name,
            subtitle: [l.company, l.source?.name].filter(Boolean).join(" · "),
            meta: [l.value ? money(l.value, currency) : null, l.assigned ? `→ ${l.assigned.firstName}` :
null].filter(Boolean).join("  "),
            href: `/admin/leads/${l.id}`,
          }))}
          onMove={moveLead}
        />
      </>
    );
  }

  const { page, skip, take } = pageArgs(sp.page);
  const [rows, total] = await Promise.all([
    db.lead.findMany({ where, skip, take, orderBy: { createdAt: "desc" }, include: { status: true, source:
true, assigned: true } }),
    db.lead.count({ where }),
  ]);
  return (
    <>
      {header}
      <Card>
        <div className="border-b border-slate-100 p-4">{filters}</div>
        {rows.length === 0 ? (
          <EmptyState title="No leads found" action={<ButtonLink href="/admin/leads/new">Add
lead</ButtonLink>} />
        ) : (
          <Table head={["Name", "Company", "Contact", "Value", "Source", "Assigned", "Status", "Created"]}>
            {rows.map((l) => (
              <tr key={l.id} className="hover:bg-slate-50/60">
                <td className="td"><Link href={`/admin/leads/${l.id}`} className="link">{l.name}</Link></td>
                <td className="td">{l.company ?? "—"}</td>
                <td className="td text-xs">
                  <div>{l.email}</div>
                  <div className="text-slate-500">{l.phone}</div>
                </td>
                <td className="td">{l.value ? money(l.value, currency) : "—"}</td>
                <td className="td">{l.source?.name ?? "—"}</td>
                <td className="td">{l.assigned ? <Avatar name={`${l.assigned.firstName}
${l.assigned.lastName}`} size={26} /> : "—"}</td>
                <td className="td">
```

```
                  <Badge color={l.status.color}>{l.status.name}</Badge>
                  {l.lost && <Badge tone="red">Lost</Badge>}
                </td>
                <td className="td text-slate-500">{fmtDate(l.createdAt)}</td>
              </tr>
            ))}
          </Table>
        )}
        <Pagination total={total} page={page} params={sp} />
      </Card>
    </>
  );
}
```

src/app/admin/page.tsx

```
import Link from "next/link";
import { requireUser } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { money, fmtDate, timeAgo } from "@/lib/format";
import { getSettings } from "@/lib/settings";
import { Card, Stat, StatusBadge, Badge, EmptyState } from "@/components/ui";
import { RevenueChart, DonutChart } from "@/components/charts";
import { TASK_STATUS, PRIORITY } from "@/lib/constants";
import { TodoWidget } from "./todo-widget";

export const metadata = { title: "Dashboard" };

export default async function Dashboard() {
  const user = await requireUser();
  const settings = await getSettings();
  const cur = settings.currency;
  const now = new Date();
  const monthStart = new Date(now.getFullYear(), now.getMonth(), 1);
  const today = new Date(now.toDateString());
  const yearAgo = new Date(now.getFullYear(), now.getMonth() - 11, 1);

  const [openInvoices, paidThisMonth, leadsThisMonth, activeProjects, myTasks, openTickets, payments,
expenses, leadStatuses, overdue, activities, todos] =
    await Promise.all([
      db.invoice.findMany({ where: { status: { in: ["UNPAID", "PARTIAL"] } }, include: { payments: true } }),
      db.payment.aggregate({ _sum: { amount: true }, where: { date: { gte: monthStart } } }),
      db.lead.count({ where: { createdAt: { gte: monthStart } } }),
      db.project.count({ where: { status: "IN_PROGRESS" } }),
      db.task.findMany({
        where: { status: { not: "COMPLETE" }, assignees: { some: { id: user.id } } },
        orderBy: [{ dueDate: "asc" }],
        take: 6,
        include: { project: { select: { name: true } } },
      }),
      db.ticket.count({ where: { status: { in: ["OPEN", "IN_PROGRESS"] } } }),
      db.payment.findMany({ where: { date: { gte: yearAgo } }, select: { amount: true, date: true } }),
      db.expense.findMany({ where: { date: { gte: yearAgo } }, select: { amount: true, date: true } }),
      db.leadStatus.findMany({ orderBy: { order: "asc" }, include: { _count: { select: { leads: { where: {
lost: false, junk: false } } } } } }),
      db.invoice.findMany({
        where: { status: { in: ["UNPAID", "PARTIAL"] }, dueDate: { lt: today } },
        include: { customer: { select: { company: true } } },
        orderBy: { dueDate: "asc" },
        take: 5,
      }),
      db.activity.findMany({ orderBy: { createdAt: "desc" }, take: 8, include: { staff: { select: {
firstName: true, lastName: true } } } }),
      db.todo.findMany({ where: { staffId: user.id }, orderBy: [{ done: "asc" }, { createdAt: "desc" }],
take: 20 }),
    ]);

  const outstanding = openInvoices.reduce((s, i) => s + i.total - i.payments.reduce((a, p) => a + p.amount,
0), 0);
  const months = Array.from({ length: 12 }, (_, k) => {
    const d = new Date(now.getFullYear(), now.getMonth() - 11 + k, 1);
    return { key: `${d.getFullYear()}-${d.getMonth()}`, month: d.toLocaleString("en", { month: "short" }),
```

```
revenue: 0, expenses: 0 };
  });
  const idx = Object.fromEntries(months.map((m, i) => [m.key, i]));
  for (const p of payments) {
    const i = idx[`${p.date.getFullYear()}-${p.date.getMonth()}`];
    if (i !== undefined) months[i].revenue += p.amount;
  }
  for (const e of expenses) {
    const i = idx[`${e.date.getFullYear()}-${e.date.getMonth()}`];
    if (i !== undefined) months[i].expenses += e.amount;
  }

  return (
    <div className="space-y-6">
      <div>
        <h1 className="text-2xl font-bold text-ink">Good {now.getHours() < 12 ? "morning" : now.getHours() <
17 ? "afternoon" : "evening"}, {user.firstName} 👋</h1>
        <p className="mt-1 text-sm text-slate-500">Here's what's happening at {settings.company_name} today.
</p>
      </div>

      <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
        {can(user, "invoices") && <Stat label="Outstanding invoices" value={money(outstanding, cur)} sub=
{`${openInvoices.length} unpaid · ${overdue.length} overdue`} accent="red" href="/admin/invoices?
status=UNPAID" />}
        {can(user, "payments") && <Stat label="Received this month" value={money(paidThisMonth._sum.amount ??
0, cur)} sub={now.toLocaleString("en", { month: "long", year: "numeric" })} accent="green"
href="/admin/payments" />}
        {can(user, "leads") && <Stat label="New leads this month" value={leadsThisMonth} sub="Across all
sources" accent="blue" href="/admin/leads" />}
        {can(user, "projects") && <Stat label="Active projects" value={activeProjects} sub={`${openTickets}
open support tickets`} accent="yellow" href="/admin/projects" />}
      </div>

      <div className="grid gap-6 xl:grid-cols-3">
        {can(user, "payments") && (
          <Card title="Payments vs expenses — last 12 months" className="xl:col-span-2">
            <div className="p-4">
              <RevenueChart data={months.map(({ month, revenue, expenses }) => ({ month, revenue, expenses
}))} />
            </div>
          </Card>
        )}
        {can(user, "leads") && (
          <Card title="Leads pipeline" actions={<Link href="/admin/leads?view=kanban" className="link text-
xs">Open board</Link>}>
            <div className="p-5">
              <DonutChart data={leadStatuses.map((s) => ({ name: s.name, value: s._count.leads, color:
s.color }))} />
            </div>
          </Card>
        )}
      </div>

      <div className="grid gap-6 xl:grid-cols-3">
        <Card title="My open tasks" actions={<Link href="/admin/tasks?mine=1" className="link text-xs">View
all</Link>}>
          {myTasks.length === 0 ? (
            <EmptyState title="Nothing assigned to you" />
          ) : (
            <ul className="divide-y divide-slate-100">
              {myTasks.map((t) => (
                <li key={t.id} className="px-5 py-3">
                  <Link href={`/admin/tasks/${t.id}`} className="text-sm font-medium text-slate-800
hover:text-primary">
                    {t.name}
                  </Link>
                  <div className="mt-1 flex flex-wrap items-center gap-2 text-xs text-slate-500">
                    <StatusBadge map={TASK_STATUS} value={t.status} />
                    <StatusBadge map={PRIORITY} value={t.priority} />
                    {t.project && <span>{t.project.name}</span>}
                    {t.dueDate && <span className={t.dueDate < today ? "font-semibold text-brand-red" :
""}>Due {fmtDate(t.dueDate)}</span>}
                  </div>
                </li>
```

```
              ))}
            </ul>
          )}
        </Card>

        {can(user, "invoices") && (
          <Card title="Overdue invoices" actions={<Link href="/admin/invoices?status=OVERDUE" className="link
text-xs">View all</Link>}>
            {overdue.length === 0 ? (
              <EmptyState title="No overdue invoices 🎉" />
            ) : (
              <ul className="divide-y divide-slate-100">
                {overdue.map((i) => (
                  <li key={i.id} className="flex items-center justify-between px-5 py-3">
                    <div>
                      <Link href={`/admin/invoices/${i.id}`} className="text-sm font-medium text-slate-800
hover:text-primary">
                        {i.prefix}
                        {String(i.number).padStart(6, "0")}
                      </Link>
                      <p className="text-xs text-slate-500">{i.customer.company}</p>
                    </div>
                    <div className="text-right">
                      <p className="text-sm font-semibold">{money(i.total, i.currency)}</p>
                      <Badge tone="red">Due {fmtDate(i.dueDate)}</Badge>
                    </div>
                  </li>
                ))}
              </ul>
            )}
          </Card>
        )}

        <TodoWidget todos={todos.map((t) => ({ id: t.id, text: t.text, done: t.done }))} />
      </div>

      <Card title="Latest activity">
        {activities.length === 0 ? (
          <EmptyState title="No activity yet" text="Actions across the CRM will appear here." />
        ) : (
          <ul className="divide-y divide-slate-100">
            {activities.map((a) => (
              <li key={a.id} className="flex items-center justify-between gap-4 px-5 py-3 text-sm">
                <span className="text-slate-700">
                  <span className="font-semibold">{a.staff ? `${a.staff.firstName} ${a.staff.lastName}` :
"System"}</span> {a.description}
                </span>
                <span className="shrink-0 text-xs text-slate-400">{timeAgo(a.createdAt)}</span>
              </li>
            ))}
          </ul>
        )}
      </Card>
    </div>
  );
}
```

src/app/admin/payments/page.tsx

```
import Link from "next/link";
import type { Prisma } from "@prisma/client";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { fmtDate, money } from "@/lib/format";
import { docNumber } from "@/lib/sales";
import { getSettings } from "@/lib/settings";
import { Card, EmptyState, PageHeader, Stat, Table } from "@/components/ui";
import { FilterSelect, SearchBox } from "@/components/client";
import { Pagination, pageArgs } from "@/components/pagination";

export const metadata = { title: "Payments" };

export default async function PaymentsPage({ searchParams }: { searchParams: Promise<Record<string, string |
```

```
undefined>> }) {
  await requirePerm("payments");
  const sp = await searchParams;
  const { page, skip, take } = pageArgs(sp.page);
  const { currency } = await getSettings();
  const where: Prisma.PaymentWhereInput = {};
  if (sp.mode) where.mode = sp.mode;
  if (sp.q) where.OR = [{ transactionId: { contains: sp.q } }, { invoice: { customer: { company: { contains:
sp.q } } } }];
  const monthStart = new Date(new Date().getFullYear(), new Date().getMonth(), 1);
  const yearStart = new Date(new Date().getFullYear(), 0, 1);
  const [rows, total, modes, month, year] = await Promise.all([
    db.payment.findMany({ where, skip, take, orderBy: { date: "desc" }, include: { invoice: { include: {
customer: { select: { id: true, company: true } } } } } }),
    db.payment.count({ where }),
    db.paymentMode.findMany(),
    db.payment.aggregate({ _sum: { amount: true }, where: { date: { gte: monthStart } } }),
    db.payment.aggregate({ _sum: { amount: true }, where: { date: { gte: yearStart } } }),
  ]);
  return (
    <>
      <PageHeader title="Payments" subtitle="Payments are recorded from an invoice" />
      <div className="mb-6 grid gap-4 sm:grid-cols-2">
        <Stat label="This month" value={money(month._sum.amount ?? 0, currency)} accent="green" />
        <Stat label="This year" value={money(year._sum.amount ?? 0, currency)} accent="blue" />
      </div>
      <Card>
        <div className="flex flex-wrap gap-2 border-b border-slate-100 p-4">
          <SearchBox placeholder="Customer or transaction ID…" />
          <FilterSelect name="mode" placeholder="All payment modes" options={modes.map((m) => ({ value:
m.name, label: m.name }))} />
        </div>
        {rows.length === 0 ? (
          <EmptyState title="No payments yet" text="Open an invoice and use “Record payment”." />
        ) : (
          <Table head={["#", "Invoice", "Customer", "Mode", "Transaction ID", "Date", "Amount"]}>
            {rows.map((p) => (
              <tr key={p.id} className="hover:bg-slate-50/60">
                <td className="td text-slate-400">{p.id}</td>
                <td className="td"><Link className="link" href={`/admin/invoices/${p.invoiceId}`}>
{docNumber(p.invoice.prefix, p.invoice.number)}</Link></td>
                <td className="td"><Link className="hover:text-primary" href=
{`/admin/customers/${p.invoice.customer.id}`}>{p.invoice.customer.company}</Link></td>
                <td className="td">{p.mode}</td>
                <td className="td text-slate-500">{p.transactionId ?? "—"}</td>
                <td className="td">{fmtDate(p.date)}</td>
                <td className="td font-semibold text-brand-green">{money(p.amount, p.invoice.currency)}</td>
              </tr>
            ))}
          </Table>
        )}
        <Pagination total={total} page={page} params={sp} />
      </Card>
    </>
  );
}
```

src/app/admin/profile/actions.ts

```
"use server";
import bcrypt from "bcryptjs";
import { db } from "@/lib/db";
import { requireUser } from "@/lib/auth";
import { reqStr, str, type ActionState } from "@/lib/forms";

export async function updateProfile(_: ActionState, f: FormData): Promise<ActionState> {
  const user = await requireUser();
  const firstName = reqStr(f, "firstName");
  if (!firstName) return { error: "First name is required." };
  await db.staff.update({ where: { id: user.id }, data: { firstName, lastName: reqStr(f, "lastName"), phone:
str(f, "phone") } });
  return { ok: true };
}
```

```
export async function changePassword(_: ActionState, f: FormData): Promise<ActionState> {
  const user = await requireUser();
  const staff = await db.staff.findUnique({ where: { id: user.id } });
  if (!staff || !(await bcrypt.compare(String(f.get("current") || ""), staff.password))) return { error:
"Current password is incorrect." };
  const next = String(f.get("password") || "");
  if (next.length < 6) return { error: "New password must be at least 6 characters." };
  if (next !== f.get("confirm")) return { error: "Passwords don't match." };
  await db.staff.update({ where: { id: user.id }, data: { password: await bcrypt.hash(next, 10) } });
  return { ok: true };
}
```

src/app/admin/profile/page.tsx

```
import { requireUser } from "@/lib/auth";
import { db } from "@/lib/db";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field, PageHeader } from "@/components/ui";
import { changePassword, updateProfile } from "./actions";

export const metadata = { title: "My profile" };
export default async function Profile() {
  const user = await requireUser();
  const s = (await db.staff.findUnique({ where: { id: user.id } }))!;
  return (
    <div className="max-w-3xl space-y-6">
      <PageHeader title="My profile" subtitle={`${s.email} · ${user.roleName ?? "Staff"}`} />
      <ActionForm action={updateProfile} savedText="Profile updated.">
        <Card title="Details">
          <div className="grid gap-4 p-5 sm:grid-cols-3">
            <Field label="First name"><input name="firstName" defaultValue={s.firstName} className="input" />
</Field>
            <Field label="Last name"><input name="lastName" defaultValue={s.lastName} className="input" />
</Field>
            <Field label="Phone"><input name="phone" defaultValue={s.phone ?? ""} className="input" />
</Field>
          </div>
          <div className="flex justify-end border-t border-slate-100 p-4"><SubmitButton>Save</SubmitButton>
</div>
        </Card>
      </ActionForm>
      <ActionForm action={changePassword} savedText="Password changed." resetOnSuccess>
        <Card title="Change password">
          <div className="grid gap-4 p-5 sm:grid-cols-3">
            <Field label="Current password"><input name="current" type="password" required className="input"
/></Field>
            <Field label="New password"><input name="password" type="password" required minLength={6}
className="input" /></Field>
            <Field label="Confirm new password"><input name="confirm" type="password" required
className="input" /></Field>
          </div>
          <div className="flex justify-end border-t border-slate-100 p-4"><SubmitButton>Change
password</SubmitButton></div>
        </Card>
      </ActionForm>
    </div>
  );
}
```

src/app/admin/projects/[id]/edit/page.tsx

```
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { ProjectForm } from "../../form";
import { saveProject } from "../../actions";

export default async function EditProject({ params }: { params: Promise<{ id: string }> }) {
  await requirePerm("projects", "edit");
  const { id } = await params;
```

```
  const p = await db.project.findUnique({ where: { id: Number(id) }, include: { members: true } });
  if (!p) notFound();
  return (
    <div className="max-w-4xl">
      <PageHeader title={`Edit ${p.name}`} back={{ href: `/admin/projects/${p.id}`, label: p.name }} />
      <ProjectForm action={saveProject.bind(null, p.id)} project={p} />
    </div>
  );
}
```

src/app/admin/projects/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { fmtDate, formatDuration, money } from "@/lib/format";
import { docNumber, isOverdue } from "@/lib/sales";
import { INVOICE_STATUS, PRIORITY, PROJECT_STATUS, TASK_STATUS } from "@/lib/constants";
import { Avatar, ButtonLink, Card, DetailRow, EmptyState, PageHeader, Stat, StatusBadge, Table, Tabs } from
"@/components/ui";
import { ConfirmButton } from "@/components/client";
import { Notes } from "@/components/notes";
import { deleteProject } from "../actions";

export default async function ProjectPage({ params, searchParams }: { params: Promise<{ id: string }>;
searchParams: Promise<{ tab?: string }> }) {
  const user = await requirePerm("projects");
  const { id } = await params;
  const { tab = "overview" } = await searchParams;
  const p = await db.project.findUnique({
    where: { id: Number(id) },
    include: {
      customer: true,
      members: true,
      tasks: { orderBy: [{ status: "asc" }, { dueDate: "asc" }], include: { assignees: true, timeEntries:
true } },
      invoices: { include: { payments: true } },
      expenses: { include: { category: true } },
    },
  });
  if (!p) notFound();
  const done = p.tasks.filter((t) => t.status === "COMPLETE").length;
  const ms = p.tasks.flatMap((t) => t.timeEntries).reduce((s, e) => s + ((e.endTime ?? new Date()).getTime()
- e.startTime.getTime()), 0);
  const billed = p.invoices.filter((i) => i.status !== "CANCELLED").reduce((s, i) => s + i.total, 0);
  const spent = p.expenses.reduce((s, e) => s + e.amount, 0);
  const daysLeft = p.deadline ? Math.ceil((p.deadline.getTime() - Date.now()) / 86400000) : null;
  const base = `/admin/projects/${p.id}`;
  const cur = p.customer.currency || "INR";

  return (
    <>
      <PageHeader
        title={p.name}
        subtitle={<Link className="link" href={`/admin/customers/${p.customerId}`}>{p.customer.company}
</Link>}
        back={{ href: "/admin/projects", label: "Projects" }}
        actions={
          <>
            <StatusBadge map={PROJECT_STATUS} value={p.status} />
            {can(user, "tasks", "create") && <ButtonLink variant="secondary" href={`/admin/tasks/new?
projectId=${p.id}`}>+ Task</ButtonLink>}
            {can(user, "invoices", "create") && <ButtonLink variant="secondary" href={`/admin/invoices/new?
customerId=${p.customerId}&projectId=${p.id}`}>Invoice project</ButtonLink>}
            {can(user, "projects", "edit") && <ButtonLink variant="secondary" href=
{`${base}/edit`}>Edit</ButtonLink>}
            {can(user, "projects", "delete") && <ConfirmButton action={deleteProject} hidden={{ id: p.id }}
/>}
          </>
        }
      />
```

```
      <div className="mb-6 grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
        <Stat label="Progress" value={`${p.progress}%`} sub={<span className="mt-2 block h-1.5 overflow-
hidden rounded-full bg-slate-100"><span className="block h-full bg-brand-green" style={{ width:
`${p.progress}%` }} /></span>} accent="green" />
        <Stat label="Tasks" value={`${done} / ${p.tasks.length}`} sub="completed" accent="blue" />
        <Stat label="Time logged" value={formatDuration(ms)} sub={p.billingType === "HOURLY" ? `≈ ${money((ms
/ 3600000) * p.rate, cur)} billable` : "Fixed price"} accent="yellow" />
        <Stat label="Deadline" value={p.deadline ? fmtDate(p.deadline) : "—"} sub={daysLeft !== null ?
(daysLeft >= 0 ? `${daysLeft} days left` : `${-daysLeft} days overdue`) : undefined} accent="red" />
      </div>
      <Tabs
        base={base}
        active={tab}
        tabs={[
          { key: "overview", label: "Overview" },
          { key: "tasks", label: "Tasks", count: p.tasks.length },
          { key: "finance", label: "Invoices & expenses", count: p.invoices.length + p.expenses.length },
          { key: "notes", label: "Notes" },
        ]}
      />
      {tab === "overview" && (
        <div className="grid gap-6 lg:grid-cols-3">
          <Card title="Project details" className="lg:col-span-2">
            <dl className="divide-y divide-slate-100 px-5">
              <DetailRow label="Customer">{p.customer.company}</DetailRow>
              <DetailRow label="Billing">{p.billingType === "FIXED" ? `Fixed · ${money(p.rate, cur)}` :
`Hourly · ${money(p.rate, cur)}/h`}</DetailRow>
              <DetailRow label="Start date">{fmtDate(p.startDate)}</DetailRow>
              <DetailRow label="Deadline">{fmtDate(p.deadline)}</DetailRow>
              <DetailRow label="Invoiced">{money(billed, cur)}</DetailRow>
              <DetailRow label="Expenses">{money(spent, cur)}</DetailRow>
            </dl>
            {p.description && <p className="whitespace-pre-wrap border-t border-slate-100 px-5 py-4 text-sm
text-slate-600">{p.description}</p>}
          </Card>
          <Card title={`Members (${p.members.length})`}>
            <ul className="divide-y divide-slate-100">
              {p.members.map((m) => (
                <li key={m.id} className="flex items-center gap-3 px-5 py-3 text-sm"><Avatar name=
{`${m.firstName} ${m.lastName}`} /> {m.firstName} {m.lastName}</li>
              ))}
              {p.members.length === 0 && <li className="px-5 py-6 text-center text-sm text-slate-400">No
members assigned.</li>}
            </ul>
          </Card>
        </div>
      )}
      {tab === "tasks" && (
        <Card actions={<Link className="link text-xs" href={`/admin/tasks?
view=kanban&project=${p.id}`}>Kanban view</Link>} title="Tasks">
          {p.tasks.length === 0 ? (
            <EmptyState title="No tasks yet" action={<ButtonLink href={`/admin/tasks/new?
projectId=${p.id}`}>Add task</ButtonLink>} />
          ) : (
            <Table head={["Task", "Assignees", "Priority", "Due", "Status"]}>
              {p.tasks.map((t) => (
                <tr key={t.id}>
                  <td className="td"><Link className="link" href={`/admin/tasks/${t.id}`}>{t.name}</Link>
</td>
                  <td className="td"><div className="flex -space-x-2">{t.assignees.map((a) => <Avatar key=
{a.id} name={`${a.firstName} ${a.lastName}`} size={24} />)}</div></td>
                  <td className="td"><StatusBadge map={PRIORITY} value={t.priority} /></td>
                  <td className="td">{fmtDate(t.dueDate)}</td>
                  <td className="td"><StatusBadge map={TASK_STATUS} value={t.status} /></td>
                </tr>
              ))}
            </Table>
          )}
        </Card>
      )}
      {tab === "finance" && (
        <div className="space-y-6">
          <Card title="Invoices">
            {p.invoices.length === 0 ? <p className="px-5 py-6 text-center text-sm text-slate-400">No
invoices linked.</p> : (
```

```
              <Table head={["Invoice", "Date", "Total", "Status"]}>
                {p.invoices.map((i) => (
                  <tr key={i.id}>
                    <td className="td"><Link className="link" href={`/admin/invoices/${i.id}`}>
{docNumber(i.prefix, i.number)}</Link></td>
                    <td className="td">{fmtDate(i.date)}</td>
                    <td className="td">{money(i.total, i.currency)}</td>
                    <td className="td"><StatusBadge map={INVOICE_STATUS} value={isOverdue(i) ? "OVERDUE" :
i.status} /></td>
                  </tr>
                ))}
              </Table>
            )}
          </Card>
          <Card title="Expenses">
            {p.expenses.length === 0 ? <p className="px-5 py-6 text-center text-sm text-slate-400">No
expenses linked.</p> : (
              <Table head={["Expense", "Category", "Date", "Amount"]}>
                {p.expenses.map((e) => (
                  <tr key={e.id}>
                    <td className="td"><Link className="link" href={`/admin/expenses/${e.id}`}>{e.name}
</Link></td>
                    <td className="td">{e.category.name}</td>
                    <td className="td">{fmtDate(e.date)}</td>
                    <td className="td">{money(e.amount, cur)}</td>
                  </tr>
                ))}
              </Table>
            )}
          </Card>
        </div>
      )}
      {tab === "notes" && <Notes relType="project" relId={p.id} path={base} />}
    </>
  );
}
```

src/app/admin/projects/actions.ts

```
"use server";
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { requirePerm } from "@/lib/auth";
import { logActivity } from "@/lib/activity";
import { str, reqStr, num, intOrNull, dateOrNull, ints, type ActionState } from "@/lib/forms";

function data(f: FormData) {
  return {
    name: reqStr(f, "name"),
    customerId: intOrNull(f, "customerId") ?? 0,
    status: reqStr(f, "status") || "NOT_STARTED",
    billingType: reqStr(f, "billingType") || "FIXED",
    rate: num(f, "rate"),
    startDate: dateOrNull(f, "startDate") ?? new Date(),
    deadline: dateOrNull(f, "deadline"),
    description: str(f, "description"),
    progress: Math.min(100, Math.max(0, Math.round(num(f, "progress")))),
  };
}

export async function saveProject(id: number | null, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("projects", id ? "edit" : "create");
  const d = data(f);
  if (!d.name || !d.customerId) return { error: "Project name and customer are required." };
  const members = ints(f, "members").map((x) => ({ id: x }));
  const p = id
    ? await db.project.update({ where: { id }, data: { ...d, members: { set: members } } })
    : await db.project.create({ data: { ...d, members: { connect: members } } });
  await logActivity("project", p.id, `${id ? "updated" : "created"} project ${p.name}`, user.id);
  redirect(`/admin/projects/${p.id}`);
}

export async function deleteProject(f: FormData) {
```

```
  const user = await requirePerm("projects", "delete");
  const id = Number(f.get("id"));
  const p = await db.project.delete({ where: { id } });
  await logActivity("project", id, `deleted project ${p.name}`, user.id);
  redirect("/admin/projects");
}
```

src/app/admin/projects/form.tsx

```
import type { Project, Staff } from "@prisma/client";
import { db } from "@/lib/db";
import { inputDate } from "@/lib/format";
import { PROJECT_STATUS } from "@/lib/constants";
import type { ActionState } from "@/lib/forms";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field } from "@/components/ui";

export async function ProjectForm({
  action,
  project,
  defaultCustomerId,
}: {
  action: (s: ActionState, f: FormData) => Promise<ActionState>;
  project?: Project & { members: Staff[] };
  defaultCustomerId?: number;
}) {
  const [customers, staff] = await Promise.all([
    db.customer.findMany({ where: { active: true }, orderBy: { company: "asc" }, select: { id: true, company:
true } }),
    db.staff.findMany({ where: { active: true }, orderBy: { firstName: "asc" } }),
  ]);
  const p = project;
  const memberIds = new Set(p?.members.map((m) => m.id));
  return (
    <ActionForm action={action}>
      <Card>
        <div className="grid gap-4 p-5 sm:grid-cols-2">
          <Field label="Project name *" className="sm:col-span-2"><input name="name" required defaultValue=
{p?.name} className="input" /></Field>
          <Field label="Customer *">
            <select name="customerId" required defaultValue={p?.customerId ?? defaultCustomerId ?? ""}
className="input">
              <option value="">Select…</option>
              {customers.map((c) => <option key={c.id} value={c.id}>{c.company}</option>)}
            </select>
          </Field>
          <Field label="Status">
            <select name="status" defaultValue={p?.status ?? "NOT_STARTED"} className="input">
              {Object.entries(PROJECT_STATUS).map(([k, v]) => <option key={k} value={k}>{v.label}</option>)}
            </select>
          </Field>
          <Field label="Billing type">
            <select name="billingType" defaultValue={p?.billingType ?? "FIXED"} className="input">
              <option value="FIXED">Fixed price</option>
              <option value="HOURLY">Hourly rate</option>
            </select>
          </Field>
          <Field label="Total price / hourly rate"><input name="rate" type="number" step="0.01" min="0"
defaultValue={p?.rate ?? ""} className="input" /></Field>
          <Field label="Start date"><input name="startDate" type="date" required defaultValue=
{inputDate(p?.startDate ?? new Date())} className="input" /></Field>
          <Field label="Deadline"><input name="deadline" type="date" defaultValue={inputDate(p?.deadline)}
className="input" /></Field>
          <Field label={`Progress (%)`}><input name="progress" type="number" min="0" max="100" defaultValue=
{p?.progress ?? 0} className="input" /></Field>
          <Field label="Description" className="sm:col-span-2"><textarea name="description" rows={4}
defaultValue={p?.description ?? ""} className="input" /></Field>
          <div className="sm:col-span-2">
            <span className="label">Members</span>
            <div className="flex flex-wrap gap-2">
              {staff.map((s) => (
                <label key={s.id} className="flex items-center gap-2 rounded-lg border border-slate-200 px-3
py-1.5 text-sm has-[:checked]:border-primary has-[:checked]:bg-blue-50">
```

```
                  <input type="checkbox" name="members" value={s.id} defaultChecked={memberIds.has(s.id)} />
{s.firstName} {s.lastName}
                </label>
              ))}
            </div>
          </div>
        </div>
        <div className="flex justify-end border-t border-slate-100 p-4"><SubmitButton>{p ? "Save project" :
"Create project"}</SubmitButton></div>
      </Card>
    </ActionForm>
  );
}
```

src/app/admin/projects/new/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { PageHeader } from "@/components/ui";
import { ProjectForm } from "../form";
import { saveProject } from "../actions";

export const metadata = { title: "New project" };
export default async function NewProject({ searchParams }: { searchParams: Promise<{ customerId?: string }>
}) {
  await requirePerm("projects", "create");
  const sp = await searchParams;
  return (
    <div className="max-w-4xl">
      <PageHeader title="New project" back={{ href: "/admin/projects", label: "Projects" }} />
      <ProjectForm action={saveProject.bind(null, null)} defaultCustomerId={sp.customerId ?
Number(sp.customerId) : undefined} />
    </div>
  );
}
```

src/app/admin/projects/page.tsx

```
import Link from "next/link";
import type { Prisma } from "@prisma/client";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { fmtDate } from "@/lib/format";
import { PROJECT_STATUS } from "@/lib/constants";
import { Avatar, ButtonLink, Card, EmptyState, PageHeader, StatusBadge, Table } from "@/components/ui";
import { FilterSelect, SearchBox } from "@/components/client";
import { Pagination, pageArgs } from "@/components/pagination";

export const metadata = { title: "Projects" };

export default async function ProjectsPage({ searchParams }: { searchParams: Promise<Record<string, string |
undefined>> }) {
  await requirePerm("projects");
  const sp = await searchParams;
  const { page, skip, take } = pageArgs(sp.page);
  const where: Prisma.ProjectWhereInput = {};
  if (sp.status) where.status = sp.status;
  if (sp.q) where.OR = [{ name: { contains: sp.q } }, { customer: { company: { contains: sp.q } } }];
  const [rows, total, groups] = await Promise.all([
    db.project.findMany({
      where, skip, take, orderBy: { createdAt: "desc" },
      include: { customer: { select: { company: true } }, members: true, _count: { select: { tasks: true } },
tasks: { where: { status: "COMPLETE" }, select: { id: true } } } },
    }),
    db.project.count({ where }),
    db.project.groupBy({ by: ["status"], _count: true }),
  ]);
  return (
    <>
      <PageHeader title="Projects" actions={<ButtonLink href="/admin/projects/new">+ New
project</ButtonLink>} />
      <div className="mb-6 grid grid-cols-2 gap-3 sm:grid-cols-5">
        {Object.entries(PROJECT_STATUS).map(([k, v]) => (
```

```
          <Link key={k} href={`?status=${k}`} className="card px-4 py-3 hover:shadow-md">
            <p className="text-2xl font-bold text-ink">{groups.find((g) => g.status === k)?._count ?? 0}</p>
            <p className="text-xs text-slate-500">{v.label}</p>
          </Link>
        ))}
      </div>
      <Card>
        <div className="flex flex-wrap gap-2 border-b border-slate-100 p-4">
          <SearchBox placeholder="Search projects…" />
          <FilterSelect name="status" placeholder="All statuses" options=
{Object.entries(PROJECT_STATUS).map(([value, s]) => ({ value, label: s.label }))} />
        </div>
        {rows.length === 0 ? (
          <EmptyState title="No projects" action={<ButtonLink href="/admin/projects/new">Create
project</ButtonLink>} />
        ) : (
          <Table head={["Project", "Customer", "Members", "Tasks", "Progress", "Start", "Deadline",
"Status"]}>
            {rows.map((p) => (
              <tr key={p.id} className="hover:bg-slate-50/60">
                <td className="td"><Link className="link" href={`/admin/projects/${p.id}`}>{p.name}</Link>
</td>
                <td className="td">{p.customer.company}</td>
                <td className="td"><div className="flex -space-x-2">{p.members.slice(0, 4).map((m) => <Avatar
key={m.id} name={`${m.firstName} ${m.lastName}`} size={26} />)}</div></td>
                <td className="td">{p.tasks.length}/{p._count.tasks}</td>
                <td className="td">
                  <div className="flex items-center gap-2">
                    <div className="h-1.5 w-20 overflow-hidden rounded-full bg-slate-100"><div className="h-
full bg-brand-green" style={{ width: `${p.progress}%` }} /></div>
                    <span className="text-xs text-slate-500">{p.progress}%</span>
                  </div>
                </td>
                <td className="td">{fmtDate(p.startDate)}</td>
                <td className="td">{fmtDate(p.deadline)}</td>
                <td className="td"><StatusBadge map={PROJECT_STATUS} value={p.status} /></td>
              </tr>
            ))}
          </Table>
        )}
        <Pagination total={total} page={page} params={sp} />
      </Card>
    </>
  );
}
```

src/app/admin/reports/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { money } from "@/lib/format";
import { getSettings } from "@/lib/settings";
import { Card, PageHeader, Stat, Table } from "@/components/ui";
import { RevenueChart, DonutChart } from "@/components/charts";
import { FilterSelect } from "@/components/client";

export const metadata = { title: "Reports" };

export default async function ReportsPage({ searchParams }: { searchParams: Promise<{ year?: string }> }) {
  await requirePerm("reports");
  const sp = await searchParams;
  const { currency } = await getSettings();
  const year = Number(sp.year) || new Date().getFullYear();
  const from = new Date(year, 0, 1);
  const to = new Date(year + 1, 0, 1);
  const [payments, expenses, invoices, leads, sources, customers] = await Promise.all([
    db.payment.findMany({ where: { date: { gte: from, lt: to } }, include: { invoice: { select: { customerId:
true } } } }),
    db.expense.findMany({ where: { date: { gte: from, lt: to } }, include: { category: true } }),
    db.invoice.findMany({ where: { date: { gte: from, lt: to }, status: { not: "CANCELLED" } }, select: {
status: true, total: true, taxTotal: true } }),
    db.lead.findMany({ where: { createdAt: { gte: from, lt: to } }, select: { sourceId: true, customerId:
true } }),
```

```
    db.leadSource.findMany(),
    db.customer.findMany({ select: { id: true, company: true } }),
  ]);
  const months = Array.from({ length: 12 }, (_, m) => ({ month: new Date(year, m, 1).toLocaleString("en", {
month: "short" }), revenue: 0, expenses: 0 }));
  payments.forEach((p) => (months[p.date.getMonth()].revenue += p.amount));
  expenses.forEach((e) => (months[e.date.getMonth()].expenses += e.amount));
  const income = payments.reduce((s, p) => s + p.amount, 0);
  const spent = expenses.reduce((s, e) => s + e.amount, 0);
  const tax = invoices.filter((i) => i.status !== "DRAFT").reduce((s, i) => s + i.taxTotal, 0);

  const byCustomer = new Map<number, number>();
  payments.forEach((p) => byCustomer.set(p.invoice.customerId, (byCustomer.get(p.invoice.customerId) ?? 0) +
p.amount));
  const top = [...byCustomer.entries()].sort((a, b) => b[1] - a[1]).slice(0, 8);
  const byCat = new Map<string, number>();
  expenses.forEach((e) => byCat.set(e.category.name, (byCat.get(e.category.name) ?? 0) + e.amount));
  const palette = ["#4f74c4", "#27ae57", "#e0a400", "#e84a37", "#7c5cc4", "#0f9fb0", "#94a3b8"];
  const firstYear = 2020;
  const years = Array.from({ length: new Date().getFullYear() - firstYear + 2 }, (_, i) => String(new
Date().getFullYear() + 1 - i));

  return (
    <>
      <PageHeader title={`Reports · ${year}`} actions={<FilterSelect name="year" placeholder={`This year`}
options={years.map((y) => ({ value: y, label: y }))} />} />
      <div className="mb-6 grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
        <Stat label="Income received" value={money(income, currency)} accent="green" />
        <Stat label="Expenses" value={money(spent, currency)} accent="red" />
        <Stat label="Net profit" value={money(income - spent, currency)} accent="blue" />
        <Stat label="Tax invoiced" value={money(tax, currency)} accent="yellow" />
      </div>
      <Card title="Income vs expenses by month" className="mb-6"><div className="p-4"><RevenueChart data=
{months} /></div></Card>
      <div className="grid gap-6 xl:grid-cols-3">
        <Card title="Top customers by payments" className="xl:col-span-2">
          <Table head={["Customer", "Paid"]}>
            {top.map(([id, amt]) => (
              <tr key={id}><td className="td">{customers.find((c) => c.id === id)?.company}</td><td
className="td font-semibold">{money(amt, currency)}</td></tr>
            ))}
            {top.length === 0 && <tr><td colSpan={2} className="px-5 py-6 text-center text-sm text-slate-
400">No payments this year.</td></tr>}
          </Table>
        </Card>
        <Card title="Expenses by category"><div className="p-5"><DonutChart data=
{[...byCat.entries()].map(([name, value], i) => ({ name, value: Math.round(value), color: palette[i %
palette.length] }))} /></div></Card>
      </div>
      <Card title="Lead conversion by source" className="mt-6">
        <Table head={["Source", "Leads", "Converted", "Conversion rate"]}>
          {[...sources, { id: null as number | null, name: "No source" }].map((s) => {
            const list = leads.filter((l) => l.sourceId === s.id);
            if (!list.length) return null;
            const conv = list.filter((l) => l.customerId).length;
            return (
              <tr key={String(s.id)}>
                <td className="td">{s.name}</td><td className="td">{list.length}</td><td className="td">
{conv}</td>
                <td className="td"><div className="flex items-center gap-2"><div className="h-1.5 w-24
overflow-hidden rounded-full bg-slate-100"><div className="h-full bg-brand-green" style={{ width: `${(conv /
list.length) * 100}%` }} /></div>{Math.round((conv / list.length) * 100)}%</div></td>
              </tr>
            );
          })}
        </Table>
      </Card>
    </>
  );
}
```

src/app/admin/search/page.tsx

```
import Link from "next/link";
import { requireUser } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { docNumber } from "@/lib/sales";
import { Card, EmptyState, PageHeader } from "@/components/ui";

export const metadata = { title: "Search" };

export default async function SearchPage({ searchParams }: { searchParams: Promise<{ q?: string }> }) {
  const user = await requireUser();
  const q = ((await searchParams).q ?? "").trim();
  if (!q) return <PageHeader title="Search" subtitle="Type in the search bar above." />;
  const n = parseInt(q.replace(/\D/g, ""), 10);
  const [customers, leads, invoices, projects, tasks, tickets] = await Promise.all([
    can(user, "customers") ? db.customer.findMany({ where: { OR: [{ company: { contains: q } }, { phone: {
contains: q } }, { contacts: { some: { OR: [{ email: { contains: q } }, { firstName: { contains: q } }] } }
}] }, take: 8 }) : [],
    can(user, "leads") ? db.lead.findMany({ where: { OR: [{ name: { contains: q } }, { company: { contains: q
} }, { email: { contains: q } }, { phone: { contains: q } }] }, take: 8 }) : [],
    can(user, "invoices") && Number.isFinite(n) ? db.invoice.findMany({ where: { number: n }, include: {
customer: true }, take: 8 }) : [],
    can(user, "projects") ? db.project.findMany({ where: { name: { contains: q } }, take: 8 }) : [],
    can(user, "tasks") ? db.task.findMany({ where: { name: { contains: q } }, take: 8 }) : [],
    can(user, "tickets") ? db.ticket.findMany({ where: { subject: { contains: q } }, take: 8 }) : [],
  ]);
  const groups = [
    { title: "Customers", items: customers.map((c) => ({ href: `/admin/customers/${c.id}`, label: c.company
})) },
    { title: "Leads", items: leads.map((l) => ({ href: `/admin/leads/${l.id}`, label: `${l.name}${l.company ?
` · ${l.company}` : ""}` })) },
    { title: "Invoices", items: invoices.map((i) => ({ href: `/admin/invoices/${i.id}`, label:
`${docNumber(i.prefix, i.number)} · ${i.customer.company}` })) },
    { title: "Projects", items: projects.map((p) => ({ href: `/admin/projects/${p.id}`, label: p.name })) },
    { title: "Tasks", items: tasks.map((t) => ({ href: `/admin/tasks/${t.id}`, label: t.name })) },
    { title: "Tickets", items: tickets.map((t) => ({ href: `/admin/tickets/${t.id}`, label: `#${t.id}
${t.subject}` })) },
  ].filter((g) => g.items.length);
  return (
    <>
      <PageHeader title={`Results for “${q}”`} />
      {groups.length === 0 ? (
        <Card><EmptyState title="Nothing found" text="Try a different name, email, phone or invoice number."
/></Card>
      ) : (
        <div className="grid gap-6 md:grid-cols-2">
          {groups.map((g) => (
            <Card key={g.title} title={g.title}>
              <ul className="divide-y divide-slate-100">
                {g.items.map((i) => <li key={i.href}><Link href={i.href} className="block px-5 py-2.5 text-sm
hover:bg-slate-50 hover:text-primary">{i.label}</Link></li>)}
              </ul>
            </Card>
          ))}
        </div>
      )}
    </>
  );
}
```

src/app/admin/settings/actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { db } from "@/lib/db";
import { requirePerm } from "@/lib/auth";
import { DEFAULT_SETTINGS, saveSettings, type Settings } from "@/lib/settings";
import type { ActionState } from "@/lib/forms";

export async function updateSettings(_: ActionState, f: FormData): Promise<ActionState> {
  await requirePerm("settings", "edit");
```

```
  const values: Partial<Settings> = {};
  for (const k of Object.keys(DEFAULT_SETTINGS) as (keyof Settings)[]) {
    const v = f.get(k);
    if (typeof v === "string") values[k] = v.trim();
  }
  await saveSettings(values);
  revalidatePath("/admin", "layout");
  return { ok: true };
}

/* ---- Generic lookup tables ---- */
export type LookupTable = "tax" | "item" | "paymentMode" | "leadStatus" | "leadSource" | "expenseCategory" |
"department";

const FIELDS: Record<LookupTable, Record<string, "string" | "number" | "bool">> = {
  tax: { name: "string", rate: "number" },
  item: { description: "string", longDescription: "string", rate: "number", unit: "string", taxRate: "number"
},
  paymentMode: { name: "string", active: "bool" },
  leadStatus: { name: "string", color: "string", order: "number" },
  leadSource: { name: "string" },
  expenseCategory: { name: "string" },
  department: { name: "string", email: "string" },
};

// eslint-disable-next-line @typescript-eslint/no-explicit-any
const model = (t: LookupTable) => (db as any)[t];

export async function saveLookup(table: LookupTable, _: ActionState, f: FormData): Promise<ActionState> {
  await requirePerm("settings", "edit");
  const fields = FIELDS[table];
  if (!fields) return { error: "Unknown table" };
  const data: Record<string, unknown> = {};
  for (const [k, type] of Object.entries(fields)) {
    const raw = f.get(k);
    if (type === "bool") data[k] = raw === "on";
    else if (type === "number") data[k] = Number(raw) || 0;
    else data[k] = typeof raw === "string" && raw.trim() ? raw.trim() : null;
  }
  const first = Object.keys(fields)[0];
  if (!data[first]) return { error: "Name is required." };
  const id = Number(f.get("id") || 0);
  if (id) await model(table).update({ where: { id }, data });
  else await model(table).create({ data });
  revalidatePath("/admin/settings");
  return { ok: true };
}

export async function deleteLookup(f: FormData) {
  await requirePerm("settings", "edit");
  const table = String(f.get("table")) as LookupTable;
  if (!FIELDS[table]) return;
  try {
    await model(table).delete({ where: { id: Number(f.get("id")) } });
  } catch {
    // in use (foreign key) — ignore silently
  }
  revalidatePath("/admin/settings");
}
```

src/app/admin/settings/lookup.tsx

```
"use client";
import { useState } from "react";
import { ActionForm, ConfirmButton, SubmitButton } from "@/components/client";
import { Card, btn } from "@/components/ui";
import type { ActionState } from "@/lib/forms";

export type Col = { key: string; label: string; type?: "text" | "number" | "color" | "checkbox" | "textarea";
width?: string };

export function LookupTable({
  title,
```

```
  table,
  cols,
  rows,
  save,
  remove,
  hint,
}: {
  title: string;
  table: string;
  cols: Col[];
  rows: Record<string, unknown>[];
  save: (s: ActionState, f: FormData) => Promise<ActionState>;
  remove: (f: FormData) => Promise<void>;
  hint?: string;
}) {
  const [editing, setEditing] = useState<Record<string, unknown> | null>(null);
  const display = (r: Record<string, unknown>, c: Col) => {
    const v = r[c.key];
    if (c.type === "checkbox") return v ? "Yes" : "No";
    if (c.type === "color") return <span className="inline-flex items-center gap-2"><span className="h-3 w-3
rounded-full" style={{ background: String(v) }} />{String(v)}</span>;
    return v === null || v === undefined || v === "" ? "—" : String(v);
  };
  return (
    <Card title={title} actions={<button className={btn("secondary", "py-1.5 text-xs")} onClick={() =>
setEditing({})}>+ Add</button>}>
      {hint && <p className="border-b border-slate-100 px-5 py-2 text-xs text-slate-500">{hint}</p>}
      {editing && (
        <ActionForm
          key={String(editing.id ?? "new")}
          action={async (s, f) => {
            const r = await save(s, f);
            if (r?.ok) setEditing(null);
            return r;
          }}
          className="flex flex-wrap items-end gap-3 border-b border-slate-100 bg-slate-50/60 p-4"
        >
          <input type="hidden" name="id" value={String(editing.id ?? "")} />
          {cols.map((c) => (
            <label key={c.key} className={c.type === "checkbox" ? "flex items-center gap-2 pb-2 text-sm" :
c.width ?? "min-w-40 flex-1"}>
              {c.type === "checkbox" ? (
                <><input type="checkbox" name={c.key} defaultChecked={editing.id ? Boolean(editing[c.key]) :
true} /> {c.label}</>
              ) : (
                <>
                  <span className="label">{c.label}</span>
                  {c.type === "textarea" ? (
                    <textarea name={c.key} rows={1} defaultValue={String(editing[c.key] ?? "")}
className="input" />
                  ) : (
                    <input name={c.key} type={c.type ?? "text"} step="any" defaultValue=
{String(editing[c.key] ?? (c.type === "color" ? "#4f74c4" : ""))} className={c.type === "color" ? "input h-10
p-1" : "input"} />
                  )}
                </>
              )}
            </label>
          ))}
          <SubmitButton>Save</SubmitButton>
          <button type="button" className={btn("ghost")} onClick={() => setEditing(null)}>Cancel</button>
        </ActionForm>
      )}
      <table className="min-w-full">
        <thead className="bg-slate-50/70"><tr>{cols.map((c) => <th key={c.key} className="th">{c.label}
</th>)}<th className="th w-32" /></tr></thead>
        <tbody className="divide-y divide-slate-100">
          {rows.map((r) => (
            <tr key={String(r.id)}>
              {cols.map((c) => <td key={c.key} className="td">{display(r, c)}</td>)}
              <td className="td text-right whitespace-nowrap">
                <button className={btn("ghost", "px-2 py-1 text-xs")} onClick={() =>
setEditing(r)}>Edit</button>
                <ConfirmButton action={remove} hidden={{ id: String(r.id), table }} label="Delete"
confirmLabel="Confirm" variant="ghost" className="px-2 py-1 text-xs" />
```

```
              </td>
            </tr>
          ))}
          {rows.length === 0 && <tr><td colSpan={cols.length + 1} className="px-5 py-6 text-center text-sm
text-slate-400">Nothing here yet.</td></tr>}
        </tbody>
      </table>
    </Card>
  );
}
```

src/app/admin/settings/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { getSettings } from "@/lib/settings";
import { CURRENCIES } from "@/lib/constants";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field, PageHeader, Tabs } from "@/components/ui";
import { LookupTable } from "./lookup";
import { deleteLookup, saveLookup, updateSettings } from "./actions";

export const metadata = { title: "Settings" };

export default async function SettingsPage({ searchParams }: { searchParams: Promise<{ tab?: string }> }) {
  await requirePerm("settings");
  const { tab = "company" } = await searchParams;
  const s = await getSettings();
  const input = (k: keyof typeof s, label: string, opts: { type?: string; span?: boolean; area?: boolean } =
{}) => (
    <Field label={label} className={opts.span ? "sm:col-span-2" : undefined}>
      {opts.area ? <textarea name={k} rows={4} defaultValue={s[k]} className="input" /> : <input name={k}
type={opts.type ?? "text"} defaultValue={s[k]} className="input" />}
    </Field>
  );

  return (
    <>
      <PageHeader title="Settings" />
      <Tabs
        base="/admin/settings"
        active={tab}
        tabs={[
          { key: "company", label: "Company" },
          { key: "finance", label: "Invoices & estimates" },
          { key: "items", label: "Items & taxes" },
          { key: "leads", label: "Leads" },
          { key: "lists", label: "Other lists" },
        ]}
      />

      {tab === "company" && (
        <ActionForm action={updateSettings} savedText="Settings saved." className="max-w-4xl">
          <Card title="Company information" actions={<SubmitButton>Save</SubmitButton>}>
            <div className="grid gap-4 p-5 sm:grid-cols-2">
              {input("company_name", "Company name")}
              {input("company_tagline", "Tagline")}
              {input("company_email", "Email", { type: "email" })}
              {input("company_phone", "Phone")}
              {input("company_address", "Address", { span: true })}
              {input("company_city", "City")}
              {input("company_state", "State")}
              {input("company_zip", "ZIP / PIN")}
              {input("company_country", "Country")}
              {input("company_vat", "GSTIN / VAT number")}
            </div>
          </Card>
        </ActionForm>
      )}

      {tab === "finance" && (
        <ActionForm action={updateSettings} savedText="Settings saved." className="max-w-4xl space-y-6">
          <Card title="General" actions={<SubmitButton>Save</SubmitButton>}>
```

```
            <div className="grid gap-4 p-5 sm:grid-cols-2">
              <Field label="Default currency">
                <select name="currency" defaultValue={s.currency} className="input">{CURRENCIES.map((c) =>
<option key={c}>{c}</option>)}</select>
              </Field>
              {input("bank_details", "Bank / payment details (shown on invoices)", { span: true, area: true
})}
            </div>
          </Card>
          <Card title="Invoices">
            <div className="grid gap-4 p-5 sm:grid-cols-2">
              {input("invoice_prefix", "Number prefix")}
              {input("invoice_due_days", "Due after (days)", { type: "number" })}
              {input("invoice_client_note", "Default client note", { span: true, area: true })}
              {input("invoice_terms", "Default terms & conditions", { span: true, area: true })}
            </div>
          </Card>
          <Card title="Estimates">
            <div className="grid gap-4 p-5 sm:grid-cols-2">
              {input("estimate_prefix", "Number prefix")}
              {input("estimate_expiry_days", "Valid for (days)", { type: "number" })}
              {input("estimate_terms", "Default terms & conditions", { span: true, area: true })}
            </div>
          </Card>
          <div className="flex justify-end"><SubmitButton>Save settings</SubmitButton></div>
        </ActionForm>
      )}

      {tab === "items" && (
        <div className="space-y-6">
          <LookupTable
            title="Saved items / services"
            table="item"
            hint="Reusable lines you can drop into invoices and estimates."
            cols={[
              { key: "description", label: "Description" },
              { key: "longDescription", label: "Long description", type: "textarea" },
              { key: "rate", label: "Rate", type: "number", width: "w-32" },
              { key: "unit", label: "Unit", width: "w-24" },
              { key: "taxRate", label: "Tax %", type: "number", width: "w-24" },
            ]}
            rows={await db.item.findMany({ orderBy: { description: "asc" } })}
            save={saveLookup.bind(null, "item")}
            remove={deleteLookup}
          />
          <div className="grid gap-6 lg:grid-cols-2">
            <LookupTable title="Tax rates" table="tax" cols={[{ key: "name", label: "Name" }, { key: "rate",
label: "Rate %", type: "number", width: "w-28" }]} rows={await db.tax.findMany({ orderBy: { rate: "desc" }
})} save={saveLookup.bind(null, "tax")} remove={deleteLookup} />
            <LookupTable title="Payment modes" table="paymentMode" cols={[{ key: "name", label: "Name" }, {
key: "active", label: "Active", type: "checkbox" }]} rows={await db.paymentMode.findMany()} save=
{saveLookup.bind(null, "paymentMode")} remove={deleteLookup} />
          </div>
        </div>
      )}

      {tab === "leads" && (
        <div className="grid gap-6 lg:grid-cols-2">
          <LookupTable
            title="Lead statuses"
            table="leadStatus"
            hint="These are the columns on the leads kanban board. Statuses in use can't be deleted."
            cols={[{ key: "name", label: "Name" }, { key: "color", label: "Color", type: "color", width: "w-
24" }, { key: "order", label: "Order", type: "number", width: "w-20" }]}
            rows={await db.leadStatus.findMany({ orderBy: { order: "asc" } })}
            save={saveLookup.bind(null, "leadStatus")}
            remove={deleteLookup}
          />
          <LookupTable title="Lead sources" table="leadSource" cols={[{ key: "name", label: "Name" }]} rows=
{await db.leadSource.findMany({ orderBy: { name: "asc" } })} save={saveLookup.bind(null, "leadSource")}
remove={deleteLookup} />
        </div>
      )}

      {tab === "lists" && (
```

```
        <div className="grid gap-6 lg:grid-cols-2">
          <LookupTable title="Expense categories" table="expenseCategory" cols={[{ key: "name", label: "Name"
}]} rows={await db.expenseCategory.findMany({ orderBy: { name: "asc" } })} save={saveLookup.bind(null,
"expenseCategory")} remove={deleteLookup} />
          <LookupTable title="Support departments" table="department" cols={[{ key: "name", label: "Name" },
{ key: "email", label: "Email" }]} rows={await db.department.findMany()} save={saveLookup.bind(null,
"department")} remove={deleteLookup} />
        </div>
      )}
    </>
  );
}
```

src/app/admin/shared-actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { db } from "@/lib/db";
import { requireUser } from "@/lib/auth";

export async function addNote(f: FormData) {
  const user = await requireUser();
  const content = String(f.get("content") || "").trim();
  const relType = String(f.get("relType"));
  const relId = Number(f.get("relId"));
  if (content && relId) await db.note.create({ data: { relType, relId, content, staffId: user.id } });
  revalidatePath(String(f.get("path") || "/admin").split("?")[0]);
}

export async function deleteNote(f: FormData) {
  const user = await requireUser();
  const id = Number(f.get("id"));
  const note = await db.note.findUnique({ where: { id } });
  if (note && (user.isAdmin || note.staffId === user.id)) await db.note.delete({ where: { id } });
  revalidatePath(String(f.get("path") || "/admin").split("?")[0]);
}
```

src/app/admin/staff/[id]/page.tsx

```
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { ConfirmButton } from "@/components/client";
import { StaffForm } from "../form";
import { deleteStaff, saveStaff } from "../actions";

export default async function EditStaff({ params }: { params: Promise<{ id: string }> }) {
  const user = await requirePerm("staff", "edit");
  const { id } = await params;
  const s = await db.staff.findUnique({ where: { id: Number(id) } });
  if (!s) notFound();
  return (
    <div className="max-w-3xl">
      <PageHeader
        title={`${s.firstName} ${s.lastName}`}
        back={{ href: "/admin/staff", label: "Staff" }}
        actions={can(user, "staff", "delete") && s.id !== user.id && s.active && <ConfirmButton action=
{deleteStaff} hidden={{ id: s.id }} label="Deactivate" />}
      />
      <StaffForm action={saveStaff.bind(null, s.id)} staff={s} canSetAdmin={user.isAdmin} />
    </div>
  );
}
```

src/app/admin/staff/actions.ts

```
"use server";
import bcrypt from "bcryptjs";
```

```
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { requirePerm, requireAdmin } from "@/lib/auth";
import { logActivity } from "@/lib/activity";
import { ACTIONS, MODULES } from "@/lib/permissions";
import { str, reqStr, intOrNull, bool, type ActionState } from "@/lib/forms";

export async function saveStaff(id: number | null, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("staff", id ? "edit" : "create");
  const email = reqStr(f, "email").toLowerCase();
  const password = str(f, "password");
  const data = {
    firstName: reqStr(f, "firstName"),
    lastName: reqStr(f, "lastName"),
    email,
    phone: str(f, "phone"),
    roleId: intOrNull(f, "roleId"),
    isAdmin: user.isAdmin ? bool(f, "isAdmin") : false,
    active: bool(f, "active"),
  };
  if (!data.firstName || !email) return { error: "First name and email are required." };
  const dupe = await db.staff.findFirst({ where: { email, ...(id ? { NOT: { id } } : {}) } });
  if (dupe) return { error: "Another staff member already uses this email." };
  if (!id && (!password || password.length < 6)) return { error: "Password must be at least 6 characters." };
  if (password && password.length < 6) return { error: "Password must be at least 6 characters." };
  if (id === user.id && (!data.active || (user.isAdmin && !data.isAdmin))) return { error: "You can't
deactivate or demote your own account." };
  if (id) {
    const existing = await db.staff.findUnique({ where: { id } });
    if (existing?.isAdmin && !user.isAdmin) return { error: "Only administrators can edit an administrator."
};
    if (!user.isAdmin) data.isAdmin = existing?.isAdmin ?? false;
  }
  const s = id
    ? await db.staff.update({ where: { id }, data: { ...data, ...(password ? { password: await
bcrypt.hash(password, 10) } : {}) } })
    : await db.staff.create({ data: { ...data, password: await bcrypt.hash(password!, 10) } });
  await logActivity("staff", s.id, `${id ? "updated" : "added"} staff member ${s.firstName} ${s.lastName}`,
user.id);
  redirect("/admin/staff");
}

export async function deleteStaff(f: FormData) {
  const user = await requirePerm("staff", "delete");
  const id = Number(f.get("id"));
  if (id === user.id) redirect("/admin/staff");
  const s = await db.staff.findUnique({ where: { id } });
  if (s?.isAdmin && !user.isAdmin) redirect("/admin/forbidden");
  // Keep history intact: deactivate rather than hard-delete
  await db.staff.update({ where: { id }, data: { active: false } });
  redirect("/admin/staff");
}

export async function saveRole(id: number | null, _: ActionState, f: FormData): Promise<ActionState> {
  await requireAdmin();
  const name = reqStr(f, "name");
  if (!name) return { error: "Role name is required." };
  const perms: Record<string, string[]> = {};
  for (const m of MODULES) {
    const acts = ACTIONS.filter((a) => f.get(`${m.key}:${a}`) === "on");
    if (acts.length) perms[m.key] = acts.includes("view") ? acts : ["view", ...acts];
  }
  const data = { name, permissions: JSON.stringify(perms) };
  try {
    if (id) await db.role.update({ where: { id }, data });
    else await db.role.create({ data });
  } catch {
    return { error: "A role with this name already exists." };
  }
  redirect("/admin/staff/roles");
}

export async function deleteRole(f: FormData) {
  await requireAdmin();
  await db.role.delete({ where: { id: Number(f.get("id")) } });
```

```
  redirect("/admin/staff/roles");
}
```

src/app/admin/staff/form.tsx

```
import type { Staff } from "@prisma/client";
import { db } from "@/lib/db";
import type { ActionState } from "@/lib/forms";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field } from "@/components/ui";

export async function StaffForm({ action, staff, canSetAdmin }: { action: (s: ActionState, f: FormData) =>
Promise<ActionState>; staff?: Staff; canSetAdmin: boolean }) {
  const roles = await db.role.findMany({ orderBy: { name: "asc" } });
  const s = staff;
  return (
    <ActionForm action={action}>
      <Card>
        <div className="grid gap-4 p-5 sm:grid-cols-2">
          <Field label="First name *"><input name="firstName" required defaultValue={s?.firstName}
className="input" /></Field>
          <Field label="Last name"><input name="lastName" defaultValue={s?.lastName} className="input" />
</Field>
          <Field label="Email *"><input name="email" type="email" required defaultValue={s?.email}
className="input" /></Field>
          <Field label="Phone"><input name="phone" defaultValue={s?.phone ?? ""} className="input" /></Field>
          <Field label="Role">
            <select name="roleId" defaultValue={s?.roleId ?? ""} className="input">
              <option value="">— No role —</option>
              {roles.map((r) => <option key={r.id} value={r.id}>{r.name}</option>)}
            </select>
          </Field>
          <Field label={s ? "New password" : "Password *"} hint={s ? "Leave blank to keep the current
password" : "At least 6 characters"}>
            <input name="password" type="password" autoComplete="new-password" className="input" />
          </Field>
          <div className="flex flex-wrap gap-6 sm:col-span-2">
            {canSetAdmin && <label className="flex items-center gap-2 text-sm"><input type="checkbox"
name="isAdmin" defaultChecked={s?.isAdmin} /> Administrator (full access)</label>}
            <label className="flex items-center gap-2 text-sm"><input type="checkbox" name="active"
defaultChecked={s?.active ?? true} /> Active</label>
          </div>
        </div>
        <div className="flex justify-end border-t border-slate-100 p-4"><SubmitButton>{s ? "Save" : "Add
staff member"}</SubmitButton></div>
      </Card>
    </ActionForm>
  );
}
```

src/app/admin/staff/new/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { PageHeader } from "@/components/ui";
import { StaffForm } from "../form";
import { saveStaff } from "../actions";

export const metadata = { title: "Add staff" };
export default async function NewStaff() {
  const user = await requirePerm("staff", "create");
  return (
    <div className="max-w-3xl">
      <PageHeader title="Add staff member" back={{ href: "/admin/staff", label: "Staff" }} />
      <StaffForm action={saveStaff.bind(null, null)} canSetAdmin={user.isAdmin} />
    </div>
  );
}
```

src/app/admin/staff/page.tsx

```
import Link from "next/link";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { fmtDateTime } from "@/lib/format";
import { Avatar, Badge, ButtonLink, Card, PageHeader, Table } from "@/components/ui";

export const metadata = { title: "Staff" };

export default async function StaffPage() {
  const user = await requirePerm("staff");
  const staff = await db.staff.findMany({ orderBy: [{ active: "desc" }, { firstName: "asc" }], include: {
role: true, _count: { select: { tasks: { where: { status: { not: "COMPLETE" } } } } } } });
  return (
    <>
      <PageHeader
        title="Staff"
        subtitle={`${staff.filter((s) => s.active).length} active members`}
        actions={<>{user.isAdmin && <ButtonLink variant="secondary" href="/admin/staff/roles">Roles &
permissions</ButtonLink>}<ButtonLink href="/admin/staff/new">+ Add staff</ButtonLink></>}
      />
      <Card>
        <Table head={["Name", "Email", "Role", "Open tasks", "Last login", "Status"]}>
          {staff.map((s) => (
            <tr key={s.id} className="hover:bg-slate-50/60">
              <td className="td">
                <Link href={`/admin/staff/${s.id}`} className="flex items-center gap-3 font-medium text-
slate-800 hover:text-primary">
                  <Avatar name={`${s.firstName} ${s.lastName}`} size={30} /> {s.firstName} {s.lastName}
                </Link>
              </td>
              <td className="td">{s.email}</td>
              <td className="td">{s.isAdmin ? <Badge tone="purple">Administrator</Badge> : s.role?.name ??
"—"}</td>
              <td className="td">{s._count.tasks}</td>
              <td className="td text-slate-500">{s.lastLogin ? fmtDateTime(s.lastLogin) : "Never"}</td>
              <td className="td">{s.active ? <Badge tone="green">Active</Badge> : <Badge>Inactive</Badge>}
</td>
            </tr>
          ))}
        </Table>
      </Card>
    </>
  );
}
```

src/app/admin/staff/roles/[id]/page.tsx

```
import { notFound } from "next/navigation";
import { requireAdmin } from "@/lib/auth";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { ConfirmButton } from "@/components/client";
import { RoleForm } from "../role-form";
import { deleteRole, saveRole } from "../../actions";

export default async function EditRole({ params }: { params: Promise<{ id: string }> }) {
  await requireAdmin();
  const { id } = await params;
  const role = await db.role.findUnique({ where: { id: Number(id) } });
  if (!role) notFound();
  return (
    <div className="max-w-4xl">
      <PageHeader title={role.name} back={{ href: "/admin/staff/roles", label: "Roles" }} actions=
{<ConfirmButton action={deleteRole} hidden={{ id: role.id }} />} />
      <RoleForm action={saveRole.bind(null, role.id)} role={role} />
    </div>
  );
}
```

src/app/admin/staff/roles/new/page.tsx

```
import { requireAdmin } from "@/lib/auth";
import { PageHeader } from "@/components/ui";
import { RoleForm } from "../role-form";
import { saveRole } from "../../actions";

export default async function NewRole() {
  await requireAdmin();
  return (
    <div className="max-w-4xl">
      <PageHeader title="New role" back={{ href: "/admin/staff/roles", label: "Roles" }} />
      <RoleForm action={saveRole.bind(null, null)} />
    </div>
  );
}
```

src/app/admin/staff/roles/page.tsx

```
import Link from "next/link";
import { requireAdmin } from "@/lib/auth";
import { db } from "@/lib/db";
import { MODULES, parsePermissions } from "@/lib/permissions";
import { ButtonLink, Card, PageHeader, Table } from "@/components/ui";

export const metadata = { title: "Roles" };
export default async function RolesPage() {
  await requireAdmin();
  const roles = await db.role.findMany({ orderBy: { name: "asc" }, include: { _count: { select: { staff: true
} } } });
  return (
    <>
      <PageHeader title="Roles & permissions" subtitle="Administrators always have full access" back={{ href:
"/admin/staff", label: "Staff" }} actions={<ButtonLink href="/admin/staff/roles/new">+ New role</ButtonLink>}
/>
      <Card>
        <Table head={["Role", "Staff", "Modules"]}>
          {roles.map((r) => {
            const p = parsePermissions(r.permissions);
            return (
              <tr key={r.id}>
                <td className="td"><Link className="link" href={`/admin/staff/roles/${r.id}`}>{r.name}</Link>
</td>
                <td className="td">{r._count.staff}</td>
                <td className="td text-xs text-slate-500">{MODULES.filter((m) => p[m.key]?.length).map((m) =>
m.label).join(", ") || "No access"}</td>
              </tr>
            );
          })}
        </Table>
      </Card>
    </>
  );
}
```

src/app/admin/staff/roles/role-form.tsx

```
import type { Role } from "@prisma/client";
import { ACTIONS, MODULES, parsePermissions } from "@/lib/permissions";
import type { ActionState } from "@/lib/forms";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field } from "@/components/ui";

export function RoleForm({ action, role }: { action: (s: ActionState, f: FormData) => Promise<ActionState>;
role?: Role }) {
  const p = parsePermissions(role?.permissions);
  return (
    <ActionForm action={action}>
      <Card>
        <div className="p-5"><Field label="Role name *"><input name="name" required defaultValue={role?.name}
className="input max-w-sm" /></Field></div>
```

```
        <table className="min-w-full border-t border-slate-100">
          <thead className="bg-slate-50/70">
            <tr><th className="th">Module</th>{ACTIONS.map((a) => <th key={a} className="th text-center
capitalize">{a}</th>)}</tr>
          </thead>
          <tbody className="divide-y divide-slate-100">
            {MODULES.map((m) => (
              <tr key={m.key}>
                <td className="td font-medium">{m.label}</td>
                {ACTIONS.map((a) => (
                  <td key={a} className="td text-center"><input type="checkbox" name={`${m.key}:${a}`}
defaultChecked={p[m.key]?.includes(a)} className="h-4 w-4" /></td>
                ))}
              </tr>
            ))}
          </tbody>
        </table>
        <div className="flex justify-end border-t border-slate-100 p-4"><SubmitButton>Save
role</SubmitButton></div>
      </Card>
    </ActionForm>
  );
}
```

src/app/admin/tasks/[id]/edit/page.tsx

```
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { PageHeader } from "@/components/ui";
import { TaskForm } from "../../form";
import { saveTask } from "../../actions";

export default async function EditTask({ params }: { params: Promise<{ id: string }> }) {
  await requirePerm("tasks", "edit");
  const { id } = await params;
  const t = await db.task.findUnique({ where: { id: Number(id) }, include: { assignees: true } });
  if (!t) notFound();
  return (
    <div className="max-w-4xl">
      <PageHeader title={`Edit task`} back={{ href: `/admin/tasks/${t.id}`, label: t.name }} />
      <TaskForm action={saveTask.bind(null, t.id)} task={t} />
    </div>
  );
}
```

src/app/admin/tasks/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { fmtDate, fmtDateTime, formatDuration, timeAgo } from "@/lib/format";
import { PRIORITY, TASK_STATUS } from "@/lib/constants";
import { Avatar, ButtonLink, Card, DetailRow, PageHeader, StatusBadge, btn, cx } from "@/components/ui";
import { ActionForm, ConfirmButton, SubmitButton } from "@/components/client";
import { addComment, deleteTask, deleteTimeEntry, setTaskStatusForm, toggleTimer } from "../actions";

export default async function TaskPage({ params }: { params: Promise<{ id: string }> }) {
  const user = await requirePerm("tasks");
  const { id } = await params;
  const t = await db.task.findUnique({
    where: { id: Number(id) },
    include: {
      project: true,
      customer: true,
      assignees: true,
      createdBy: true,
      comments: { orderBy: { createdAt: "asc" }, include: { staff: true } },
      timeEntries: { orderBy: { startTime: "desc" }, include: { staff: true } },
    },
```

```
  });
  if (!t) notFound();
  const canEdit = can(user, "tasks", "edit");
  const myRunning = t.timeEntries.find((e) => !e.endTime && e.staffId === user.id);
  const totalMs = t.timeEntries.reduce((s, e) => s + ((e.endTime ?? new Date()).getTime() -
e.startTime.getTime()), 0);

  return (
    <>
      <PageHeader
        title={t.name}
        subtitle={t.project ? <>Project: <Link className="link" href={`/admin/projects/${t.project.id}`}>
{t.project.name}</Link></> : t.customer?.company}
        back={{ href: t.project ? `/admin/projects/${t.project.id}?tab=tasks` : "/admin/tasks", label:
t.project ? t.project.name : "Tasks" }}
        actions={
          <>
            <form action={toggleTimer}>
              <input type="hidden" name="taskId" value={t.id} />
              <button className={btn(myRunning ? "danger" : "success")}>{myRunning ? "■ Stop timer" : "▶
Start timer"}</button>
            </form>
            {canEdit && <ButtonLink variant="secondary" href={`/admin/tasks/${t.id}/edit`}>Edit</ButtonLink>}
            {can(user, "tasks", "delete") && <ConfirmButton action={deleteTask} hidden={{ id: t.id }} />}
          </>
        }
      />
      <div className="grid gap-6 lg:grid-cols-3">
        <div className="space-y-6 lg:col-span-2">
          <Card title="Description">
            <p className="whitespace-pre-wrap p-5 text-sm text-slate-700">{t.description || <span
className="text-slate-400">No description.</span>}</p>
          </Card>
          {canEdit && (
            <Card title="Status">
              <div className="flex flex-wrap gap-2 p-4">
                {Object.entries(TASK_STATUS).map(([k, v]) => (
                  <form key={k} action={setTaskStatusForm}>
                    <input type="hidden" name="id" value={t.id} />
                    <input type="hidden" name="status" value={k} />
                    <button className={cx(btn(t.status === k ? "primary" : "secondary"), "text-xs")}>
{v.label}</button>
                  </form>
                ))}
              </div>
            </Card>
          )}
          <Card title={`Comments (${t.comments.length})`}>
            <ul className="divide-y divide-slate-100">
              {t.comments.map((c) => (
                <li key={c.id} className="flex gap-3 px-5 py-4">
                  <Avatar name={c.staff ? `${c.staff.firstName} ${c.staff.lastName}` : "?"} size={30} />
                  <div>
                    <p className="text-xs text-slate-500"><span className="font-semibold text-slate-700">
{c.staff?.firstName} {c.staff?.lastName}</span> · {timeAgo(c.createdAt)}</p>
                    <p className="mt-1 whitespace-pre-wrap text-sm text-slate-700">{c.content}</p>
                  </div>
                </li>
              ))}
            </ul>
            <ActionForm action={addComment.bind(null, t.id)} resetOnSuccess className="space-y-2 border-t
border-slate-100 p-4">
              <textarea name="content" rows={3} required placeholder="Add a comment…" className="input" />
              <div className="flex justify-end"><SubmitButton>Comment</SubmitButton></div>
            </ActionForm>
          </Card>
        </div>
        <div className="space-y-6">
          <Card title="Details">
            <dl className="divide-y divide-slate-100 px-5">
              <DetailRow label="Status"><StatusBadge map={TASK_STATUS} value={t.status} /></DetailRow>
              <DetailRow label="Priority"><StatusBadge map={PRIORITY} value={t.priority} /></DetailRow>
              <DetailRow label="Start date">{fmtDate(t.startDate)}</DetailRow>
              <DetailRow label="Due date">{fmtDate(t.dueDate)}</DetailRow>
              <DetailRow label="Billable">{t.billable ? "Yes" : "No"}</DetailRow>
```

```
              <DetailRow label="Created by">{t.createdBy ? `${t.createdBy.firstName} ${t.createdBy.lastName}`
: null}</DetailRow>
              {t.completedAt && <DetailRow label="Completed">{fmtDateTime(t.completedAt)}</DetailRow>}
            </dl>
          </Card>
          <Card title="Assignees">
            <ul className="divide-y divide-slate-100">
              {t.assignees.map((a) => <li key={a.id} className="flex items-center gap-3 px-5 py-2.5 text-sm">
<Avatar name={`${a.firstName} ${a.lastName}`} size={28} /> {a.firstName} {a.lastName}</li>)}
              {t.assignees.length === 0 && <li className="px-5 py-4 text-sm text-slate-400">Nobody assigned.
</li>}
            </ul>
          </Card>
          <Card title={`Timesheet · ${formatDuration(totalMs)}`}>
            <ul className="divide-y divide-slate-100">
              {t.timeEntries.length === 0 && <li className="px-5 py-4 text-sm text-slate-400">No time logged
yet.</li>}
              {t.timeEntries.map((e) => (
                <li key={e.id} className="flex items-center justify-between gap-2 px-5 py-2.5 text-sm">
                  <div>
                    <p className="font-medium">{e.staff.firstName} · {e.endTime ?
formatDuration(e.endTime.getTime() - e.startTime.getTime()) : <span className="text-brand-green">running…
</span>}</p>
                    <p className="text-xs text-slate-500">{fmtDateTime(e.startTime)}</p>
                  </div>
                  {(user.isAdmin || e.staffId === user.id) && <ConfirmButton action={deleteTimeEntry} hidden=
{{ id: e.id }} label="×" confirmLabel="Delete?" variant="ghost" className="px-2 py-1 text-xs" />}
                </li>
              ))}
            </ul>
          </Card>
        </div>
      </div>
    </>
  );
}
```

src/app/admin/tasks/actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { requirePerm } from "@/lib/auth";
import { logActivity } from "@/lib/activity";
import { str, reqStr, intOrNull, dateOrNull, ints, bool, type ActionState } from "@/lib/forms";

function data(f: FormData) {
  return {
    name: reqStr(f, "name"),
    description: str(f, "description"),
    priority: reqStr(f, "priority") || "MEDIUM",
    status: reqStr(f, "status") || "NOT_STARTED",
    startDate: dateOrNull(f, "startDate") ?? new Date(),
    dueDate: dateOrNull(f, "dueDate"),
    projectId: intOrNull(f, "projectId"),
    customerId: intOrNull(f, "customerId"),
    billable: bool(f, "billable"),
  };
}

async function syncProjectProgress(projectId: number | null) {
  if (!projectId) return;
  const tasks = await db.task.findMany({ where: { projectId }, select: { status: true } });
  if (!tasks.length) return;
  const progress = Math.round((tasks.filter((t) => t.status === "COMPLETE").length / tasks.length) * 100);
  await db.project.update({ where: { id: projectId }, data: { progress } });
}

export async function saveTask(id: number | null, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("tasks", id ? "edit" : "create");
  const d = data(f);
  if (!d.name) return { error: "Task name is required." };
```

```
  if (d.projectId && !d.customerId) d.customerId = (await db.project.findUnique({ where: { id: d.projectId } }))?.customerId ?? null;
  const assignees = ints(f, "assignees").map((x) => ({ id: x }));
  const completedAt = d.status === "COMPLETE" ? new Date() : null;
  const t = id
    ? await db.task.update({ where: { id }, data: { ...d, completedAt, assignees: { set: assignees } } })
    : await db.task.create({ data: { ...d, completedAt, createdById: user.id, assignees: { connect: assignees } } });
  await syncProjectProgress(t.projectId);
  await logActivity("task", t.id, `${id ? "updated" : "created"} task ${t.name}`, user.id);
  redirect(`/admin/tasks/${t.id}`);
}

export async function setTaskStatus(id: number, status: string) {
  const user = await requirePerm("tasks", "edit");
  const t = await db.task.update({ where: { id }, data: { status, completedAt: status === "COMPLETE" ? new Date() : null } });
  await syncProjectProgress(t.projectId);
  await logActivity("task", id, `moved task ${t.name} to ${status.replace(/_/g, " ").toLowerCase()}`, user.id);
  revalidatePath("/admin/tasks");
  revalidatePath(`/admin/tasks/${id}`);
}

export async function setTaskStatusForm(f: FormData) {
  await setTaskStatus(Number(f.get("id")), String(f.get("status")));
}

export async function addComment(taskId: number, _: unknown, f: FormData) {
  const user = await requirePerm("tasks");
  const content = reqStr(f, "content");
  if (content) await db.taskComment.create({ data: { taskId, staffId: user.id, content } });
  revalidatePath(`/admin/tasks/${taskId}`);
  return { ok: true };
}

export async function toggleTimer(f: FormData) {
  const user = await requirePerm("tasks");
  const taskId = Number(f.get("taskId"));
  const running = await db.timeEntry.findFirst({ where: { staffId: user.id, endTime: null } });
  if (running) await db.timeEntry.update({ where: { id: running.id }, data: { endTime: new Date() } });
  if (!running || running.taskId !== taskId) await db.timeEntry.create({ data: { taskId, staffId: user.id, startTime: new Date() } });
  revalidatePath(`/admin/tasks/${taskId}`);
  if (running && running.taskId !== taskId) revalidatePath(`/admin/tasks/${running.taskId}`);
}

export async function deleteTimeEntry(f: FormData) {
  const user = await requirePerm("tasks", "edit");
  const id = Number(f.get("id"));
  const e = await db.timeEntry.findUnique({ where: { id } });
  if (e && (user.isAdmin || e.staffId === user.id)) await db.timeEntry.delete({ where: { id } });
  if (e) revalidatePath(`/admin/tasks/${e.taskId}`);
}

export async function deleteTask(f: FormData) {
  const user = await requirePerm("tasks", "delete");
  const id = Number(f.get("id"));
  const t = await db.task.delete({ where: { id } });
  await syncProjectProgress(t.projectId);
  await logActivity("task", id, `deleted task ${t.name}`, user.id);
  redirect(t.projectId ? `/admin/projects/${t.projectId}?tab=tasks` : "/admin/tasks");
}
```

src/app/admin/tasks/form.tsx

```
import type { Task, Staff } from "@prisma/client";
import { db } from "@/lib/db";
import { inputDate } from "@/lib/format";
import { PRIORITY, TASK_STATUS } from "@/lib/constants";
import type { ActionState } from "@/lib/forms";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field } from "@/components/ui";
```

```
export async function TaskForm({
  action,
  task,
  defaults,
}: {
  action: (s: ActionState, f: FormData) => Promise<ActionState>;
  task?: Task & { assignees: Staff[] };
  defaults?: { projectId?: number; assigneeId?: number };
}) {
  const [projects, customers, staff] = await Promise.all([
    db.project.findMany({ where: { status: { notIn: ["CANCELLED", "FINISHED"] } }, orderBy: { name: "asc" },
select: { id: true, name: true } }),
    db.customer.findMany({ where: { active: true }, orderBy: { company: "asc" }, select: { id: true, company:
true } }),
    db.staff.findMany({ where: { active: true }, orderBy: { firstName: "asc" } }),
  ]);
  const t = task;
  const assigned = new Set(t ? t.assignees.map((a) => a.id) : defaults?.assigneeId ? [defaults.assigneeId] :
[]);
  return (
    <ActionForm action={action}>
      <Card>
        <div className="grid gap-4 p-5 sm:grid-cols-2">
          <Field label="Task name *" className="sm:col-span-2"><input name="name" required defaultValue=
{t?.name} className="input" /></Field>
          <Field label="Project">
            <select name="projectId" defaultValue={t?.projectId ?? defaults?.projectId ?? ""}
className="input">
              <option value="">— No project —</option>
              {projects.map((p) => <option key={p.id} value={p.id}>{p.name}</option>)}
            </select>
          </Field>
          <Field label="Customer" hint="Auto-filled from the project if left empty">
            <select name="customerId" defaultValue={t?.customerId ?? ""} className="input">
              <option value="">—</option>
              {customers.map((c) => <option key={c.id} value={c.id}>{c.company}</option>)}
            </select>
          </Field>
          <Field label="Priority">
            <select name="priority" defaultValue={t?.priority ?? "MEDIUM"} className="input">
              {Object.entries(PRIORITY).map(([k, v]) => <option key={k} value={k}>{v.label}</option>)}
            </select>
          </Field>
          <Field label="Status">
            <select name="status" defaultValue={t?.status ?? "NOT_STARTED"} className="input">
              {Object.entries(TASK_STATUS).map(([k, v]) => <option key={k} value={k}>{v.label}</option>)}
            </select>
          </Field>
          <Field label="Start date"><input name="startDate" type="date" defaultValue={inputDate(t?.startDate
?? new Date())} className="input" /></Field>
          <Field label="Due date"><input name="dueDate" type="date" defaultValue={inputDate(t?.dueDate)}
className="input" /></Field>
          <Field label="Description" className="sm:col-span-2"><textarea name="description" rows={5}
defaultValue={t?.description ?? ""} className="input" /></Field>
          <div className="sm:col-span-2">
            <span className="label">Assignees</span>
            <div className="flex flex-wrap gap-2">
              {staff.map((s) => (
                <label key={s.id} className="flex items-center gap-2 rounded-lg border border-slate-200 px-3
py-1.5 text-sm has-[:checked]:border-primary has-[:checked]:bg-blue-50">
                  <input type="checkbox" name="assignees" value={s.id} defaultChecked={assigned.has(s.id)} />
{s.firstName} {s.lastName}
                </label>
              ))}
            </div>
          </div>
          <label className="flex items-center gap-2 text-sm"><input type="checkbox" name="billable"
defaultChecked={t?.billable} /> Billable</label>
        </div>
        <div className="flex justify-end border-t border-slate-100 p-4"><SubmitButton>{t ? "Save task" :
"Create task"}</SubmitButton></div>
      </Card>
    </ActionForm>
```

```
  );
}
```

src/app/admin/tasks/new/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { PageHeader } from "@/components/ui";
import { TaskForm } from "../form";
import { saveTask } from "../actions";

export const metadata = { title: "New task" };
export default async function NewTask({ searchParams }: { searchParams: Promise<{ projectId?: string }> }) {
  const user = await requirePerm("tasks", "create");
  const sp = await searchParams;
  return (
    <div className="max-w-4xl">
      <PageHeader title="New task" back={{ href: "/admin/tasks", label: "Tasks" }} />
      <TaskForm action={saveTask.bind(null, null)} defaults={{ projectId: sp.projectId ? Number(sp.projectId)
: undefined, assigneeId: user.id }} />
    </div>
  );
}
```

src/app/admin/tasks/page.tsx

```
import Link from "next/link";
import type { Prisma } from "@prisma/client";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { fmtDate } from "@/lib/format";
import { PRIORITY, TASK_STATUS } from "@/lib/constants";
import { Avatar, ButtonLink, Card, EmptyState, PageHeader, StatusBadge, Table, btn, cx } from
"@/components/ui";
import { FilterSelect, SearchBox } from "@/components/client";
import { Pagination, pageArgs } from "@/components/pagination";
import { Kanban } from "@/components/kanban";
import { setTaskStatus } from "./actions";

export const metadata = { title: "Tasks" };
const COLORS: Record<string, string> = { NOT_STARTED: "#94a3b8", IN_PROGRESS: "#4f74c4", TESTING: "#7c5cc4",
AWAITING_FEEDBACK: "#e0a400", COMPLETE: "#27ae57" };

export default async function TasksPage({ searchParams }: { searchParams: Promise<Record<string, string |
undefined>> }) {
  const user = await requirePerm("tasks");
  const sp = await searchParams;
  const view = sp.view === "kanban" ? "kanban" : "list";
  const where: Prisma.TaskWhereInput = {};
  if (sp.mine === "1") where.assignees = { some: { id: user.id } };
  if (sp.status) where.status = sp.status;
  else if (view === "list" && sp.all !== "1") where.status = { not: "COMPLETE" };
  if (sp.priority) where.priority = sp.priority;
  if (sp.project) where.projectId = Number(sp.project);
  if (sp.q) where.name = { contains: sp.q };
  const projects = await db.project.findMany({ orderBy: { name: "asc" }, select: { id: true, name: true } });
  const today = new Date(new Date().toDateString());

  const toggle = (key: string, val: string) => {
    const p = new URLSearchParams(Object.entries(sp).filter(([k, v]) => v && k !== "page") as [string,
string][]);
    if (p.get(key) === val) p.delete(key);
    else p.set(key, val);
    return `?${p}`;
  };
  const viewHref = (v: string | null) => {
    const p = new URLSearchParams(Object.entries(sp).filter(([k, x]) => x && k !== "page" && k !== "view" &&
k !== "status") as [string, string][]);
    if (v) p.set("view", v);
    return `?${p}`;
  };
  const header = (
    <PageHeader
```

```
      title="Tasks"
      actions={
        <>
          <div className="flex rounded-lg border border-slate-300 bg-white p-0.5">
            <Link href={viewHref(null)} className={cx("rounded-md px-3 py-1.5 text-sm", view === "list" ?
"bg-slate-100 font-medium" : "text-slate-500")}>List</Link>
            <Link href={viewHref("kanban")} className={cx("rounded-md px-3 py-1.5 text-sm", view === "kanban"
? "bg-slate-100 font-medium" : "text-slate-500")}>Kanban</Link>
          </div>
          <ButtonLink href={`/admin/tasks/new${sp.project ? `?projectId=${sp.project}` : ""}`}>+ New
task</ButtonLink>
        </>
      }
    />
  );
  const filters = (
    <div className="flex flex-wrap gap-2">
      <SearchBox placeholder="Search tasks…" />
      <FilterSelect name="project" placeholder="All projects" options={projects.map((p) => ({ value:
String(p.id), label: p.name }))} />
      <FilterSelect name="priority" placeholder="Any priority" options={Object.entries(PRIORITY).map(([value,
s]) => ({ value, label: s.label }))} />
      {view === "list" && <FilterSelect name="status" placeholder="Open tasks" options=
{Object.entries(TASK_STATUS).map(([value, s]) => ({ value, label: s.label }))} />}
      <Link href={toggle("mine", "1")} className={btn(sp.mine === "1" ? "primary" : "secondary")}>My
tasks</Link>
    </div>
  );

  if (view === "kanban") {
    const tasks = await db.task.findMany({ where, orderBy: [{ dueDate: "asc" }], include: { project: {
select: { name: true } }, assignees: true } });
    return (
      <>
        {header}
        <div className="mb-4">{filters}</div>
        <Kanban
          columns={Object.entries(TASK_STATUS).map(([id, s]) => ({ id, name: s.label, color: COLORS[id] }))}
          cards={tasks.map((t) => ({
            id: t.id,
            columnId: t.status,
            title: t.name,
            subtitle: t.project?.name,
            meta: [PRIORITY[t.priority]?.label, t.dueDate ? `Due ${fmtDate(t.dueDate)}` : null,
t.assignees.map((a) => a.firstName).join(", ")].filter(Boolean).join(" · "),
            href: `/admin/tasks/${t.id}`,
            accent: { LOW: "#94a3b8", MEDIUM: "#4f74c4", HIGH: "#e0a400", URGENT: "#e84a37" }[t.priority],
          }))}
          onMove={setTaskStatus}
        />
      </>
    );
  }

  const { page, skip, take } = pageArgs(sp.page);
  const [rows, total] = await Promise.all([
    db.task.findMany({ where, skip, take, orderBy: [{ dueDate: "asc" }, { createdAt: "desc" }], include: {
project: { select: { id: true, name: true } }, assignees: true } }),
    db.task.count({ where }),
  ]);
  return (
    <>
      {header}
      <Card>
        <div className="border-b border-slate-100 p-4">{filters}</div>
        {rows.length === 0 ? (
          <EmptyState title="No tasks found" action={<ButtonLink href="/admin/tasks/new">Add
task</ButtonLink>} />
        ) : (
          <Table head={["Task", "Project", "Assignees", "Priority", "Start", "Due", "Status"]}>
            {rows.map((t) => (
              <tr key={t.id} className="hover:bg-slate-50/60">
                <td className="td"><Link className="link" href={`/admin/tasks/${t.id}`}>{t.name}</Link></td>
                <td className="td">{t.project ? <Link className="hover:text-primary" href=
{`/admin/projects/${t.project.id}`}>{t.project.name}</Link> : "—"}</td>
```

```
                <td className="td"><div className="flex -space-x-2">{t.assignees.map((a) => <Avatar key=
{a.id} name={`${a.firstName} ${a.lastName}`} size={24} />)}</div></td>
                <td className="td"><StatusBadge map={PRIORITY} value={t.priority} /></td>
                <td className="td">{fmtDate(t.startDate)}</td>
                <td className={cx("td", t.dueDate && t.dueDate < today && t.status !== "COMPLETE" && "font-
semibold text-brand-red")}>{fmtDate(t.dueDate)}</td>
                <td className="td"><StatusBadge map={TASK_STATUS} value={t.status} /></td>
              </tr>
            ))}
          </Table>
        )}
        <Pagination total={total} page={page} params={sp} />
      </Card>
    </>
  );
}
```

src/app/admin/tickets/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { requirePerm } from "@/lib/auth";
import { can } from "@/lib/permissions";
import { db } from "@/lib/db";
import { fmtDateTime } from "@/lib/format";
import { PRIORITY, TICKET_STATUS } from "@/lib/constants";
import { Avatar, Card, DetailRow, Field, PageHeader, StatusBadge, btn } from "@/components/ui";
import { ActionForm, ConfirmButton, SubmitButton } from "@/components/client";
import { deleteTicket, replyTicket, updateTicketMeta } from "../actions";

export default async function TicketPage({ params }: { params: Promise<{ id: string }> }) {
  const user = await requirePerm("tickets");
  const { id } = await params;
  const t = await db.ticket.findUnique({
    where: { id: Number(id) },
    include: { customer: true, contact: true, department: true, assigned: true, project: true, replies: {
orderBy: { createdAt: "asc" }, include: { staff: true } } },
  });
  if (!t) notFound();
  const [depts, staff] = await Promise.all([db.department.findMany(), db.staff.findMany({ where: { active:
true } })]);
  const canEdit = can(user, "tickets", "edit");
  const from = t.contact ? `${t.contact.firstName} ${t.contact.lastName}` : t.name || "Customer";

  return (
    <>
      <PageHeader
        title={<>#{t.id} · {t.subject}</>}
        subtitle={t.customer ? <Link className="link" href={`/admin/customers/${t.customer.id}`}>
{t.customer.company}</Link> : t.email}
        back={{ href: "/admin/tickets", label: "Support" }}
        actions={<><StatusBadge map={TICKET_STATUS} value={t.status} /><StatusBadge map={PRIORITY} value=
{t.priority} />{can(user, "tickets", "delete") && <ConfirmButton action={deleteTicket} hidden={{ id: t.id }}
/>}</>}
      />
      <div className="grid gap-6 lg:grid-cols-3">
        <div className="space-y-4 lg:col-span-2">
          <div className="card p-5">
            <div className="mb-3 flex items-center gap-3">
              <Avatar name={from} />
              <div><p className="text-sm font-semibold">{from}</p><p className="text-xs text-slate-500">
{fmtDateTime(t.createdAt)}</p></div>
            </div>
            <p className="whitespace-pre-wrap text-sm text-slate-700">{t.message}</p>
          </div>
          {t.replies.map((r) => {
            const name = r.staff ? `${r.staff.firstName} ${r.staff.lastName}` : r.fromName || from;
            return (
              <div key={r.id} className={`card p-5 ${r.staff ? "ml-6 border-l-4 border-l-brand-blue" : ""}`}>
                <div className="mb-3 flex items-center gap-3">
                  <Avatar name={name} size={30} />
                  <div><p className="text-sm font-semibold">{name} {r.staff && <span className="ml-1 rounded
bg-blue-50 px-1.5 py-0.5 text-[10px] text-blue-700">Staff</span>}</p><p className="text-xs text-slate-500">
```

```
{fmtDateTime(r.createdAt)}</p></div>
                </div>
                <p className="whitespace-pre-wrap text-sm text-slate-700">{r.message}</p>
              </div>
            );
          })}
          {canEdit && (
            <Card title="Reply">
              <ActionForm action={replyTicket.bind(null, t.id)} resetOnSuccess className="space-y-3 p-5">
                <textarea name="message" rows={5} required className="input" placeholder="Type your reply…"
/>
                <div className="flex flex-wrap items-center justify-end gap-2">
                  <span className="text-xs text-slate-500">Set status to</span>
                  <select name="status" defaultValue="ANSWERED" className="input w-auto">
                    {Object.entries(TICKET_STATUS).map(([k, v]) => <option key={k} value={k}>{v.label}
</option>)}
                  </select>
                  <SubmitButton>Send reply</SubmitButton>
                </div>
              </ActionForm>
            </Card>
          )}
        </div>
        <div className="space-y-6">
          <Card title="Ticket info">
            <dl className="divide-y divide-slate-100 px-5">
              <DetailRow label="Contact">{t.contact ? `${t.contact.firstName} ${t.contact.lastName}` :
t.name}</DetailRow>
              <DetailRow label="Email">{t.contact?.email ?? t.email}</DetailRow>
              <DetailRow label="Project">{t.project && <Link className="link" href=
{`/admin/projects/${t.project.id}`}>{t.project.name}</Link>}</DetailRow>
              <DetailRow label="Opened">{fmtDateTime(t.createdAt)}</DetailRow>
              <DetailRow label="Last reply">{t.lastReply ? fmtDateTime(t.lastReply) : null}</DetailRow>
            </dl>
          </Card>
          {canEdit && (
            <Card title="Settings">
              <form action={updateTicketMeta} className="space-y-3 p-5">
                <input type="hidden" name="id" value={t.id} />
                <Field label="Status"><select name="status" defaultValue={t.status} className="input">
{Object.entries(TICKET_STATUS).map(([k, v]) => <option key={k} value={k}>{v.label}</option>)}</select>
</Field>
                <Field label="Priority"><select name="priority" defaultValue={t.priority} className="input">
{Object.entries(PRIORITY).map(([k, v]) => <option key={k} value={k}>{v.label}</option>)}</select></Field>
                <Field label="Department"><select name="departmentId" defaultValue={t.departmentId ?? ""}
className="input"><option value="">—</option>{depts.map((d) => <option key={d.id} value={d.id}>{d.name}
</option>)}</select></Field>
                <Field label="Assigned to"><select name="assignedId" defaultValue={t.assignedId ?? ""}
className="input"><option value="">Unassigned</option>{staff.map((s) => <option key={s.id} value={s.id}>
{s.firstName} {s.lastName}</option>)}</select></Field>
                <button className={btn("primary", "w-full")}>Update</button>
              </form>
            </Card>
          )}
        </div>
      </div>
    </>
  );
}
```

src/app/admin/tickets/actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { requirePerm } from "@/lib/auth";
import { logActivity } from "@/lib/activity";
import { str, reqStr, intOrNull, type ActionState } from "@/lib/forms";

export async function createTicket(_: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("tickets", "create");
  const subject = reqStr(f, "subject");
```

```
  const message = reqStr(f, "message");
  if (!subject || !message) return { error: "Subject and message are required." };
  const contactId = intOrNull(f, "contactId");
  let customerId = intOrNull(f, "customerId");
  if (contactId && !customerId) customerId = (await db.contact.findUnique({ where: { id: contactId }
}))?.customerId ?? null;
  const t = await db.ticket.create({
    data: {
      subject, message, customerId, contactId,
      name: str(f, "name"), email: str(f, "email"),
      departmentId: intOrNull(f, "departmentId"), projectId: intOrNull(f, "projectId"),
      priority: reqStr(f, "priority") || "MEDIUM", assignedId: intOrNull(f, "assignedId"),
    },
  });
  await logActivity("ticket", t.id, `opened ticket #${t.id} ${t.subject}`, user.id);
  redirect(`/admin/tickets/${t.id}`);
}

export async function replyTicket(ticketId: number, _: ActionState, f: FormData): Promise<ActionState> {
  const user = await requirePerm("tickets", "edit");
  const message = reqStr(f, "message");
  if (!message) return { error: "Reply cannot be empty." };
  const status = reqStr(f, "status") || "ANSWERED";
  await db.ticketReply.create({ data: { ticketId, staffId: user.id, message } });
  await db.ticket.update({ where: { id: ticketId }, data: { status, lastReply: new Date() } });
  revalidatePath(`/admin/tickets/${ticketId}`);
  return { ok: true };
}

export async function updateTicketMeta(f: FormData) {
  const user = await requirePerm("tickets", "edit");
  const id = Number(f.get("id"));
  await db.ticket.update({
    where: { id },
    data: { status: reqStr(f, "status"), priority: reqStr(f, "priority"), assignedId: intOrNull(f,
"assignedId"), departmentId: intOrNull(f, "departmentId") },
  });
  await logActivity("ticket", id, `updated ticket #${id}`, user.id);
  revalidatePath(`/admin/tickets/${id}`);
}

export async function deleteTicket(f: FormData) {
  await requirePerm("tickets", "delete");
  await db.ticket.delete({ where: { id: Number(f.get("id")) } });
  redirect("/admin/tickets");
}
```

src/app/admin/tickets/new/page.tsx

```
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { PRIORITY } from "@/lib/constants";
import { ActionForm, SubmitButton } from "@/components/client";
import { Card, Field, PageHeader } from "@/components/ui";
import { createTicket } from "../actions";

export const metadata = { title: "Open ticket" };

export default async function NewTicket() {
  const user = await requirePerm("tickets", "create");
  const [customers, depts, staff, projects] = await Promise.all([
    db.customer.findMany({ where: { active: true }, orderBy: { company: "asc" }, include: { contacts: true }
}),
    db.department.findMany(),
    db.staff.findMany({ where: { active: true } }),
    db.project.findMany({ orderBy: { name: "asc" }, select: { id: true, name: true } }),
  ]);
  return (
    <div className="max-w-4xl">
      <PageHeader title="Open support ticket" back={{ href: "/admin/tickets", label: "Support" }} />
      <ActionForm action={createTicket}>
        <Card>
          <div className="grid gap-4 p-5 sm:grid-cols-2">
```

```
            <Field label="Subject *" className="sm:col-span-2"><input name="subject" required
className="input" /></Field>
            <Field label="Contact">
              <select name="contactId" className="input">
                <option value="">— Not a saved contact —</option>
                {customers.map((c) => (
                  <optgroup key={c.id} label={c.company}>
                    {c.contacts.map((ct) => <option key={ct.id} value={ct.id}>{ct.firstName} {ct.lastName}
({ct.email})</option>)}
                  </optgroup>
                ))}
              </select>
            </Field>
            <Field label="Customer">
              <select name="customerId" className="input">
                <option value="">— From contact —</option>
                {customers.map((c) => <option key={c.id} value={c.id}>{c.company}</option>)}
              </select>
            </Field>
            <Field label="Name (if not a contact)"><input name="name" className="input" /></Field>
            <Field label="Email (if not a contact)"><input name="email" type="email" className="input" />
</Field>
            <Field label="Department">
              <select name="departmentId" className="input">{depts.map((d) => <option key={d.id} value=
{d.id}>{d.name}</option>)}</select>
            </Field>
            <Field label="Priority">
              <select name="priority" defaultValue="MEDIUM" className="input">
{Object.entries(PRIORITY).map(([k, v]) => <option key={k} value={k}>{v.label}</option>)}</select>
            </Field>
            <Field label="Assign to">
              <select name="assignedId" defaultValue={user.id} className="input">
                <option value="">Unassigned</option>
                {staff.map((s) => <option key={s.id} value={s.id}>{s.firstName} {s.lastName}</option>)}
              </select>
            </Field>
            <Field label="Project">
              <select name="projectId" className="input"><option value="">—</option>{projects.map((p) =>
<option key={p.id} value={p.id}>{p.name}</option>)}</select>
            </Field>
            <Field label="Message *" className="sm:col-span-2"><textarea name="message" rows={7} required
className="input" /></Field>
          </div>
          <div className="flex justify-end border-t border-slate-100 p-4"><SubmitButton>Open
ticket</SubmitButton></div>
        </Card>
      </ActionForm>
    </div>
  );
}
```

src/app/admin/tickets/page.tsx

```
import Link from "next/link";
import type { Prisma } from "@prisma/client";
import { requirePerm } from "@/lib/auth";
import { db } from "@/lib/db";
import { timeAgo } from "@/lib/format";
import { PRIORITY, TICKET_STATUS } from "@/lib/constants";
import { ButtonLink, Card, EmptyState, PageHeader, StatusBadge, Table } from "@/components/ui";
import { FilterSelect, SearchBox } from "@/components/client";
import { Pagination, pageArgs } from "@/components/pagination";

export const metadata = { title: "Support" };

export default async function TicketsPage({ searchParams }: { searchParams: Promise<Record<string, string |
undefined>> }) {
  await requirePerm("tickets");
  const sp = await searchParams;
  const { page, skip, take } = pageArgs(sp.page);
  const where: Prisma.TicketWhereInput = {};
  if (sp.status) where.status = sp.status;
  else if (sp.all !== "1") where.status = { not: "CLOSED" };
```

```
  if (sp.department) where.departmentId = Number(sp.department);
  if (sp.q) where.OR = [{ subject: { contains: sp.q } }, { customer: { company: { contains: sp.q } } }];
  const [rows, total, groups, depts] = await Promise.all([
    db.ticket.findMany({ where, skip, take, orderBy: [{ lastReply: "desc" }, { createdAt: "desc" }], include:
{ customer: true, contact: true, department: true, assigned: true, _count: { select: { replies: true } } }
}),
    db.ticket.count({ where }),
    db.ticket.groupBy({ by: ["status"], _count: true }),
    db.department.findMany(),
  ]);
  return (
    <>
      <PageHeader title="Support tickets" actions={<ButtonLink href="/admin/tickets/new">+ Open
ticket</ButtonLink>} />
      <div className="mb-6 grid grid-cols-2 gap-3 sm:grid-cols-5">
        {Object.entries(TICKET_STATUS).map(([k, v]) => (
          <Link key={k} href={`?status=${k}`} className="card px-4 py-3 hover:shadow-md">
            <p className="text-2xl font-bold text-ink">{groups.find((g) => g.status === k)?._count ?? 0}</p>
            <p className="text-xs text-slate-500">{v.label}</p>
          </Link>
        ))}
      </div>
      <Card>
        <div className="flex flex-wrap gap-2 border-b border-slate-100 p-4">
          <SearchBox placeholder="Subject or customer…" />
          <FilterSelect name="status" placeholder="All open" options=
{Object.entries(TICKET_STATUS).map(([value, s]) => ({ value, label: s.label }))} />
          <FilterSelect name="department" placeholder="All departments" options={depts.map((d) => ({ value:
String(d.id), label: d.name }))} />
        </div>
        {rows.length === 0 ? (
          <EmptyState title="No tickets" text="Great — the support queue is empty." />
        ) : (
          <Table head={["#", "Subject", "Customer", "Department", "Assigned", "Priority", "Status", "Last
reply"]}>
            {rows.map((t) => (
              <tr key={t.id} className="hover:bg-slate-50/60">
                <td className="td text-slate-400">#{t.id}</td>
                <td className="td"><Link className="link" href={`/admin/tickets/${t.id}`}>{t.subject}</Link>
<span className="text-xs text-slate-400">({t._count.replies})</span></td>
                <td className="td">{t.customer?.company ?? t.name ?? "—"}</td>
                <td className="td">{t.department?.name ?? "—"}</td>
                <td className="td">{t.assigned ? `${t.assigned.firstName} ${t.assigned.lastName}` : "—"}</td>
                <td className="td"><StatusBadge map={PRIORITY} value={t.priority} /></td>
                <td className="td"><StatusBadge map={TICKET_STATUS} value={t.status} /></td>
                <td className="td text-xs text-slate-500">{timeAgo(t.lastReply ?? t.createdAt)}</td>
              </tr>
            ))}
          </Table>
        )}
        <Pagination total={total} page={page} params={sp} />
      </Card>
    </>
  );
}
```

src/app/admin/todo-actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { db } from "@/lib/db";
import { requireUser } from "@/lib/auth";

export async function addTodo(f: FormData) {
  const user = await requireUser();
  const text = String(f.get("text") || "").trim();
  if (text) await db.todo.create({ data: { staffId: user.id, text } });
  revalidatePath("/admin");
}
export async function toggleTodo(id: number) {
  const user = await requireUser();
  const t = await db.todo.findFirst({ where: { id, staffId: user.id } });
  if (t) await db.todo.update({ where: { id }, data: { done: !t.done } });
```

```
  revalidatePath("/admin");
}
export async function deleteTodo(id: number) {
  const user = await requireUser();
  await db.todo.deleteMany({ where: { id, staffId: user.id } });
  revalidatePath("/admin");
}
```

src/app/admin/todo-widget.tsx

```
"use client";
import { useRef, useTransition } from "react";
import { X } from "lucide-react";
import { addTodo, toggleTodo, deleteTodo } from "./todo-actions";
import { Card } from "@/components/ui";

export function TodoWidget({ todos }: { todos: { id: number; text: string; done: boolean }[] }) {
  const ref = useRef<HTMLFormElement>(null);
  const [, start] = useTransition();
  return (
    <Card title="My to-do list">
      <form
        ref={ref}
        action={async (f) => {
          await addTodo(f);
          ref.current?.reset();
        }}
        className="flex gap-2 border-b border-slate-100 p-4"
      >
        <input name="text" placeholder="Add a to-do and press Enter" className="input" />
      </form>
      <ul className="max-h-72 divide-y divide-slate-100 overflow-y-auto">
        {todos.length === 0 && <li className="px-5 py-6 text-center text-sm text-slate-400">Nothing to do.
Nice!</li>}
        {todos.map((t) => (
          <li key={t.id} className="group flex items-center gap-3 px-5 py-2.5">
            <input type="checkbox" checked={t.done} onChange={() => start(() => toggleTodo(t.id))}
className="h-4 w-4 rounded accent-[#27ae57]" />
            <span className={`flex-1 text-sm ${t.done ? "text-slate-400 line-through" : "text-slate-700"}`}>
{t.text}</span>
            <button onClick={() => start(() => deleteTodo(t.id))} className="text-slate-300 opacity-0
hover:text-brand-red group-hover:opacity-100" aria-label="Delete">
              <X className="h-4 w-4" />
            </button>
          </li>
        ))}
      </ul>
    </Card>
  );
}
```

src/app/estimate/[hash]/actions.ts

```
"use server";
import { revalidatePath } from "next/cache";
import { db } from "@/lib/db";
import { logActivity } from "@/lib/activity";

export async function respondEstimate(f: FormData) {
  const hash = String(f.get("hash"));
  const decision = f.get("decision") === "accept" ? "ACCEPTED" : "DECLINED";
  const est = await db.estimate.findUnique({ where: { hash } });
  if (!est || est.status !== "SENT") return;
  await db.estimate.update({ where: { id: est.id }, data: { status: decision } });
  await logActivity("estimate", est.id, `received client response: estimate #${est.number}
${decision.toLowerCase()}`);
  revalidatePath(`/estimate/${hash}`);
}
```

src/app/estimate/[hash]/page.tsx

```
import { notFound } from "next/navigation";
import { db } from "@/lib/db";
import { getSettings } from "@/lib/settings";
import { docNumber } from "@/lib/sales";
import { ESTIMATE_STATUS } from "@/lib/constants";
import { StatusBadge, btn } from "@/components/ui";
import { PrintButton } from "@/components/client";
import { SalesDocument } from "@/components/sales-document";
import { respondEstimate } from "./actions";

export const metadata = { title: "Estimate", robots: { index: false } };

export default async function PublicEstimate({ params }: { params: Promise<{ hash: string }> }) {
  const { hash } = await params;
  const est = await db.estimate.findUnique({ where: { hash }, include: { items: { orderBy: { sortOrder: "asc"
} }, customer: { include: { contacts: true } } } });
  if (!est || est.status === "DRAFT") notFound();
  const settings = await getSettings();
  return (
    <div className="mx-auto max-w-4xl px-4 py-10">
      <div className="no-print mb-4 flex flex-wrap items-center justify-end gap-2">
        {est.status === "SENT" && (
          <>
            <form action={respondEstimate}><input type="hidden" name="hash" value={hash} /><input
type="hidden" name="decision" value="decline" /><button className={btn("secondary")}>Decline</button></form>
            <form action={respondEstimate}><input type="hidden" name="hash" value={hash} /><input
type="hidden" name="decision" value="accept" /><button className={btn("success")}>Accept estimate</button>
</form>
          </>
        )}
        {est.status === "ACCEPTED" && <p className="text-sm font-medium text-brand-green">Thank you — this
estimate has been accepted.</p>}
        <PrintButton />
      </div>
      <SalesDocument settings={settings} doc={{ ...est, title: "Estimate", number: docNumber(est.prefix,
est.number), secondLabel: "Valid until", secondDate: est.expiryDate, statusBadge: <StatusBadge map=
{ESTIMATE_STATUS} value={est.status} /> }} />
    </div>
  );
}
```

src/app/globals.css

```
@import "tailwindcss";

@theme {
  --color-brand-green: #27ae57;
  --color-brand-red: #e84a37;
  --color-brand-blue: #4f74c4;
  --color-brand-yellow: #fbbc15;
  --color-primary: #3f63b5;
  --color-primary-dark: #324f93;
  --color-ink: #0f1729;
  --color-sidebar: #0e1424;
  --font-sans: "Inter", ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
}

html, body { background: #f5f7fb; color: #1e293b; }
body { -webkit-font-smoothing: antialiased; }

@layer components {
  .input {
    @apply w-full rounded-lg border border-slate-300 bg-white px-3 py-2 text-sm text-slate-800 shadow-sm
outline-none transition placeholder:text-slate-400 focus:border-primary focus:ring-2 focus:ring-primary/20
disabled:bg-slate-50;
  }
  .label { @apply mb-1 block text-xs font-semibold uppercase tracking-wide text-slate-500; }
  .card { @apply rounded-xl border border-slate-200 bg-white shadow-sm; }
  .th { @apply px-4 py-3 text-left text-xs font-semibold uppercase tracking-wide text-slate-500; }
  .td { @apply px-4 py-3 text-sm text-slate-700; }
  .link { @apply font-medium text-primary hover:underline; }
```

```
}

@media print {
  .no-print { display: none !important; }
  body { background: white; }
}
```

src/app/invoice/[hash]/page.tsx

```
import { notFound } from "next/navigation";
import { db } from "@/lib/db";
import { getSettings } from "@/lib/settings";
import { docNumber, isOverdue } from "@/lib/sales";
import { INVOICE_STATUS } from "@/lib/constants";
import { StatusBadge } from "@/components/ui";
import { PrintButton } from "@/components/client";
import { SalesDocument } from "@/components/sales-document";

export const metadata = { title: "Invoice", robots: { index: false } };

export default async function PublicInvoice({ params }: { params: Promise<{ hash: string }> }) {
  const { hash } = await params;
  const inv = await db.invoice.findUnique({ where: { hash }, include: { items: { orderBy: { sortOrder: "asc"
} }, customer: { include: { contacts: true } }, payments: true } });
  if (!inv || inv.status === "DRAFT") notFound();
  const settings = await getSettings();
  return (
    <div className="mx-auto max-w-4xl px-4 py-10">
      <div className="no-print mb-4 flex justify-end"><PrintButton /></div>
      <SalesDocument
        settings={settings}
        doc={{ ...inv, title: "Invoice", number: docNumber(inv.prefix, inv.number), secondLabel: "Due date",
secondDate: inv.dueDate, statusBadge: <StatusBadge map={INVOICE_STATUS} value={isOverdue(inv) ? "OVERDUE" :
inv.status} /> }}
      />
    </div>
  );
}
```

src/app/layout.tsx

```
import type { Metadata } from "next";
import "./globals.css";

export const metadata: Metadata = { title: { default: "Squareit CRM", template: "%s · Squareit CRM" },
description: "Squareit CRM" };

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <head>
        <link rel="preconnect" href="https://fonts.googleapis.com" />
        <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap"
rel="stylesheet" />
      </head>
      <body>{children}</body>
    </html>
  );
}
```

src/app/login/actions.ts

```
"use server";
import bcrypt from "bcryptjs";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { db } from "@/lib/db";
import { SESSION_COOKIE, signSession } from "@/lib/session";
import type { ActionState } from "@/lib/forms";
```

```
export async function login(_: ActionState, f: FormData): Promise<ActionState> {
  const email = String(f.get("email") || "").trim().toLowerCase();
  const password = String(f.get("password") || "");
  const staff = await db.staff.findUnique({ where: { email } });
  if (!staff || !(await bcrypt.compare(password, staff.password))) return { error: "Invalid email or
password." };
  if (!staff.active) return { error: "Your account is inactive. Contact an administrator." };
  await db.staff.update({ where: { id: staff.id }, data: { lastLogin: new Date() } });
  const jar = await cookies();
  jar.set(SESSION_COOKIE, await signSession(staff.id), {
    httpOnly: true,
    sameSite: "lax",
    secure: process.env.NODE_ENV === "production",
    path: "/",
    maxAge: 60 * 60 * 24 * (f.get("remember") ? 30 : 7),
  });
  const next = String(f.get("next") || "/admin");
  redirect(next.startsWith("/admin") ? next : "/admin");
}

export async function logout() {
  const jar = await cookies();
  jar.delete(SESSION_COOKIE);
  redirect("/login");
}
```

src/app/login/page.tsx

```
import { ActionForm, SubmitButton } from "@/components/client";
import { login } from "./actions";

export const metadata = { title: "Login" };

export default async function LoginPage({ searchParams }: { searchParams: Promise<{ next?: string }> }) {
  const { next } = await searchParams;
  return (
    <div className="grid min-h-screen lg:grid-cols-2">
      <div className="relative hidden overflow-hidden bg-sidebar lg:flex lg:flex-col lg:justify-between lg:p-
12">
        <img src="/logo.png" alt="Squareit" className="w-56" />
        <div className="relative z-10">
          <h2 className="text-4xl font-bold leading-tight text-white">
            Your clients, projects
            <br />
            and billing — <span className="text-brand-yellow">in one place.</span>
          </h2>
          <p className="mt-4 max-w-md text-slate-400">Leads, customers, invoices, projects, tasks and
support, rebuilt for speed.</p>
        </div>
        <p className="text-xs text-slate-500">© {new Date().getFullYear()} Squareit · Say Digital, Way
Digital</p>
        <div className="pointer-events-none absolute -right-20 -top-16 grid grid-cols-2 gap-6 opacity-15">
          <span className="h-40 w-40 rounded-2xl border-[14px] border-brand-green" />
          <span className="h-40 w-40 rounded-2xl border-[14px] border-brand-red" />
          <span className="h-40 w-40 rounded-2xl border-[14px] border-brand-blue" />
          <span className="h-40 w-40 rounded-2xl border-[14px] border-brand-yellow" />
        </div>
      </div>
      <div className="flex items-center justify-center p-6">
        <div className="w-full max-w-sm">
          <div className="mb-8 rounded-xl bg-sidebar p-4 lg:hidden">
            <img src="/logo.png" alt="Squareit" className="mx-auto w-44" />
          </div>
          <h1 className="text-2xl font-bold text-ink">Welcome back</h1>
          <p className="mb-6 mt-1 text-sm text-slate-500">Sign in to the admin panel</p>
          <ActionForm action={login} className="space-y-4">
            <input type="hidden" name="next" value={next ?? ""} />
            <label className="block">
              <span className="label">Email</span>
              <input name="email" type="email" required autoComplete="email" className="input"
placeholder="you@squareit.in" />
            </label>
            <label className="block">
```

```
              <span className="label">Password</span>
              <input name="password" type="password" required autoComplete="current-password"
className="input" />
            </label>
            <label className="flex items-center gap-2 text-sm text-slate-600">
              <input type="checkbox" name="remember" className="rounded" /> Remember me
            </label>
            <SubmitButton className="w-full py-2.5">Sign in</SubmitButton>
          </ActionForm>
        </div>
      </div>
    </div>
  );
}
```

src/app/not-found.tsx

```
import Link from "next/link";
export default function NotFound() {
  return (
    <div className="flex min-h-[60vh] flex-col items-center justify-center text-center">
      <p className="text-6xl font-extrabold text-slate-200">404</p>
      <p className="mt-2 font-semibold text-slate-700">Page not found</p>
      <Link href="/admin" className="link mt-4 text-sm">Go to dashboard</Link>
    </div>
  );
}
```

src/app/page.tsx

```
import { redirect } from "next/navigation";
export default function Home() {
  redirect("/admin");
}
```

src/components/charts.tsx

```
"use client";
import { Bar, BarChart, CartesianGrid, ResponsiveContainer, Tooltip, XAxis, YAxis, Cell, Pie, PieChart } from
"recharts";

const fmt = (v: number) => (v >= 100000 ? `₹${(v / 100000).toFixed(1)}L` : v >= 1000 ? `₹${(v /
1000).toFixed(0)}k` : `₹${v}`);

export function RevenueChart({ data }: { data: { month: string; revenue: number; expenses: number }[] }) {
  return (
    <ResponsiveContainer width="100%" height={260}>
      <BarChart data={data} margin={{ top: 8, right: 8, left: 0, bottom: 0 }} barGap={2}>
        <CartesianGrid vertical={false} stroke="#eef1f6" />
        <XAxis dataKey="month" tickLine={false} axisLine={false} tick={{ fontSize: 12, fill: "#64748b" }} />
        <YAxis tickFormatter={fmt} tickLine={false} axisLine={false} tick={{ fontSize: 12, fill: "#64748b" }}
width={56} />
        <Tooltip
          formatter={(v, n) => [new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR"
}).format(Number(v)), n === "revenue" ? "Payments received" : "Expenses"]}
          cursor={{ fill: "#f1f5f9" }}
          contentStyle={{ borderRadius: 10, border: "1px solid #e2e8f0", fontSize: 12 }}
        />
        <Bar isAnimationActive={false} dataKey="revenue" fill="#4f74c4" radius={[4, 4, 0, 0]} maxBarSize={28}
/>
        <Bar isAnimationActive={false} dataKey="expenses" fill="#e84a37" radius={[4, 4, 0, 0]} maxBarSize=
{28} opacity={0.75} />
      </BarChart>
    </ResponsiveContainer>
  );
}

export function DonutChart({ data }: { data: { name: string; value: number; color: string }[] }) {
  const total = data.reduce((s, d) => s + d.value, 0);
  return (
```

```
    <div className="flex items-center gap-6">
      <div className="relative h-40 w-40 shrink-0">
        <ResponsiveContainer width="100%" height="100%">
          <PieChart>
            <Pie isAnimationActive={false} data={total ? data : [{ name: "none", value: 1, color: "#e2e8f0"
}]} dataKey="value" innerRadius={50} outerRadius={72} paddingAngle={total ? 2 : 0} stroke="none">
              {(total ? data : [{ color: "#e2e8f0" }]).map((d, i) => (
                <Cell key={i} fill={d.color} />
              ))}
            </Pie>
          </PieChart>
        </ResponsiveContainer>
        <div className="absolute inset-0 flex flex-col items-center justify-center">
          <span className="text-2xl font-bold text-ink">{total}</span>
          <span className="text-[10px] uppercase tracking-wide text-slate-500">total</span>
        </div>
      </div>
      <ul className="min-w-0 flex-1 space-y-1.5 text-sm">
        {data.map((d) => (
          <li key={d.name} className="flex items-center gap-2">
            <span className="h-2.5 w-2.5 shrink-0 rounded-full" style={{ background: d.color }} />
            <span className="flex-1 truncate text-slate-600">{d.name}</span>
            <span className="font-semibold text-slate-800">{d.value}</span>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

src/components/client.tsx

```
"use client";
import { useActionState, useEffect, useRef, useState, type ReactNode } from "react";
import { useFormStatus } from "react-dom";
import { useRouter, useSearchParams, usePathname } from "next/navigation";
import { btn, type BtnVariant } from "./ui";
import type { ActionState } from "@/lib/forms";

export function SubmitButton({ children, variant = "primary", className }: { children: ReactNode; variant?:
BtnVariant; className?: string }) {
  const { pending } = useFormStatus();
  return (
    <button type="submit" disabled={pending} className={btn(variant, className)}>
      {pending ? "Saving…" : children}
    </button>
  );
}

/** Form wired to a server action that returns ActionState (or redirects). Shows error inline. */
export function ActionForm({
  action,
  children,
  className,
  resetOnSuccess,
  savedText,
}: {
  action: (state: ActionState, f: FormData) => Promise<ActionState>;
  children: ReactNode;
  className?: string;
  resetOnSuccess?: boolean;
  savedText?: string;
}) {
  const [state, formAction] = useActionState(action, undefined);
  const ref = useRef<HTMLFormElement>(null);
  useEffect(() => {
    if (state?.ok && resetOnSuccess) ref.current?.reset();
  }, [state, resetOnSuccess]);
  return (
    <form ref={ref} action={formAction} className={className}>
      {state?.error && <div className="mb-4 rounded-lg border border-red-200 bg-red-50 px-4 py-2.5 text-sm
text-red-700">{state.error}</div>}
      {state?.ok && savedText && <div className="mb-4 rounded-lg border border-green-200 bg-green-50 px-4 py-
```

```
2.5 text-sm text-green-700">{savedText}</div>}
      {children}
    </form>
  );
}

/** Two-step confirm button that submits a server action (no browser dialogs). */
export function ConfirmButton({
  action,
  label = "Delete",
  confirmLabel = "Click to confirm",
  variant = "secondary",
  hidden,
  className,
}: {
  action: (f: FormData) => Promise<void>;
  label?: ReactNode;
  confirmLabel?: string;
  variant?: BtnVariant;
  hidden?: Record<string, string | number>;
  className?: string;
}) {
  const [armed, setArmed] = useState(false);
  useEffect(() => {
    if (!armed) return;
    const t = setTimeout(() => setArmed(false), 3500);
    return () => clearTimeout(t);
  }, [armed]);
  return (
    <form action={action} className="inline">
      {hidden && Object.entries(hidden).map(([k, v]) => <input key={k} type="hidden" name={k} value={v} />)}
      {armed ? (
        <button type="submit" className={btn("danger", className)}>
          {confirmLabel}
        </button>
      ) : (
        <button type="button" onClick={() => setArmed(true)} className={btn(variant, className)}>
          {label}
        </button>
      )}
    </form>
  );
}

/** Select that navigates by updating a search param (for filters). */
export function FilterSelect({ name, options, placeholder }: { name: string; options: { value: string; label:
string }[]; placeholder: string }) {
  const router = useRouter();
  const pathname = usePathname();
  const sp = useSearchParams();
  return (
    <select
      className="input w-auto min-w-40"
      value={sp.get(name) ?? ""}
      onChange={(e) => {
        const p = new URLSearchParams(sp.toString());
        if (e.target.value) p.set(name, e.target.value);
        else p.delete(name);
        p.delete("page");
        router.push(`${pathname}?${p.toString()}`);
      }}
    >
      <option value="">{placeholder}</option>
      {options.map((o) => (
        <option key={o.value} value={o.value}>
          {o.label}
        </option>
      ))}
    </select>
  );
}

export function SearchBox({ placeholder = "Search…" }: { placeholder?: string }) {
  const router = useRouter();
  const pathname = usePathname();
```

```
  const sp = useSearchParams();
  const [q, setQ] = useState(sp.get("q") ?? "");
  return (
    <form
      onSubmit={(e) => {
        e.preventDefault();
        const p = new URLSearchParams(sp.toString());
        if (q) p.set("q", q);
        else p.delete("q");
        p.delete("page");
        router.push(`${pathname}?${p.toString()}`);
      }}
      className="relative"
    >
      <svg className="pointer-events-none absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-slate-400"
fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>
        <circle cx="11" cy="11" r="7" />
        <path d="m20 20-3.5-3.5" />
      </svg>
      <input value={q} onChange={(e) => setQ(e.target.value)} placeholder={placeholder} className="input w-64
pl-9" />
    </form>
  );
}

export function PrintButton() {
  return (
    <button type="button" onClick={() => window.print()} className={btn("secondary")}>
      Print / PDF
    </button>
  );
}

export function CopyLink({ path, label = "Copy client link" }: { path: string; label?: string }) {
  const [done, setDone] = useState(false);
  return (
    <button
      type="button"
      className={btn("secondary")}
      onClick={async () => {
        await navigator.clipboard.writeText(window.location.origin + path);
        setDone(true);
        setTimeout(() => setDone(false), 1500);
      }}
    >
      {done ? "Copied!" : label}
    </button>
  );
}
```

src/components/kanban.tsx

```
"use client";
import Link from "next/link";
import { useOptimistic, useTransition, useState } from "react";
import clsx from "clsx";

export type KanbanCard = { id: number; columnId: number | string; title: string; subtitle?: string; meta?:
string; href: string; accent?: string };
export type KanbanColumn = { id: number | string; name: string; color: string };

/** Generic drag-and-drop board (native HTML5 DnD). */
export function Kanban({ columns, cards, onMove }: { columns: KanbanColumn[]; cards: KanbanCard[]; onMove:
(id: number, columnId: never) => Promise<void> }) {
  const [optimistic, setOptimistic] = useOptimistic(cards, (state, { id, columnId }: { id: number; columnId:
number | string }) =>
    state.map((c) => (c.id === id ? { ...c, columnId } : c))
  );
  const [, start] = useTransition();
  const [over, setOver] = useState<number | string | null>(null);

  return (
    <div className="flex gap-4 overflow-x-auto pb-4">
```

```
      {columns.map((col) => {
        const list = optimistic.filter((c) => c.columnId === col.id);
        return (
          <div
            key={col.id}
            onDragOver={(e) => {
              e.preventDefault();
              setOver(col.id);
            }}
            onDragLeave={() => setOver(null)}
            onDrop={(e) => {
              e.preventDefault();
              setOver(null);
              const id = Number(e.dataTransfer.getData("text/plain"));
              const card = optimistic.find((c) => c.id === id);
              if (!card || card.columnId === col.id) return;
              start(async () => {
                setOptimistic({ id, columnId: col.id });
                await onMove(id, col.id as never);
              });
            }}
            className={clsx("flex w-72 shrink-0 flex-col rounded-xl bg-slate-100/80 transition", over ===
col.id && "ring-2 ring-primary/40")}
          >
            <div className="flex items-center gap-2 rounded-t-xl border-t-4 px-3 py-3" style={{ borderColor:
col.color }}>
              <span className="text-sm font-semibold text-slate-700">{col.name}</span>
              <span className="rounded-full bg-white px-2 text-xs font-medium text-slate-500">{list.length}
</span>
            </div>
            <div className="flex min-h-24 flex-1 flex-col gap-2 px-2 pb-3">
              {list.map((c) => (
                <div
                  key={c.id}
                  draggable
                  onDragStart={(e) => e.dataTransfer.setData("text/plain", String(c.id))}
                  className="cursor-grab rounded-lg border border-slate-200 bg-white p-3 shadow-sm transition
hover:shadow-md active:cursor-grabbing"
                  style={c.accent ? { borderLeft: `3px solid ${c.accent}` } : undefined}
                >
                  <Link href={c.href} className="block text-sm font-medium text-slate-800 hover:text-
primary">{c.title}</Link>
                  {c.subtitle && <p className="mt-0.5 text-xs text-slate-500">{c.subtitle}</p>}
                  {c.meta && <p className="mt-2 text-xs font-medium text-slate-600">{c.meta}</p>}
                </div>
              ))}
            </div>
          </div>
        );
      })}
    </div>
  );
}
```

src/components/line-items.tsx

```
"use client";
import { useMemo, useState } from "react";
import { Trash2, Plus } from "lucide-react";

export type EditorItem = { description: string; longDescription?: string | null; qty: number; rate: number;
unit?: string | null; taxRate: number };
type Saved = { id: number; description: string; longDescription: string | null; rate: number; unit: string |
null; taxRate: number };

const r2 = (n: number) => Math.round((n + Number.EPSILON) * 100) / 100;

export function LineItemsEditor({
  initial,
  savedItems,
  taxRates,
  initialDiscount = 0,
  initialAdjustment = 0,
```

```
  currency,
}: {
  initial: EditorItem[];
  savedItems: Saved[];
  taxRates: { label: string; rate: number }[];
  initialDiscount?: number;
  initialAdjustment?: number;
  currency: string;
}) {
  const blank: EditorItem = { description: "", longDescription: "", qty: 1, rate: 0, unit: "", taxRate:
taxRates[0]?.rate ?? 0 };
  const [items, setItems] = useState<EditorItem[]>(initial.length ? initial : [blank]);
  const [discount, setDiscount] = useState(initialDiscount);
  const [adjustment, setAdjustment] = useState(initialAdjustment);
  const fmt = (n: number) => {
    try {
      return new Intl.NumberFormat("en-IN", { style: "currency", currency }).format(n);
    } catch {
      return n.toFixed(2);
    }
  };

  const totals = useMemo(() => {
    const subtotal = r2(items.reduce((s, i) => s + (i.qty || 0) * (i.rate || 0), 0));
    const discountTotal = r2((subtotal * (discount || 0)) / 100);
    const factor = subtotal > 0 ? (subtotal - discountTotal) / subtotal : 1;
    const taxes: Record<string, number> = {};
    for (const i of items) if (i.taxRate) taxes[i.taxRate] = (taxes[i.taxRate] || 0) + (i.qty || 0) * (i.rate
|| 0) * factor * (i.taxRate / 100);
    const taxTotal = r2(Object.values(taxes).reduce((a, b) => a + b, 0));
    return { subtotal, discountTotal, taxes, taxTotal, total: r2(subtotal - discountTotal + taxTotal +
(adjustment || 0)) };
  }, [items, discount, adjustment]);

  const update = (idx: number, patch: Partial<EditorItem>) => setItems((list) => list.map((it, i) => (i ===
idx ? { ...it, ...patch } : it)));

  return (
    <div>
      <input type="hidden" name="items" value={JSON.stringify(items)} />
      <input type="hidden" name="discountPercent" value={discount} />
      <input type="hidden" name="adjustment" value={adjustment} />

      {savedItems.length > 0 && (
        <div className="flex items-center gap-2 border-b border-slate-100 p-4">
          <select
            className="input max-w-sm"
            value=""
            onChange={(e) => {
              const s = savedItems.find((x) => x.id === Number(e.target.value));
              if (!s) return;
              const it = { description: s.description, longDescription: s.longDescription, qty: 1, rate:
s.rate, unit: s.unit, taxRate: s.taxRate };
              setItems((list) => (list.length === 1 && !list[0].description ? [it] : [...list, it]));
            }}
          >
            <option value="">+ Add a saved item…</option>
            {savedItems.map((s) => (
              <option key={s.id} value={s.id}>
                {s.description} — {fmt(s.rate)}
              </option>
            ))}
          </select>
        </div>
      )}

      <div className="overflow-x-auto">
        <table className="min-w-full">
          <thead className="bg-slate-50/70">
            <tr>
              <th className="th w-[40%]">Item</th>
              <th className="th w-24">Qty</th>
              <th className="th w-32">Rate</th>
              <th className="th w-32">Tax</th>
              <th className="th w-32 text-right">Amount</th>
```

```
              <th className="th w-10" />
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-100">
            {items.map((it, idx) => (
              <tr key={idx} className="align-top">
                <td className="px-4 py-3">
                  <input className="input" placeholder="Description" value={it.description} onChange={(e) =>
update(idx, { description: e.target.value })} />
                  <textarea className="input mt-2 text-xs" rows={2} placeholder="Long description (optional)"
value={it.longDescription ?? ""} onChange={(e) => update(idx, { longDescription: e.target.value })} />
                </td>
                <td className="px-4 py-3">
                  <input className="input" type="number" step="any" min="0" value={it.qty} onChange={(e) =>
update(idx, { qty: Number(e.target.value) })} />
                  <input className="input mt-2 text-xs" placeholder="Unit" value={it.unit ?? ""} onChange=
{(e) => update(idx, { unit: e.target.value })} />
                </td>
                <td className="px-4 py-3">
                  <input className="input" type="number" step="any" min="0" value={it.rate} onChange={(e) =>
update(idx, { rate: Number(e.target.value) })} />
                </td>
                <td className="px-4 py-3">
                  <select className="input" value={it.taxRate} onChange={(e) => update(idx, { taxRate:
Number(e.target.value) })}>
                    <option value={0}>No tax</option>
                    {taxRates.map((t) => (
                      <option key={t.label} value={t.rate}>
                        {t.label}
                      </option>
                    ))}
                  </select>
                </td>
                <td className="px-4 py-3 pt-5 text-right text-sm font-medium">{fmt((it.qty || 0) * (it.rate
|| 0))}</td>
                <td className="px-2 py-3 pt-4">
                  <button type="button" className="rounded p-1.5 text-slate-400 hover:bg-red-50 hover:text-
brand-red" onClick={() => setItems((l) => (l.length > 1 ? l.filter((_, i) => i !== idx) : [blank]))} aria-
label="Remove line">
                    <Trash2 className="h-4 w-4" />
                  </button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
      <div className="flex flex-col gap-6 border-t border-slate-100 p-5 md:flex-row md:justify-between">
        <button type="button" onClick={() => setItems((l) => [...l, blank])} className="inline-flex h-fit
items-center gap-1 text-sm font-medium text-primary hover:underline">
          <Plus className="h-4 w-4" /> Add line
        </button>
        <dl className="w-full max-w-sm space-y-2 text-sm">
          <div className="flex justify-between"><dt className="text-slate-500">Subtotal</dt><dd
className="font-medium">{fmt(totals.subtotal)}</dd></div>
          <div className="flex items-center justify-between gap-3">
            <dt className="text-slate-500">Discount %</dt>
            <dd className="flex items-center gap-2">
              <input type="number" step="any" min="0" max="100" value={discount} onChange={(e) =>
setDiscount(Number(e.target.value))} className="input w-20 py-1" />
              <span className="w-24 text-right">-{fmt(totals.discountTotal)}</span>
            </dd>
          </div>
          {Object.entries(totals.taxes).map(([rate, amt]) => (
            <div key={rate} className="flex justify-between"><dt className="text-slate-500">Tax {rate}%</dt>
<dd>{fmt(r2(amt))}</dd></div>
          ))}
          <div className="flex items-center justify-between gap-3">
            <dt className="text-slate-500">Adjustment</dt>
            <dd><input type="number" step="any" value={adjustment} onChange={(e) =>
setAdjustment(Number(e.target.value))} className="input w-32 py-1 text-right" /></dd>
          </div>
          <div className="flex justify-between border-t border-slate-200 pt-2 text-base"><dt className="font-
semibold">Total</dt><dd className="font-bold text-ink">{fmt(totals.total)}</dd></div>
        </dl>
```

```
      </div>
    </div>
  );
}
```

src/components/notes.tsx

```
import { db } from "@/lib/db";
import { fmtDateTime } from "@/lib/format";
import { addNote, deleteNote } from "@/app/admin/shared-actions";
import { Avatar, Card } from "./ui";
import { ConfirmButton, SubmitButton } from "./client";

export async function Notes({ relType, relId, path }: { relType: string; relId: number; path: string }) {
  const notes = await db.note.findMany({ where: { relType, relId }, orderBy: { createdAt: "desc" }, include:
{ staff: true } });
  return (
    <Card title={`Notes (${notes.length})`}>
      <form action={addNote} className="space-y-2 border-b border-slate-100 p-4">
        <input type="hidden" name="relType" value={relType} />
        <input type="hidden" name="relId" value={relId} />
        <input type="hidden" name="path" value={path} />
        <textarea name="content" rows={3} required placeholder="Write a note…" className="input" />
        <div className="flex justify-end">
          <SubmitButton>Add note</SubmitButton>
        </div>
      </form>
      <ul className="divide-y divide-slate-100">
        {notes.map((n) => (
          <li key={n.id} className="flex gap-3 px-5 py-4">
            <Avatar name={n.staff ? `${n.staff.firstName} ${n.staff.lastName}` : "?"} size={30} />
            <div className="min-w-0 flex-1">
              <p className="text-xs text-slate-500">
                <span className="font-semibold text-slate-700">{n.staff ? `${n.staff.firstName}
${n.staff.lastName}` : "Deleted user"}</span> · {fmtDateTime(n.createdAt)}
              </p>
              <p className="mt-1 whitespace-pre-wrap text-sm text-slate-700">{n.content}</p>
            </div>
            <ConfirmButton action={deleteNote} hidden={{ id: n.id, path }} label="×" confirmLabel="Delete?"
variant="ghost" className="px-2 py-1 text-xs" />
          </li>
        ))}
      </ul>
    </Card>
  );
}
```

src/components/pagination.tsx

```
import Link from "next/link";
import { PAGE_SIZE } from "@/lib/constants";

export function Pagination({ total, page, params }: { total: number; page: number; params: Record<string,
string | undefined> }) {
  const pages = Math.max(1, Math.ceil(total / PAGE_SIZE));
  if (pages <= 1) return <div className="border-t border-slate-100 px-4 py-3 text-xs text-slate-500">{total}
record{total === 1 ? "" : "s"}</div>;
  const href = (p: number) => {
    const q = new URLSearchParams();
    for (const [k, v] of Object.entries(params)) if (v && k !== "page") q.set(k, v);
    q.set("page", String(p));
    return `?${q.toString()}`;
  };
  return (
    <div className="flex items-center justify-between border-t border-slate-100 px-4 py-3 text-xs text-slate-
500">
      <span>
        {total} records · page {page} of {pages}
      </span>
      <div className="flex gap-1">
        {page > 1 && <Link className="rounded-md border border-slate-200 px-2.5 py-1 hover:bg-slate-50" href=
{href(page - 1)}>Prev</Link>}
```

```
        {page < pages && <Link className="rounded-md border border-slate-200 px-2.5 py-1 hover:bg-slate-50"
href={href(page + 1)}>Next</Link>}
      </div>
    </div>
  );
}

export function pageArgs(page?: string) {
  const p = Math.max(1, parseInt(page || "1", 10) || 1);
  return { page: p, skip: (p - 1) * PAGE_SIZE, take: PAGE_SIZE };
}
```

src/components/sales-document.tsx

```
import type { LineItem, Customer, Contact, Payment } from "@prisma/client";
import { money, fmtDate, round2 } from "@/lib/format";
import type { Settings } from "@/lib/settings";
import type { ReactNode } from "react";

type Doc = {
  title: string;
  number: string;
  date: Date;
  secondLabel: string;
  secondDate: Date | null;
  currency: string;
  subtotal: number;
  discountPercent: number;
  discountTotal: number;
  taxTotal: number;
  adjustment: number;
  total: number;
  clientNote: string | null;
  terms: string | null;
  items: LineItem[];
  customer: Customer & { contacts: Contact[] };
  payments?: Payment[];
  statusBadge?: ReactNode;
};

/** Printable invoice/estimate body used in admin and public pages. */
export function SalesDocument({ doc, settings }: { doc: Doc; settings: Settings }) {
  const c = doc.customer;
  const contact = c.contacts.find((x) => x.isPrimary) ?? c.contacts[0];
  const factor = doc.subtotal > 0 ? (doc.subtotal - doc.discountTotal) / doc.subtotal : 1;
  const taxes: Record<string, number> = {};
  for (const i of doc.items) if (i.taxRate) taxes[i.taxRate] = (taxes[i.taxRate] || 0) + i.qty * i.rate *
factor * (i.taxRate / 100);
  const paid = round2((doc.payments ?? []).reduce((s, p) => s + p.amount, 0));
  const m = (n: number) => money(n, doc.currency);

  return (
    <div className="card overflow-hidden print:border-0 print:shadow-none">
      <div className="flex h-1.5">
        <span className="flex-1 bg-brand-green" /><span className="flex-1 bg-brand-red" /><span
className="flex-1 bg-brand-blue" /><span className="flex-1 bg-brand-yellow" />
      </div>
      <div className="p-8 sm:p-10">
        <div className="flex flex-wrap items-start justify-between gap-6">
          <div>
            <div className="mb-3 inline-block rounded-lg bg-sidebar px-3 py-2 print:bg-sidebar">
              <img src="/logo.png" alt={settings.company_name} className="h-10" />
            </div>
            <p className="font-semibold text-ink">{settings.company_name}</p>
            <p className="whitespace-pre-line text-sm text-slate-500">
              {[settings.company_address, [settings.company_city, settings.company_state,
settings.company_zip].filter(Boolean).join(", "), settings.company_country].filter(Boolean).join("\n")}
            </p>
            {settings.company_vat && <p className="text-sm text-slate-500">GSTIN: {settings.company_vat}</p>}
            {settings.company_email && <p className="text-sm text-slate-500">{settings.company_email}</p>}
            {settings.company_phone && <p className="text-sm text-slate-500">{settings.company_phone}</p>}
          </div>
          <div className="text-right">
```

```
            <p className="text-3xl font-extrabold uppercase tracking-tight text-ink">{doc.title}</p>
            <p className="mt-1 text-lg font-semibold text-slate-600"># {doc.number}</p>
            <div className="mt-2">{doc.statusBadge}</div>
          </div>
        </div>

        <div className="mt-10 flex flex-wrap justify-between gap-6">
          <div>
            <p className="label">Bill to</p>
            <p className="font-semibold text-ink">{c.company}</p>
            {contact && <p className="text-sm text-slate-600">{contact.firstName} {contact.lastName}</p>}
            <p className="whitespace-pre-line text-sm text-slate-500">
              {[c.address, [c.city, c.state, c.zip].filter(Boolean).join(", "),
c.country].filter(Boolean).join("\n")}
            </p>
            {c.vat && <p className="text-sm text-slate-500">GSTIN: {c.vat}</p>}
            {contact?.email && <p className="text-sm text-slate-500">{contact.email}</p>}
          </div>
          <dl className="space-y-1 text-sm">
            <div className="flex justify-end gap-6"><dt className="text-slate-500">{doc.title} date</dt><dd
className="w-32 text-right font-medium">{fmtDate(doc.date)}</dd></div>
            <div className="flex justify-end gap-6"><dt className="text-slate-500">{doc.secondLabel}</dt><dd
className="w-32 text-right font-medium">{fmtDate(doc.secondDate)}</dd></div>
          </dl>
        </div>

        <table className="mt-8 min-w-full">
          <thead>
            <tr className="border-y border-slate-200 bg-slate-50 text-left text-xs uppercase tracking-wide
text-slate-500">
              <th className="px-3 py-2.5">#</th>
              <th className="px-3 py-2.5">Item</th>
              <th className="px-3 py-2.5 text-right">Qty</th>
              <th className="px-3 py-2.5 text-right">Rate</th>
              <th className="px-3 py-2.5 text-right">Tax</th>
              <th className="px-3 py-2.5 text-right">Amount</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-100 text-sm">
            {doc.items.map((i, idx) => (
              <tr key={i.id} className="align-top">
                <td className="px-3 py-3 text-slate-400">{idx + 1}</td>
                <td className="px-3 py-3">
                  <p className="font-medium text-slate-800">{i.description}</p>
                  {i.longDescription && <p className="whitespace-pre-line text-xs text-slate-500">
{i.longDescription}</p>}
                </td>
                <td className="px-3 py-3 text-right">{i.qty} {i.unit}</td>
                <td className="px-3 py-3 text-right">{m(i.rate)}</td>
                <td className="px-3 py-3 text-right">{i.taxRate ? `${i.taxRate}%` : "—"}</td>
                <td className="px-3 py-3 text-right font-medium">{m(i.qty * i.rate)}</td>
              </tr>
            ))}
          </tbody>
        </table>

        <div className="mt-6 flex justify-end">
          <dl className="w-full max-w-xs space-y-1.5 text-sm">
            <div className="flex justify-between"><dt className="text-slate-500">Subtotal</dt><dd>
{m(doc.subtotal)}</dd></div>
            {doc.discountTotal > 0 && <div className="flex justify-between"><dt className="text-slate-
500">Discount ({doc.discountPercent}%)</dt><dd>-{m(doc.discountTotal)}</dd></div>}
            {Object.entries(taxes).map(([rate, amt]) => (
              <div key={rate} className="flex justify-between"><dt className="text-slate-500">Tax {rate}%
</dt><dd>{m(round2(amt))}</dd></div>
            ))}
            {doc.adjustment !== 0 && <div className="flex justify-between"><dt className="text-slate-
500">Adjustment</dt><dd>{m(doc.adjustment)}</dd></div>}
            <div className="flex justify-between border-t border-slate-200 pt-2 text-base font-bold text-
ink"><dt>Total</dt><dd>{m(doc.total)}</dd></div>
            {doc.payments && paid > 0 && (
              <>
                <div className="flex justify-between text-brand-green"><dt>Paid</dt><dd>-{m(paid)}</dd></div>
                <div className="flex justify-between font-semibold text-brand-red"><dt>Amount due</dt><dd>
{m(Math.max(0, round2(doc.total - paid)))}</dd></div>
```

```
              </>
            )}
          </dl>
        </div>

        {(doc.clientNote || doc.terms || settings.bank_details) && (
          <div className="mt-10 grid gap-6 border-t border-slate-100 pt-6 text-sm sm:grid-cols-2">
            {doc.clientNote && <div><p className="label">Note</p><p className="whitespace-pre-line text-
slate-600">{doc.clientNote}</p></div>}
            {settings.bank_details && doc.title === "Invoice" && <div><p className="label">Payment
details</p><p className="whitespace-pre-line text-slate-600">{settings.bank_details}</p></div>}
            {doc.terms && <div className="sm:col-span-2"><p className="label">Terms & conditions</p><p
className="whitespace-pre-line text-slate-600">{doc.terms}</p></div>}
          </div>
        )}
      </div>
    </div>
  );
}
```

src/components/sales-form.tsx

```
import { db } from "@/lib/db";
import { getSettings } from "@/lib/settings";
import { inputDate } from "@/lib/format";
import { CURRENCIES } from "@/lib/constants";
import type { ActionState } from "@/lib/forms";
import { ActionForm, SubmitButton } from "./client";
import { Card, Field } from "./ui";
import { LineItemsEditor, type EditorItem } from "./line-items";

export type SalesDocValues = {
  customerId?: number;
  projectId?: number | null;
  date?: Date;
  secondDate?: Date | null; // due date / expiry date
  currency?: string;
  discountPercent?: number;
  adjustment?: number;
  clientNote?: string | null;
  terms?: string | null;
  items?: EditorItem[];
  number?: string;
};

export async function SalesForm({
  kind,
  action,
  values,
  submitLabel,
}: {
  kind: "invoice" | "estimate";
  action: (s: ActionState, f: FormData) => Promise<ActionState>;
  values: SalesDocValues;
  submitLabel: string;
}) {
  const [customers, projects, items, taxes, settings] = await Promise.all([
    db.customer.findMany({ where: { active: true }, orderBy: { company: "asc" }, select: { id: true, company:
true } }),
    db.project.findMany({ where: { status: { not: "CANCELLED" } }, orderBy: { name: "asc" }, select: { id:
true, name: true, customerId: true } }),
    db.item.findMany({ orderBy: { description: "asc" } }),
    db.tax.findMany({ orderBy: { rate: "desc" } }),
    getSettings(),
  ]);
  const currency = values.currency || settings.currency;
  return (
    <ActionForm action={action} className="space-y-6">
      <Card title={kind === "invoice" ? "Invoice details" : "Estimate details"} actions={values.number &&
<span className="text-sm font-semibold text-slate-500">{values.number}</span>}>
        <div className="grid gap-4 p-5 sm:grid-cols-2 lg:grid-cols-4">
          <Field label="Customer *" className="sm:col-span-2">
            <select name="customerId" required defaultValue={values.customerId ?? ""} className="input">
```

```
              <option value="">Select customer…</option>
              {customers.map((c) => (
                <option key={c.id} value={c.id}>{c.company}</option>
              ))}
            </select>
          </Field>
          <Field label="Project">
            <select name="projectId" defaultValue={values.projectId ?? ""} className="input">
              <option value="">—</option>
              {projects.map((p) => (
                <option key={p.id} value={p.id}>{p.name}</option>
              ))}
            </select>
          </Field>
          <Field label="Currency">
            <select name="currency" defaultValue={currency} className="input">
              {CURRENCIES.map((c) => <option key={c}>{c}</option>)}
            </select>
          </Field>
          <Field label={kind === "invoice" ? "Invoice date" : "Estimate date"}>
            <input type="date" name="date" required defaultValue={inputDate(values.date ?? new Date())}
className="input" />
          </Field>
          <Field label={kind === "invoice" ? "Due date" : "Expiry date"}>
            <input type="date" name="secondDate" defaultValue={inputDate(values.secondDate)}
className="input" />
          </Field>
        </div>
      </Card>
      <Card title="Items">
        <LineItemsEditor
          initial={values.items ?? []}
          savedItems={items}
          taxRates={taxes.map((t) => ({ label: `${t.name} ${t.rate}%`, rate: t.rate }))}
          initialDiscount={values.discountPercent}
          initialAdjustment={values.adjustment}
          currency={currency}
        />
      </Card>
      <Card title="Notes & terms">
        <div className="grid gap-4 p-5 md:grid-cols-2">
          <Field label="Client note">
            <textarea name="clientNote" rows={4} defaultValue={values.clientNote ?? (kind === "invoice" ?
settings.invoice_client_note : "")} className="input" />
          </Field>
          <Field label="Terms & conditions">
            <textarea name="terms" rows={4} defaultValue={values.terms ?? (kind === "invoice" ?
settings.invoice_terms : settings.estimate_terms)} className="input" />
          </Field>
        </div>
      </Card>
      <div className="flex justify-end gap-2">
        <SubmitButton>{submitLabel}</SubmitButton>
      </div>
    </ActionForm>
  );
}
```

src/components/sidebar.tsx

```
"use client";
import Link from "next/link";
import { usePathname } from "next/navigation";
import { useState, useEffect } from "react";
import clsx from "clsx";
import {
  LayoutDashboard, Users, Target, FileText, FileSignature, CreditCard, Receipt, FolderKanban,
  CheckSquare, LifeBuoy, BarChart3, UserCog, Settings, Menu, X,
} from "lucide-react";

type Item = { href: string; label: string; icon: React.ElementType; perm?: string; adminOnly?: boolean;
badge?: string };
const groups: { title?: string; items: Item[] }[] = [
```

```
  { items: [{ href: "/admin", label: "Dashboard", icon: LayoutDashboard }] },
  {
    title: "CRM",
    items: [
      { href: "/admin/leads", label: "Leads", icon: Target, perm: "leads" },
      { href: "/admin/customers", label: "Customers", icon: Users, perm: "customers" },
    ],
  },
  {
    title: "Sales",
    items: [
      { href: "/admin/invoices", label: "Invoices", icon: FileText, perm: "invoices" },
      { href: "/admin/estimates", label: "Estimates", icon: FileSignature, perm: "estimates" },
      { href: "/admin/payments", label: "Payments", icon: CreditCard, perm: "payments" },
      { href: "/admin/expenses", label: "Expenses", icon: Receipt, perm: "expenses" },
    ],
  },
  {
    title: "Work",
    items: [
      { href: "/admin/projects", label: "Projects", icon: FolderKanban, perm: "projects" },
      { href: "/admin/tasks", label: "Tasks", icon: CheckSquare, perm: "tasks" },
      { href: "/admin/tickets", label: "Support", icon: LifeBuoy, perm: "tickets", badge: "tickets" },
    ],
  },
  {
    title: "Admin",
    items: [
      { href: "/admin/reports", label: "Reports", icon: BarChart3, perm: "reports" },
      { href: "/admin/staff", label: "Staff & roles", icon: UserCog, perm: "staff" },
      { href: "/admin/settings", label: "Settings", icon: Settings, perm: "settings" },
    ],
  },
];

export function Sidebar({ allowed, isAdmin, badges }: { allowed: Record<string, boolean>; isAdmin: boolean;
badges: Record<string, number> }) {
  const pathname = usePathname();
  const [open, setOpen] = useState(false);
  useEffect(() => setOpen(false), [pathname]);
  const isActive = (href: string) => (href === "/admin" ? pathname === "/admin" : pathname.startsWith(href));

  const nav = (
    <nav className="flex-1 space-y-5 overflow-y-auto px-3 py-4">
      {groups.map((g, gi) => {
        const items = g.items.filter((i) => (!i.perm || allowed[i.perm]) && (!i.adminOnly || isAdmin));
        if (!items.length) return null;
        return (
          <div key={gi}>
            {g.title && <p className="mb-1.5 px-3 text-[10px] font-semibold uppercase tracking-widest text-
slate-500">{g.title}</p>}
            <div className="space-y-0.5">
              {items.map((i) => {
                const active = isActive(i.href);
                const count = i.badge ? badges[i.badge] : 0;
                return (
                  <Link
                    key={i.href}
                    href={i.href}
                    className={clsx(
                      "group flex items-center gap-3 rounded-lg px-3 py-2 text-sm font-medium transition",
                      active ? "bg-white/10 text-white" : "text-slate-400 hover:bg-white/5 hover:text-slate-
200"
                    )}
                  >
                    <i.icon className={clsx("h-4 w-4", active ? "text-brand-yellow" : "text-slate-500 group-
hover:text-slate-300")} />
                    <span className="flex-1">{i.label}</span>
                    {count > 0 && <span className="rounded-full bg-brand-red px-1.5 text-[10px] font-bold
text-white">{count}</span>}
                  </Link>
                );
              })}
            </div>
          </div>
```

```
        );
      })}
    </nav>
  );

  return (
    <>
      <button onClick={() => setOpen(true)} className="no-print fixed left-3 top-3 z-40 rounded-lg bg-sidebar
p-2 text-white lg:hidden" aria-label="Open menu">
        <Menu className="h-5 w-5" />
      </button>
      {open && <div className="fixed inset-0 z-40 bg-black/40 lg:hidden" onClick={() => setOpen(false)} />}
      <aside
        className={clsx(
          "no-print fixed inset-y-0 left-0 z-50 flex w-64 flex-col bg-sidebar transition-transform
lg:translate-x-0",
          open ? "translate-x-0" : "-translate-x-full"
        )}
      >
        <div className="flex h-16 items-center justify-between border-b border-white/5 px-5">
          <Link href="/admin">
            <img src="/logo.png" alt="Squareit" className="h-12" />
          </Link>
          <button onClick={() => setOpen(false)} className="text-slate-400 lg:hidden" aria-label="Close
menu">
            <X className="h-5 w-5" />
          </button>
        </div>
        {nav}
        <div className="flex h-1">
          <span className="flex-1 bg-brand-green" />
          <span className="flex-1 bg-brand-red" />
          <span className="flex-1 bg-brand-blue" />
          <span className="flex-1 bg-brand-yellow" />
        </div>
      </aside>
    </>
  );
}
```

src/components/topbar.tsx

```
import Link from "next/link";
import { LogOut, Search, Plus } from "lucide-react";
import { logout } from "@/app/login/actions";
import { Avatar } from "./ui";

export function Topbar({ name, role }: { name: string; role: string }) {
  return (
    <header className="no-print sticky top-0 z-30 flex h-16 items-center gap-4 border-b border-slate-200 bg-
white/85 px-4 pl-16 backdrop-blur sm:px-6 lg:px-8">
      <form action="/admin/search" className="relative max-w-md flex-1">
        <Search className="pointer-events-none absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-slate-
400" />
        <input name="q" placeholder="Search customers, leads, invoices, projects…" className="input border-
slate-200 bg-slate-50 pl-9 shadow-none" />
      </form>
      <div className="ml-auto flex items-center gap-3">
        <details className="relative">
          <summary className="flex cursor-pointer list-none items-center gap-1 rounded-lg bg-primary px-3 py-
2 text-sm font-medium text-white hover:bg-primary-dark">
            <Plus className="h-4 w-4" /> <span className="hidden sm:inline">New</span>
          </summary>
          <div className="absolute right-0 mt-2 w-48 overflow-hidden rounded-xl border border-slate-200 bg-
white py-1 shadow-lg">
            {[
              ["/admin/leads/new", "Lead"],
              ["/admin/customers/new", "Customer"],
              ["/admin/invoices/new", "Invoice"],
              ["/admin/estimates/new", "Estimate"],
              ["/admin/projects/new", "Project"],
              ["/admin/tasks/new", "Task"],
              ["/admin/tickets/new", "Ticket"],
```

```
              ["/admin/expenses/new", "Expense"],
            ].map(([href, label]) => (
              <Link key={href} href={href} className="block px-4 py-2 text-sm text-slate-700 hover:bg-slate-
50">
                {label}
              </Link>
            ))}
          </div>
        </details>
        <Link href="/admin/profile" className="flex items-center gap-2.5">
          <Avatar name={name} size={34} />
          <span className="hidden text-left leading-tight md:block">
            <span className="block text-sm font-semibold text-slate-800">{name}</span>
            <span className="block text-xs text-slate-500">{role}</span>
          </span>
        </Link>
        <form action={logout}>
          <button className="rounded-lg p-2 text-slate-500 hover:bg-slate-100 hover:text-brand-red"
title="Log out">
            <LogOut className="h-4 w-4" />
          </button>
        </form>
      </div>
    </header>
  );
}
```

src/components/ui.tsx

```
import Link from "next/link";
import clsx from "clsx";
import type { ReactNode } from "react";
import type { Tone } from "@/lib/constants";

export { clsx as cx };

const btnBase =
  "inline-flex items-center justify-center gap-1.5 rounded-lg px-3.5 py-2 text-sm font-medium transition
disabled:opacity-50 disabled:pointer-events-none whitespace-nowrap";
const btnVariants = {
  primary: "bg-primary text-white shadow-sm hover:bg-primary-dark",
  secondary: "border border-slate-300 bg-white text-slate-700 shadow-sm hover:bg-slate-50",
  ghost: "text-slate-600 hover:bg-slate-100",
  danger: "bg-brand-red text-white hover:bg-red-600",
  success: "bg-brand-green text-white hover:bg-green-700",
};
export type BtnVariant = keyof typeof btnVariants;
export const btn = (v: BtnVariant = "primary", extra?: string) => clsx(btnBase, btnVariants[v], extra);

export function ButtonLink({ href, variant = "primary", children, className }: { href: string; variant?:
BtnVariant; children: ReactNode; className?: string }) {
  return (
    <Link href={href} className={btn(variant, className)}>
      {children}
    </Link>
  );
}

export function Card({ children, className, title, actions }: { children: ReactNode; className?: string;
title?: ReactNode; actions?: ReactNode }) {
  return (
    <div className={clsx("card", className)}>
      {(title || actions) && (
        <div className="flex items-center justify-between gap-3 border-b border-slate-100 px-5 py-3.5">
          <h3 className="text-sm font-semibold text-slate-800">{title}</h3>
          {actions}
        </div>
      )}
      {children}
    </div>
  );
}
```

```
const tones: Record<Tone, string> = {
  gray: "bg-slate-100 text-slate-700 ring-slate-200",
  blue: "bg-blue-50 text-blue-700 ring-blue-200",
  green: "bg-green-50 text-green-700 ring-green-200",
  amber: "bg-amber-50 text-amber-800 ring-amber-200",
  red: "bg-red-50 text-red-700 ring-red-200",
  purple: "bg-purple-50 text-purple-700 ring-purple-200",
};

export function Badge({ tone = "gray", children, color }: { tone?: Tone; children: ReactNode; color?: string
}) {
  if (color)
    return (
      <span className="inline-flex items-center gap-1.5 rounded-full px-2.5 py-0.5 text-xs font-medium ring-1
ring-inset" style={{ color, backgroundColor: color + "14", borderColor: color, ["--tw-ring-color" as string]:
color + "55" }}>
        {children}
      </span>
    );
  return <span className={clsx("inline-flex items-center rounded-full px-2.5 py-0.5 text-xs font-medium ring-
1 ring-inset", tones[tone])}>{children}</span>;
}

export function StatusBadge({ map, value }: { map: Record<string, { label: string; tone: Tone }>; value:
string }) {
  const s = map[value] ?? { label: value, tone: "gray" as Tone };
  return <Badge tone={s.tone}>{s.label}</Badge>;
}

export function PageHeader({ title, subtitle, actions, back }: { title: ReactNode; subtitle?: ReactNode;
actions?: ReactNode; back?: { href: string; label: string } }) {
  return (
    <div className="mb-6">
      {back && (
        <Link href={back.href} className="mb-2 inline-block text-xs font-medium text-slate-500 hover:text-
primary">
          ← {back.label}
        </Link>
      )}
      <div className="flex flex-wrap items-center justify-between gap-3">
        <div className="min-w-0">
          <h1 className="text-2xl font-bold tracking-tight text-ink">{title}</h1>
          {subtitle && <p className="mt-1 text-sm text-slate-500">{subtitle}</p>}
        </div>
        {actions && <div className="flex flex-wrap items-center gap-2">{actions}</div>}
      </div>
    </div>
  );
}

export function Field({ label, children, className, hint }: { label: string; children: ReactNode; className?:
string; hint?: string }) {
  return (
    <label className={clsx("block", className)}>
      <span className="label">{label}</span>
      {children}
      {hint && <span className="mt-1 block text-xs text-slate-400">{hint}</span>}
    </label>
  );
}

export function EmptyState({ title, text, action }: { title: string; text?: string; action?: ReactNode }) {
  return (
    <div className="flex flex-col items-center justify-center px-6 py-14 text-center">
      <div className="mb-3 grid h-12 w-12 grid-cols-2 gap-1 opacity-60">
        <span className="rounded-sm bg-brand-green" />
        <span className="rounded-sm bg-brand-red" />
        <span className="rounded-sm bg-brand-blue" />
        <span className="rounded-sm bg-brand-yellow" />
      </div>
      <p className="font-semibold text-slate-700">{title}</p>
      {text && <p className="mt-1 max-w-sm text-sm text-slate-500">{text}</p>}
      {action && <div className="mt-4">{action}</div>}
    </div>
  );
```

```
}

export function Table({ head, children }: { head: ReactNode[]; children: ReactNode }) {
  return (
    <div className="overflow-x-auto">
      <table className="min-w-full divide-y divide-slate-100">
        <thead className="bg-slate-50/70">
          <tr>
            {head.map((h, i) => (
              <th key={i} className="th">
                {h}
              </th>
            ))}
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100">{children}</tbody>
      </table>
    </div>
  );
}

export function Stat({ label, value, sub, accent = "blue", href }: { label: string; value: ReactNode; sub?:
ReactNode; accent?: "blue" | "green" | "red" | "yellow"; href?: string }) {
  const bar = { blue: "bg-brand-blue", green: "bg-brand-green", red: "bg-brand-red", yellow: "bg-brand-
yellow" }[accent];
  const inner = (
    <div className="card relative overflow-hidden p-5 transition hover:shadow-md">
      <span className={clsx("absolute inset-y-0 left-0 w-1", bar)} />
      <p className="text-xs font-semibold uppercase tracking-wide text-slate-500">{label}</p>
      <p className="mt-2 text-2xl font-bold text-ink">{value}</p>
      {sub && <p className="mt-1 text-xs text-slate-500">{sub}</p>}
    </div>
  );
  return href ? <Link href={href}>{inner}</Link> : inner;
}

export function Avatar({ name, size = 32 }: { name: string; size?: number }) {
  const colors = ["#27ae57", "#e84a37", "#4f74c4", "#e0a400", "#7c5cc4", "#0f9fb0"];
  const idx = [...name].reduce((s, c) => s + c.charCodeAt(0), 0) % colors.length;
  const ini = name.split(" ").map((p) => p[0]).slice(0, 2).join("").toUpperCase();
  return (
    <span title={name} className="inline-flex shrink-0 items-center justify-center rounded-full font-semibold
text-white ring-2 ring-white" style={{ width: size, height: size, fontSize: size * 0.38, backgroundColor:
colors[idx] }}>
      {ini}
    </span>
  );
}

export function DetailRow({ label, children }: { label: string; children: ReactNode }) {
  return (
    <div className="flex justify-between gap-4 py-2 text-sm">
      <dt className="text-slate-500">{label}</dt>
      <dd className="text-right font-medium text-slate-800">{children ?? "—"}</dd>
    </div>
  );
}

export function Tabs({ tabs, active, base }: { tabs: { key: string; label: string; count?: number }[];
active: string; base: string }) {
  return (
    <div className="mb-5 flex gap-1 overflow-x-auto border-b border-slate-200">
      {tabs.map((t) => (
        <Link
          key={t.key}
          href={`${base}${base.includes("?") ? "&" : "?"}tab=${t.key}`}
          className={clsx(
            "-mb-px whitespace-nowrap border-b-2 px-4 py-2.5 text-sm font-medium transition",
            active === t.key ? "border-primary text-primary" : "border-transparent text-slate-500 hover:text-
slate-800"
          )}
        >
          {t.label}
          {t.count !== undefined && <span className="ml-1.5 rounded-full bg-slate-100 px-1.5 py-0.5 text-
[10px] text-slate-600">{t.count}</span>}
```

```
        </Link>
      ))}
    </div>
  );
}
```

src/global.d.ts

```
declare module "*.css";
```

src/lib/activity.ts

```
import "server-only";
import { db } from "./db";

export async function logActivity(relType: string, relId: number, description: string, staffId?: number |
null) {
  await db.activity.create({ data: { relType, relId, description, staffId: staffId ?? null } });
}
```

src/lib/auth.ts

```
import "server-only";
import { cache } from "react";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { db } from "./db";
import { SESSION_COOKIE, verifySession } from "./session";
import { can, parsePermissions, type Action, type ModuleKey, type PermUser } from "./permissions";

export type CurrentUser = PermUser & {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  roleName: string | null;
};

export const getCurrentUser = cache(async (): Promise<CurrentUser | null> => {
  const jar = await cookies();
  const id = await verifySession(jar.get(SESSION_COOKIE)?.value);
  if (!id) return null;
  const staff = await db.staff.findUnique({ where: { id }, include: { role: true } });
  if (!staff || !staff.active) return null;
  return {
    id: staff.id,
    firstName: staff.firstName,
    lastName: staff.lastName,
    email: staff.email,
    isAdmin: staff.isAdmin,
    roleName: staff.isAdmin ? "Administrator" : staff.role?.name ?? null,
    permissions: parsePermissions(staff.role?.permissions),
  };
});

export async function requireUser() {
  const user = await getCurrentUser();
  if (!user) redirect("/login");
  return user;
}

/** Use in pages and server actions. Redirects to /admin/forbidden when not allowed. */
export async function requirePerm(module: ModuleKey, action: Action = "view") {
  const user = await requireUser();
  if (!can(user, module, action)) redirect("/admin/forbidden");
  return user;
}

export async function requireAdmin() {
  const user = await requireUser();
```

```
  if (!user.isAdmin) redirect("/admin/forbidden");
  return user;
}
```

src/lib/constants.ts

```
export type Tone = "gray" | "blue" | "green" | "amber" | "red" | "purple";

export const INVOICE_STATUS: Record<string, { label: string; tone: Tone }> = {
  DRAFT: { label: "Draft", tone: "gray" },
  UNPAID: { label: "Unpaid", tone: "red" },
  PARTIAL: { label: "Partially paid", tone: "amber" },
  PAID: { label: "Paid", tone: "green" },
  OVERDUE: { label: "Overdue", tone: "red" },
  CANCELLED: { label: "Cancelled", tone: "gray" },
};

export const ESTIMATE_STATUS: Record<string, { label: string; tone: Tone }> = {
  DRAFT: { label: "Draft", tone: "gray" },
  SENT: { label: "Sent", tone: "blue" },
  ACCEPTED: { label: "Accepted", tone: "green" },
  DECLINED: { label: "Declined", tone: "red" },
  EXPIRED: { label: "Expired", tone: "amber" },
};

export const PROJECT_STATUS: Record<string, { label: string; tone: Tone }> = {
  NOT_STARTED: { label: "Not started", tone: "gray" },
  IN_PROGRESS: { label: "In progress", tone: "blue" },
  ON_HOLD: { label: "On hold", tone: "amber" },
  CANCELLED: { label: "Cancelled", tone: "red" },
  FINISHED: { label: "Finished", tone: "green" },
};

export const TASK_STATUS: Record<string, { label: string; tone: Tone }> = {
  NOT_STARTED: { label: "Not started", tone: "gray" },
  IN_PROGRESS: { label: "In progress", tone: "blue" },
  TESTING: { label: "Testing", tone: "purple" },
  AWAITING_FEEDBACK: { label: "Awaiting feedback", tone: "amber" },
  COMPLETE: { label: "Complete", tone: "green" },
};

export const PRIORITY: Record<string, { label: string; tone: Tone }> = {
  LOW: { label: "Low", tone: "gray" },
  MEDIUM: { label: "Medium", tone: "blue" },
  HIGH: { label: "High", tone: "amber" },
  URGENT: { label: "Urgent", tone: "red" },
};

export const TICKET_STATUS: Record<string, { label: string; tone: Tone }> = {
  OPEN: { label: "Open", tone: "red" },
  IN_PROGRESS: { label: "In progress", tone: "blue" },
  ANSWERED: { label: "Answered", tone: "green" },
  ON_HOLD: { label: "On hold", tone: "amber" },
  CLOSED: { label: "Closed", tone: "gray" },
};

export const CURRENCIES = ["INR", "USD", "EUR", "GBP", "AED", "AUD", "CAD", "SGD"];
export const PAGE_SIZE = 20;
```

src/lib/db.ts

```
import { PrismaClient } from "@prisma/client";

const g = globalThis as unknown as { prisma?: PrismaClient };
export const db = g.prisma ?? new PrismaClient();
if (process.env.NODE_ENV !== "production") g.prisma = db;
```

src/lib/format.ts

```
export function money(n: number | null | undefined, currency = "INR") {
  const v = Number(n || 0);
  try {
    return new Intl.NumberFormat("en-IN", { style: "currency", currency, minimumFractionDigits: 2
}).format(v);
  } catch {
    return `${currency} ${v.toFixed(2)}`;
  }
}

export function round2(n: number) {
  return Math.round((n + Number.EPSILON) * 100) / 100;
}

export function fmtDate(d: Date | string | null | undefined) {
  if (!d) return "—";
  const date = typeof d === "string" ? new Date(d) : d;
  return date.toLocaleDateString("en-GB", { day: "2-digit", month: "short", year: "numeric" });
}

export function fmtDateTime(d: Date | string | null | undefined) {
  if (!d) return "—";
  const date = typeof d === "string" ? new Date(d) : d;
  return date.toLocaleString("en-GB", { day: "2-digit", month: "short", year: "numeric", hour: "2-digit",
minute: "2-digit" });
}

/** yyyy-mm-dd for <input type="date"> */
export function inputDate(d: Date | string | null | undefined) {
  if (!d) return "";
  const date = typeof d === "string" ? new Date(d) : d;
  return date.toISOString().slice(0, 10);
}

export function timeAgo(d: Date) {
  const s = Math.floor((Date.now() - d.getTime()) / 1000);
  if (s < 60) return "just now";
  const m = Math.floor(s / 60);
  if (m < 60) return `${m}m ago`;
  const h = Math.floor(m / 60);
  if (h < 24) return `${h}h ago`;
  const days = Math.floor(h / 24);
  if (days < 30) return `${days}d ago`;
  return fmtDate(d);
}

export function humanize(s: string) {
  return s.toLowerCase().split("_").map((w) => w[0]?.toUpperCase() + w.slice(1)).join(" ");
}

export function initials(first: string, last?: string) {
  return ((first?.[0] || "") + (last?.[0] || "")).toUpperCase();
}

export function formatDuration(ms: number) {
  const totalMin = Math.floor(ms / 60000);
  const h = Math.floor(totalMin / 60);
  const m = totalMin % 60;
  return `${h}h ${m.toString().padStart(2, "0")}m`;
}
```

src/lib/forms.ts

```
/** Small helpers for reading FormData in server actions. */
export const str = (f: FormData, k: string) => {
  const v = f.get(k);
  return typeof v === "string" && v.trim() !== "" ? v.trim() : null;
};
export const reqStr = (f: FormData, k: string) => str(f, k) ?? "";
export const num = (f: FormData, k: string, d = 0) => {
  const v = str(f, k);
```

```
  const n = v === null ? NaN : Number(v);
  return Number.isFinite(n) ? n : d;
};
export const intOrNull = (f: FormData, k: string) => {
  const v = str(f, k);
  const n = v === null ? NaN : parseInt(v, 10);
  return Number.isFinite(n) ? n : null;
};
export const dateOrNull = (f: FormData, k: string) => {
  const v = str(f, k);
  return v ? new Date(v) : null;
};
export const bool = (f: FormData, k: string) => f.get(k) === "on" || f.get(k) === "true" || f.get(k) === "1";
export const ints = (f: FormData, k: string) =>
  f.getAll(k).map((v) => parseInt(String(v), 10)).filter((n) => Number.isFinite(n));

export type ActionState = { error?: string; ok?: boolean } | undefined;
```

src/lib/permissions.ts

```
export const MODULES = [
  { key: "customers", label: "Customers" },
  { key: "leads", label: "Leads" },
  { key: "invoices", label: "Invoices" },
  { key: "estimates", label: "Estimates" },
  { key: "payments", label: "Payments" },
  { key: "expenses", label: "Expenses" },
  { key: "projects", label: "Projects" },
  { key: "tasks", label: "Tasks" },
  { key: "tickets", label: "Support" },
  { key: "reports", label: "Reports" },
  { key: "staff", label: "Staff" },
  { key: "settings", label: "Settings" },
] as const;

export type ModuleKey = (typeof MODULES)[number]["key"];
export const ACTIONS = ["view", "create", "edit", "delete"] as const;
export type Action = (typeof ACTIONS)[number];
export type PermissionMap = Partial<Record<ModuleKey, Action[]>>;

export function parsePermissions(json?: string | null): PermissionMap {
  try {
    return json ? JSON.parse(json) : {};
  } catch {
    return {};
  }
}

export type PermUser = { isAdmin: boolean; permissions: PermissionMap };

export function can(user: PermUser | null | undefined, module: ModuleKey, action: Action = "view") {
  if (!user) return false;
  if (user.isAdmin) return true;
  return !!user.permissions[module]?.includes(action);
}
```

src/lib/sales.ts

```
import "server-only";
import { randomBytes } from "crypto";
import { db } from "./db";
import { round2 } from "./format";

export type LineInput = {
  description: string;
  longDescription?: string | null;
  qty: number;
  rate: number;
  unit?: string | null;
  taxRate: number;
};
```

```
export function calcTotals(items: LineInput[], discountPercent: number, adjustment: number) {
  const subtotal = round2(items.reduce((s, i) => s + i.qty * i.rate, 0));
  const discountTotal = round2((subtotal * (discountPercent || 0)) / 100);
  const factor = subtotal > 0 ? (subtotal - discountTotal) / subtotal : 1;
  const taxTotal = round2(items.reduce((s, i) => s + i.qty * i.rate * factor * (i.taxRate / 100), 0));
  const total = round2(subtotal - discountTotal + taxTotal + (adjustment || 0));
  return { subtotal, discountTotal, taxTotal, total };
}

/** Parse line items posted from <LineItemsEditor/> (hidden JSON field "items"). */
export function parseItems(f: FormData): LineInput[] {
  try {
    const raw = JSON.parse(String(f.get("items") || "[]")) as LineInput[];
    return raw
      .filter((i) => i.description && String(i.description).trim())
      .map((i) => ({
        description: String(i.description).trim(),
        longDescription: i.longDescription ? String(i.longDescription) : null,
        qty: Number(i.qty) || 0,
        rate: Number(i.rate) || 0,
        unit: i.unit ? String(i.unit) : null,
        taxRate: Number(i.taxRate) || 0,
      }));
  } catch {
    return [];
  }
}

export const newHash = () => randomBytes(16).toString("hex");

export async function nextInvoiceNumber() {
  const last = await db.invoice.findFirst({ orderBy: { number: "desc" }, select: { number: true } });
  return (last?.number ?? 0) + 1;
}
export async function nextEstimateNumber() {
  const last = await db.estimate.findFirst({ orderBy: { number: "desc" }, select: { number: true } });
  return (last?.number ?? 0) + 1;
}

export const docNumber = (prefix: string, n: number) => `${prefix}${String(n).padStart(6, "0")}`;

/** Recompute invoice status from payments (keeps DRAFT/CANCELLED). */
export async function refreshInvoiceStatus(invoiceId: number) {
  const inv = await db.invoice.findUnique({ where: { id: invoiceId }, include: { payments: true } });
  if (!inv || inv.status === "CANCELLED") return;
  const paid = round2(inv.payments.reduce((s, p) => s + p.amount, 0));
  let status = inv.status;
  if (paid >= inv.total && inv.total > 0) status = "PAID";
  else if (paid > 0) status = "PARTIAL";
  else if (inv.status !== "DRAFT") status = "UNPAID";
  if (status !== inv.status) await db.invoice.update({ where: { id: invoiceId }, data: { status } });
}

export function isOverdue(inv: { status: string; dueDate: Date | null }) {
  return (inv.status === "UNPAID" || inv.status === "PARTIAL") && !!inv.dueDate && inv.dueDate < new Date(new
Date().toDateString());
}
```

src/lib/session.ts

```
import { SignJWT, jwtVerify } from "jose";

export const SESSION_COOKIE = "sq_session";
const secret = () => new TextEncoder().encode(process.env.AUTH_SECRET || "dev-secret-change-me");

export async function signSession(staffId: number) {
  return new SignJWT({ sid: staffId })
    .setProtectedHeader({ alg: "HS256" })
    .setIssuedAt()
    .setExpirationTime("7d")
    .sign(secret());
}
```

```ts
export async function verifySession(token?: string): Promise<number | null> {
  if (!token) return null;
  try {
    const { payload } = await jwtVerify(token, secret());
    return typeof payload.sid === "number" ? payload.sid : null;
  } catch {
    return null;
  }
}
```

src/lib/settings.ts

```ts
import "server-only";
import { db } from "./db";

export const DEFAULT_SETTINGS = {
  company_name: "Squareit",
  company_tagline: "Say Digital, Way Digital",
  company_email: "",
  company_phone: "",
  company_address: "",
  company_city: "",
  company_state: "",
  company_zip: "",
  company_country: "India",
  company_vat: "",
  currency: "INR",
  invoice_prefix: "INV-",
  invoice_due_days: "15",
  invoice_terms: "",
  invoice_client_note: "Thank you for your business.",
  estimate_prefix: "EST-",
  estimate_expiry_days: "15",
  estimate_terms: "",
  bank_details: "",
};
export type SettingKey = keyof typeof DEFAULT_SETTINGS;
export type Settings = Record<SettingKey, string>;

export async function getSettings(): Promise<Settings> {
  const rows = await db.setting.findMany();
  const s = { ...DEFAULT_SETTINGS } as Settings;
  for (const r of rows) if (r.key in s) s[r.key as SettingKey] = r.value;
  return s;
}

export async function saveSettings(values: Partial<Settings>) {
  await db.$transaction(
    Object.entries(values).map(([key, value]) =>
      db.setting.upsert({ where: { key }, create: { key, value: value ?? "" }, update: { value: value ?? "" }
})
    )
  );
}
```

src/middleware.ts

```ts
import { NextResponse, type NextRequest } from "next/server";
import { SESSION_COOKIE, verifySession } from "@/lib/session";

export async function middleware(req: NextRequest) {
  const id = await verifySession(req.cookies.get(SESSION_COOKIE)?.value);
  if (!id) {
    const url = new URL("/login", req.url);
    url.searchParams.set("next", req.nextUrl.pathname);
    return NextResponse.redirect(url);
  }
  return NextResponse.next();
}

export const config = { matcher: ["/admin/:path*"] };
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": ["./src/*"] }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```